MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND
COMMUNICATION TECHNOLOGY
DEPARTMENT OF SOFTWARE ENGINEERING

GRADUATION THESIS
BACHELOR OF ENGINEERING IN
INFORMATION TECHNOLOGY

(HIGH-QUALITY PROGRAM)

BUILDING A FASHION E-COMMERCE APPLICATION WITH IMAGE-BASED PRODUCT SEARCH AND MODEL BENCHMARKING

Student: Nguyen Thanh Phat
Student ID: B2005853
Class: DI20V7F1 (K46)
Advisor: Dr. Tran Cong An

Can Tho, December 2025

MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY
DEPARTMENT OF SOFTWARE ENGINEERING

GRADUATION THESIS
BACHELOR OF ENGINEERING IN INFORMATION TECHNOLOGY

(HIGH-QUALITY PROGRAM)

BUILDING A FASHION E-COMMERCE APPLICATION WITH IMAGE-BASED PRODUCT SEARCH AND MODEL BENCHMARKING

Student: Nguyen Thanh Phat
Student ID: B2005853
Class: DI20V7F1 (K46)
Advisor: Dr. Tran Cong An

Can Tho, 01/2026

EVALUATION OF ADVISOR

.....

.....

.....

.....

.....

.....

.....

Advisor:

Dr. Tran Cong An

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to Dr. Tran Cong An, my thesis advisor, for his guidance, continuous feedback, and support throughout this research.

I also thank the members of the Thesis Defense Committee for their time, thorough evaluation, and constructive comments.

My sincere appreciation goes to the Faculty of Information Technology, Can Tho University, for providing the academic foundation that supported this work, and to the High-Quality Program for fostering the engineering discipline applied throughout this thesis.

Finally, I am deeply grateful to my family for their constant encouragement, patience, and unwavering support throughout my studies and the completion of this work.

Sincerely,

Can Tho, December 2025

Nguyen Thanh Phat

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
ANN	Approximate Nearest Neighbor
CBIR	Content-Based Image Retrieval
CNN	Convolutional Neural Network
CQRS	Command Query Responsibility Segregation
DDD	Domain-Driven Design
DSR	Design Science Research
EF Core	Entity Framework Core
HNSW	Hierarchical Navigable Small World
JWT	JSON Web Token
mAP	Mean Average Precision
P@K	Precision at K
R@K	Recall at K
REST	Representational State Transfer
SPA	Single Page Application
ViT	Vision Transformer
VSA	Vertical Slice Architecture

ABSTRACT

E-commerce platforms rely primarily on text-based search, yet fashion products are inherently visual. Patterns, silhouettes, and textures resist keyword description. This thesis presents a fashion e-commerce platform with integrated Content-Based Image Retrieval (CBIR) that enables customers to search for products by uploading images rather than typing keywords. The system implements a modular architecture with a .NET backend, Vue.js frontend, and a Python machine learning sidecar for embedding generation.

A systematic benchmark evaluates four pre-trained deep learning models across both retrieval accuracy and operational efficiency on a curated fashion product dataset. Fashion-CLIP, a CLIP variant fine-tuned on over 700,000 fashion images, achieved the highest mean Average Precision at 0.8788 (SD 0.0022), outperforming the generic CLIP (mAP 0.8341, SD 0.0043) by 5.4%, EfficientNet-B0 (mAP 0.8158, SD 0.0007) by 7.7%, and ResNet-50 (mAP 0.8120, SD 0.0052) by 8.2%. At the opposite end of the speed spectrum, EfficientNet-B0 delivered inference at 23.9 ms per image (SD 2.5) while maintaining competitive retrieval quality, representing a 3.8x speed advantage over Fashion-CLIP (92.0 ms, SD 5.8) for a 7.7% accuracy trade-off.

Results demonstrate that domain-specific pre-training provides measurable advantages for visual fashion retrieval. The pluggable model architecture, switchable via a single environment variable, allows production deployments to select the optimal accuracy-speed profile for their infrastructure. The thesis provides a reference implementation for systematic comparison of embedding models in the fashion domain.

Keywords: e-commerce, visual search, deep learning, modular architecture, computer vision, benchmarking

TÓM TẮT

Các sàn thương mại điện tử phụ thuộc nhiều vào tìm kiếm văn bản, một cơ chế kém hiệu quả trong lĩnh vực thời trang, nơi hoa văn, chất liệu và kiểu dáng khó diễn đạt chính xác qua từ khóa. Hệ thống được xây dựng theo hướng module, bao gồm ba thành phần chính: backend .NET xử lý logic nghiệp vụ, frontend Vue.js phục vụ giao diện người dùng, và dịch vụ Python chuyên biệt cho tác vụ trích xuất đặc trưng hình ảnh từ các mô hình học sâu tiền huấn luyện.

Thực nghiệm được thiết lập trên bộ dữ liệu ảnh sản phẩm thời trang với quy trình đánh giá chéo ba lần (3-fold cross-validation), so sánh bốn mô hình embedding trên hai chiều: chất lượng truy xuất (đo bằng mAP, Precision at K, Recall at K) và hiệu năng vận hành (đo bằng độ trễ suy luận, thông lượng, và dung lượng lưu trữ). Mô hình Fashion-CLIP, được tinh chỉnh trên tập dữ liệu hơn 700.000 ảnh thời trang, đạt mAP cao nhất 0,8788 với độ lệch chuẩn 0,0022. CLIP gốc đạt mAP 0,8341 (SD 0,0043) và EfficientNet-B0 đạt 0,8158 (SD 0,0007). Ở chiều ngược lại, mô hình EfficientNet-B0 cho tốc độ suy luận nhanh nhất, 23,9 ms mỗi ảnh (SD 2,5), nhanh gấp 3,8 lần Fashion-CLIP (92,0 ms, SD 5,8) trong khi chỉ đánh đổi 7,7% về chất lượng truy xuất.

Kết quả thực nghiệm khẳng định giá trị của huấn luyện chuyên biệt theo lĩnh vực trong bài toán truy xuất hình ảnh thời trang, đồng thời cung cấp dữ liệu định lượng cho việc lựa chọn mô hình dựa trên ràng buộc hạ tầng thực tế. Kiến trúc mô hình hoán đổi linh hoạt, được điều khiển qua biến môi trường, cho phép hệ thống thích ứng với nhiều cấu hình triển khai khác nhau mà không cần thay đổi mã nguồn.

Từ khóa: thương mại điện tử, tìm kiếm hình ảnh, học sâu, kiến trúc module, thị giác máy tính, đánh giá hiệu năng

PART 1: INTRODUCTION

I. CONTEXT AND MOTIVATION

Global fashion e-commerce revenue exceeded **770 billion USD** in 2024, with projections surpassing **one trillion by 2030** [1]. Yet its dominant interface, keyword search, fails where the domain succeeds: fashion products are defined by silhouette, drape, print density, and colour relationships – attributes that resist textual description. This **semantic gap**, the discrepancy between a garment's visual richness and a user's ability to express it in words, is well documented. A customer can recognise a desired aesthetic instantly from a photograph yet cannot produce query terms that retrieve it. When catalogue indexing uses inconsistent terminology (one vendor tags a pattern as "floral," another as "botanical," a third as "flower print"), relevant products are systematically excluded. Industry estimates place the **session abandonment rate** after an unsuccessful search at approximately **30 percent** [2].

Content-Based Image Retrieval (CBIR) addresses this gap by replacing textual intermediaries with direct visual comparison. Products are indexed not by human-authored labels but by **dense vector embeddings** computed from images, with similarity measured through mathematical distance functions. A query image of a dress with a particular neckline and print pattern retrieves visually similar products without any keyword translation step. Pre-trained convolutional neural networks [3], [4], vision transformers [5], and fashion-specific models [6] have substantially advanced this capability.

The contribution of this work is **architectural**, not algorithmic. It investigates how to embed existing CBIR capabilities into a practical e-commerce system built with conventional web technologies, and provides empirical data on which embedding models deliver the optimal balance of accuracy, latency, and resource efficiency. The work bridges two distinct software ecosystems, the **Python machine learning stack** and the **.NET enterprise web stack**, under real-time latency constraints appropriate for interactive search.

II. PROBLEM STATEMENT

Keyword-reliant fashion search suffers from four compounding inefficiencies.

Catalogue vocabulary mismatch. Descriptors vary across vendors, such that a single visual pattern appears under multiple labels and fragments result sets. Users must reformulate queries iteratively as the gap between catalogue

indexing vocabulary and customer search vocabulary silently excludes relevant products.

Visual inexpressibility. Attributes such as fabric drape, texture gradient, silhouette proportion, and pattern rhythm cannot be captured reliably through text queries. A customer identifies a desired aesthetic instantly in a photograph yet cannot produce keywords that retrieve it. The search engine cannot match what the user cannot name.

Cold-start invisibility. Recommendation models based on collaborative filtering depend on historical user-item interactions. Newly listed products lack this data at their point of highest commercial value: initial release. Visual feature extraction bypasses this limitation, as product images are available from catalogue ingestion and embeddings enable similarity-based discovery without interaction history.

Polyglot integration cost. Integrating pre-trained vision models into a .NET transactional backend introduces a recurring engineering challenge in applied machine learning. The Python deep learning ecosystem (PyTorch, HuggingFace) does not natively interoperate with the .NET enterprise stack. Achieving **sub-second response latency** across this boundary requires architectural design that isolates the ML workload from the main application while bridging incompatible package managers, runtime environments, and deployment conventions.

III. OBJECTIVES

This project builds a functional fashion e-commerce platform with integrated image-based search and evaluates the effectiveness of pre-trained deep learning models within that system. The contribution is not a novel AI architecture but the **engineering demonstration** of embedding existing models into a conventional web application stack.

Technical Objectives

- **Model integration.** Integrate pre-trained vision models into a PostgreSQL and .NET e-commerce stack, establishing a reference pattern for teams with existing web infrastructure.
- **Polyglot architecture.** Architect a polyglot system in which a dedicated Python sidecar handles AI inference while the .NET backend manages transactional logic, business rules, and API routing.
- **Vector storage validation.** Validate **pgvector** (an open-source PostgreSQL extension) as the sole vector storage and retrieval layer, evaluating whether it meets real-time search latency requirements at catalogue scales representative of small-to-medium fashion retailers.

- **Empirical benchmarking.** Benchmark multiple embedding models spanning convolutional and transformer architectures on shared hardware, producing empirical guidance for model selection in resource-constrained deployments.

Research Questions

Three questions guide the investigation and are answered empirically in Chapter 3.

RQ1: Model comparison. How do fashion-specific embedding models compare with general-purpose CNN and ViT architectures on fashion product retrieval? This question tests whether domain-specific training on fashion data yields measurable improvements over models trained on generic image corpora.

RQ2: Accuracy-speed trade-off. What trade-offs exist between retrieval accuracy and inference latency across pre-trained embedding models, and which model offers the best balance for real-time search? The most accurate model is rarely the fastest; deployment decisions require weighing both dimensions.

RQ3: Architecture viability. Can a service-oriented architecture with a dedicated AI sidecar separate image inference from the main application while maintaining interactive response times? This question evaluates whether the chosen polyglot pattern (Python ML service alongside a .NET application) is practical for production use.

Tasks Completed

- **Build an AI service.** A Python FastAPI service that loads multiple pre-trained embedding models and generates feature vectors from product images within interactive latency bounds.
- **Set up vector search.** PostgreSQL configured with pgvector to store and index high-dimensional embeddings, with similarity queries validated for correctness and performance on a catalogue of representative size.
- **Connect the services.** A .NET backend layer that orchestrates image upload, embedding generation, vector database query, and result assembly into a single end-to-end search flow.
- **Create the user interface.** A Vue.js storefront with drag-and-drop image upload and a results grid displaying visually similar products with similarity scores.
- **Evaluate the results.** A systematic benchmark measuring retrieval accuracy via standard information retrieval metrics, comparing inference speed across models, and analyzing operational trade-offs that inform production model selection.

IV. SCOPE AND LIMITATIONS

The thesis encompasses the design, implementation, and empirical evaluation of a fashion e-commerce platform with integrated visual search. Five areas define the included scope:

- Visual search from the storefront, with image upload via drag-and-drop or file selection.
- Product recommendations derived from embedding similarity scores.
- Core e-commerce functionality: product catalogue browsing, shopping cart, and simulated checkout.
- Multi-model comparison spanning several CNN and transformer architectures.
- End-to-end latency, throughput, and storage footprint measurement.

The following areas are explicitly excluded:

- Real payment processing (transactions are simulated for demonstration purposes).
- Shipping, logistics, and warehouse management workflows.
- User authentication via social login providers.
- Mobile application development (the system targets desktop and tablet web browsers).
- Custom model training or fine-tuning (all models are used as published).

Known Limitations

Four limitations define the boundaries of this work and are revisited in the concluding chapter.

- **Dataset size.** Evaluation uses 5,000 fashion product images from the Fashion Product Images dataset [7]. Controlled comparative benchmarking is feasible at this scale, but results may not extrapolate to production catalogues containing millions of items.
- **Hardware.** Experiments ran on consumer-grade hardware (Intel i7-1165G7, 16 GB RAM) with all inference executed on CPU. Reported latency and throughput figures are relative to this CPU-only profile. A dedicated inference server with GPU acceleration would likely improve both metrics.
- **Evaluation method.** Evaluation is exclusively quantitative: retrieval accuracy, inference latency, and throughput. Search output was reviewed qualitatively through visual inspection, but no formal user experience study was conducted. The relationship between measured accuracy metrics and subjective user satisfaction remains an open question.
- **Model training.** All embedding models were used as published by their original authors, without fine-tuning on the evaluation dataset. Domain-specific

fine-tuning, particularly for models pre-trained on generic image corpora rather than fashion data, might improve retrieval quality but was beyond scope.

V. RESEARCH METHODOLOGY

This thesis follows a **Design Science Research (DSR)** methodology [8], [9], a problem-solving paradigm that produces and evaluates an IT artifact (here, the e-commerce platform with integrated visual search) to address a defined problem domain.

The project progressed through four phases:

- **Research and planning.** Literature survey on visual search in fashion and e-commerce architectures; exploration of available pre-trained embedding models; evaluation of vector database options; technology stack selection.
- **Design.** Formalization of a modular .NET monolith architecture with a Python AI sidecar communicating via HTTP; database schema design supporting both relational and vector data within a single PostgreSQL instance.
- **Implementation.** Construction of three principal components: the .NET backend following vertical slice architecture, the Python embedding service with lazy model loading, and the Vue.js storefront.
- **Test and evaluation.** Systematic benchmark measuring retrieval accuracy through standard information retrieval metrics; latency and throughput measurement on consumer-grade hardware; qualitative review of search output.

This methodology suits the project's engineering focus: the primary output is not a theoretical contribution but a working system accompanied by empirical data on performance trade-offs practitioners encounter when integrating academic models into production web stacks.

VI. THESIS OUTLINE

The thesis is organized into five chapters across three parts.

Part I: Introduction. Chapter 1 establishes research context, defines the problem, states objectives and research questions, delineates scope and limitations, describes the methodology, and provides the present outline.

Part II: Thesis Content contains three chapters:

- **Chapter 1: Background and Related Work.** Surveys vector embeddings, convolutional and transformer-based neural architectures, vector database technologies including pgvector, prior work in fashion image retrieval, and the technology stack selected for this project.
- **Chapter 2: Design and Implementation.** Translates the problem into functional and non-functional requirements (Section 2.1), presents the system

architecture including domain-driven design, C4 diagrams, database design, API design, and security model (Section 2.2), and describes the concrete implementation of the .NET backend, Python ML sidecar, and Vue.js storefront (Section 2.3).

- **Chapter 3: Testing and Evaluation.** Reports a systematic benchmark comparing retrieval accuracy and inference efficiency across multiple embedding models using a cross-validation protocol on 5,000 fashion product images, and analyzes the accuracy-speed trade-offs that inform model selection.

Part III: Conclusion. Chapter 4 synthesizes findings against each research objective, evaluates contributions and limitations, and proposes directions for future work.

PART 2: THESIS CONTENT

1 BACKGROUND AND RELATED WORK

This chapter surveys the theoretical foundations and technical prerequisites for the project.

- **Fashion E-commerce.** Market context, semantic gap, business case.
- **Content-Based Image Retrieval.** Embeddings, cosine similarity, CBIR pipeline.
- **Machine Learning Models.** CNN, ViT, CLIP, Fashion-CLIP architectures.
- **Vector Databases.** ANN search, HNSW, IVFFlat, pgvector.
- **Platform Architecture and Technology Stack.** Modular monolith, vertical slice, .NET, Vue, PostgreSQL, Redis, Python sidecar, orchestration, auth, benchmarks.
- **Related Work and Research Gap.** Academic research, commercial systems, contribution differentiators.

1.1 FASHION E-COMMERCE

Global fashion e-commerce reached **770 billion USD** in 2024 and is projected to surpass **one trillion USD by 2030** [1]. However, traditional text search bars cause a **30 percent search abandonment rate** because keyword queries struggle to match visual intent [2].

Fashion is fundamentally visual. Attributes like silhouette, drape, color harmony, and pattern density do not translate easily into search terms. A customer can easily recognize a garment in an image but often fails to find it using keywords.

Major platforms like ASOS, Zalando, and Shopee invest in visual search because millions of catalog items rely on inconsistent vendor metadata. Identical patterns often carry different labels across suppliers, hiding relevant products from keyword search. Visual search solves this metadata problem, directly preventing lost revenue from search abandonment.

1.2 CONTENT-BASED IMAGE RETRIEVAL

Content-Based Image Retrieval replaces text queries with image queries. Rather than indexing products by human-authored labels, CBIR systems encode images into dense vector representations and measure similarity through mathematical

distance functions. This section introduces the core concepts and mathematical foundations of CBIR as applied to fashion product search.

1.2.1 Visual Search Concepts

Content-Based Image Retrieval (CBIR) replaces text queries with image queries. Rather than requiring users to describe what they want in words, CBIR lets them search by example.

The system encodes a query image into a **dense vector embedding**: a fixed-length sequence of numbers that captures shape, texture, colour, and pattern. It then retrieves catalog items whose embeddings are nearest in vector space. Visually similar products produce similar vectors; dissimilar products produce distant ones.

This approach bypasses the need for consistent textual labels. A photograph of a dress with a distinctive neckline retrieves visually similar products regardless of how the catalog describes them.

1.2.2 The Semantic Gap

The semantic gap, introduced in Section 1.1, is the discrepancy between the visual richness of a garment and a user's ability to express that richness in keywords. CBIR bridges this gap by operating directly on visual content rather than on human-authored metadata. The embedding serves as a universal descriptor that captures attributes such as fabric texture, silhouette proportion, and colour gradients automatically, without any keyword translation step.

1.2.3 Mathematical Foundations of Embeddings

An embedding model processes an input image and transforms its visual content into a fixed-length numerical array:

$$\boldsymbol{e} = [e_1, e_2, e_3, ..., e_d]^T \in \mathbb{R}^d$$

For a model with a 512-dimensional output, this vector $\boldsymbol{e}$ contains 512 continuous numbers. These arrays are called **vector embeddings**. Visually similar items yield nearby numerical values across these dimensions, converting visual comparison into a standard mathematical distance calculation.

1.2.3.1 The Latent Space

Embeddings act as coordinates within a high-dimensional continuous space known as the **latent space**. A 512-dimensional vector occupies a coordinate system with 512 orthogonal axes.

Within this space:

- Visually and semantically similar items sit close to one another.
- Dissimilar items lie further apart.
- The term "latent" indicates that individual axes rarely map directly to single human visual labels like "stripes" or "red." Instead, each dimension captures abstract, non-linear feature combinations learned by the model during training.

1.2.3.2 Measuring Similarity: Cosine Similarity

To evaluate how closely two images match, the system measures the directional angle between their corresponding embedding vectors $\boldsymbol{A}$ and $\boldsymbol{B}$ using **cosine similarity**:

$$\text{Cosine Similarity}(\boldsymbol{A}, \boldsymbol{B}) = \frac{\boldsymbol{A} \cdot \boldsymbol{B}}{\|\boldsymbol{A}\|_2 \times \|\boldsymbol{B}\|_2}$$

Where $\boldsymbol{A} \cdot \boldsymbol{B}$ represents the vector dot product, and $\|\boldsymbol{A}\|_2$ and $\|\boldsymbol{B}\|_2$ denote the Euclidean norms (L_2 norms) of each vector.

Cosine similarity produces values ranging from $+1.0$ (identical vector orientation) to 0.0 (orthogonal vectors) down to -1.0 (opposite orientations). For normalized fashion embeddings, scores above 0.70 generally correspond to strong visual similarity perceptible to human shoppers.

1.2.3.3 The CBIR Pipeline

A standard Content-Based Image Retrieval system operates through four core sequential stages:

1. **Image Input:** The user uploads or selects a target query photograph.
2. **Embedding Generation:** A deep learning model processes the image to output its feature vector.
3. **Vector Comparison:** The database compares the query vector against pre-indexed catalog vectors using cosine distance or cosine similarity.
4. **Ranking & Retrieval:** The system orders products by their similarity scores and renders the top nearest neighbors to the user.

1.3 MACHINE LEARNING MODELS

This section surveys the three families of architectures used for visual feature extraction: convolutional neural networks, vision transformers, and multimodal CLIP-based models. Each subsection explains how the architecture works, introduces the specific variants evaluated, and identifies their trade-offs for fashion retrieval. The section concludes with the model selection decision.

1.3.1 Convolutional Neural Networks

Convolutional neural networks (CNNs) have been the dominant architecture for computer vision. This section explains how CNNs extract features from fashion

images and introduces ResNet and EfficientNet, the two CNN families evaluated in the benchmark.

1.3.1.1 Hierarchical Feature Extraction

A CNN processes an image through a stack of learned filters. Each filter is a small window (typically 3 by 3 pixels) that slides across the image detecting local patterns [3]. This process builds increasingly complex representations.

Early layers detect simple edges, colours, and corners. Middle layers combine these into textures, shapes, and garment parts such as lapels and sleeves. Late layers recognize high-level features: garment categories, styles, and overall silhouettes. This hierarchical organization gives CNNs an **inductive bias** toward local patterns. They excel at detecting texture and colour but may miss relationships between distant image regions, such as matching the collar and hem of a dress when they are far apart.

1.3.1.2 ResNet and Skip Connections

Deeper networks capture richer features but suffer from **vanishing gradients**: training signals decay as they propagate backward through many layers. ResNet (Residual Network) solves this with **skip connections**: identity paths that bypass convolutional blocks and add the block input directly to its output [3]. This identity path lets gradients flow unimpeded, enabling networks of 50, 101, or 152 layers to train effectively.

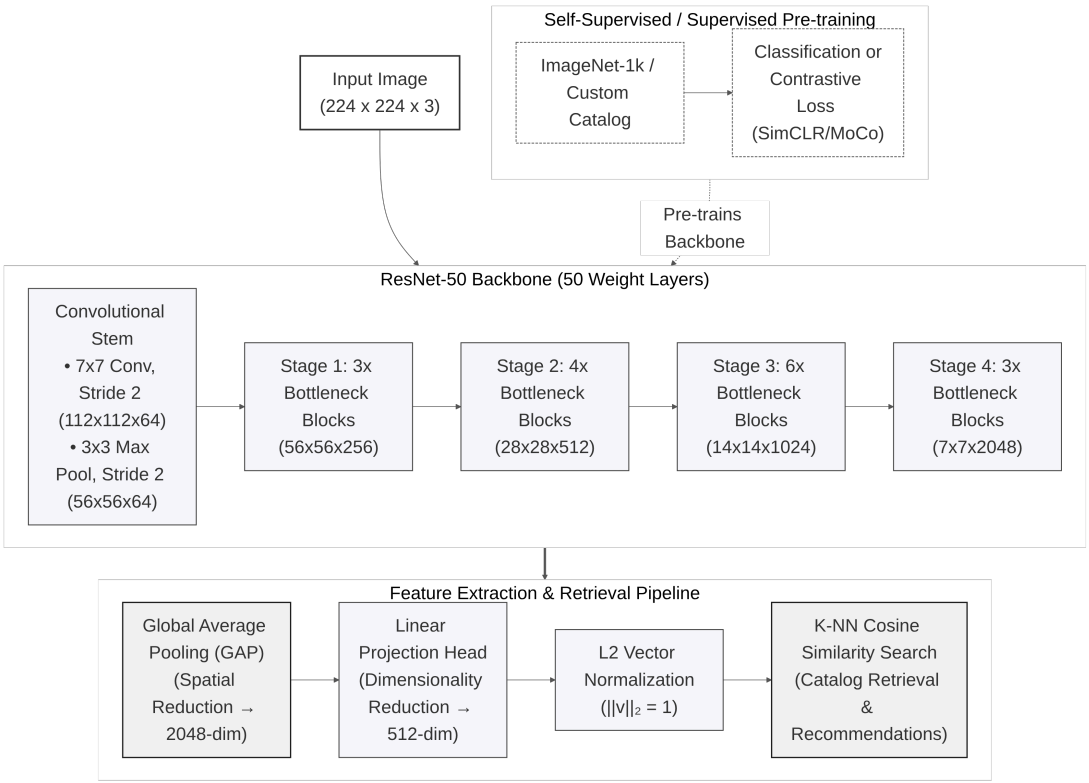

Figure 1: ResNet architecture with residual skip connections enabling gradient flow across very deep networks

ResNet-50 contains 25.6 million parameters and produces 2,048-dimensional embeddings. ResNet-101, with 44.5 million parameters and the same embedding dimension, provides additional depth for comparison.

1.3.1.3 EfficientNet and Compound Scaling

Traditional scaling strategies enlarge a network along a single dimension: depth (more layers), width (more channels), or resolution (larger inputs). EfficientNet introduces **compound scaling**, which balances all three dimensions simultaneously using a learned coefficient [4]. This produces a family of models (B0 through B7) that achieve competitive accuracy with far fewer parameters than conventional scaling.

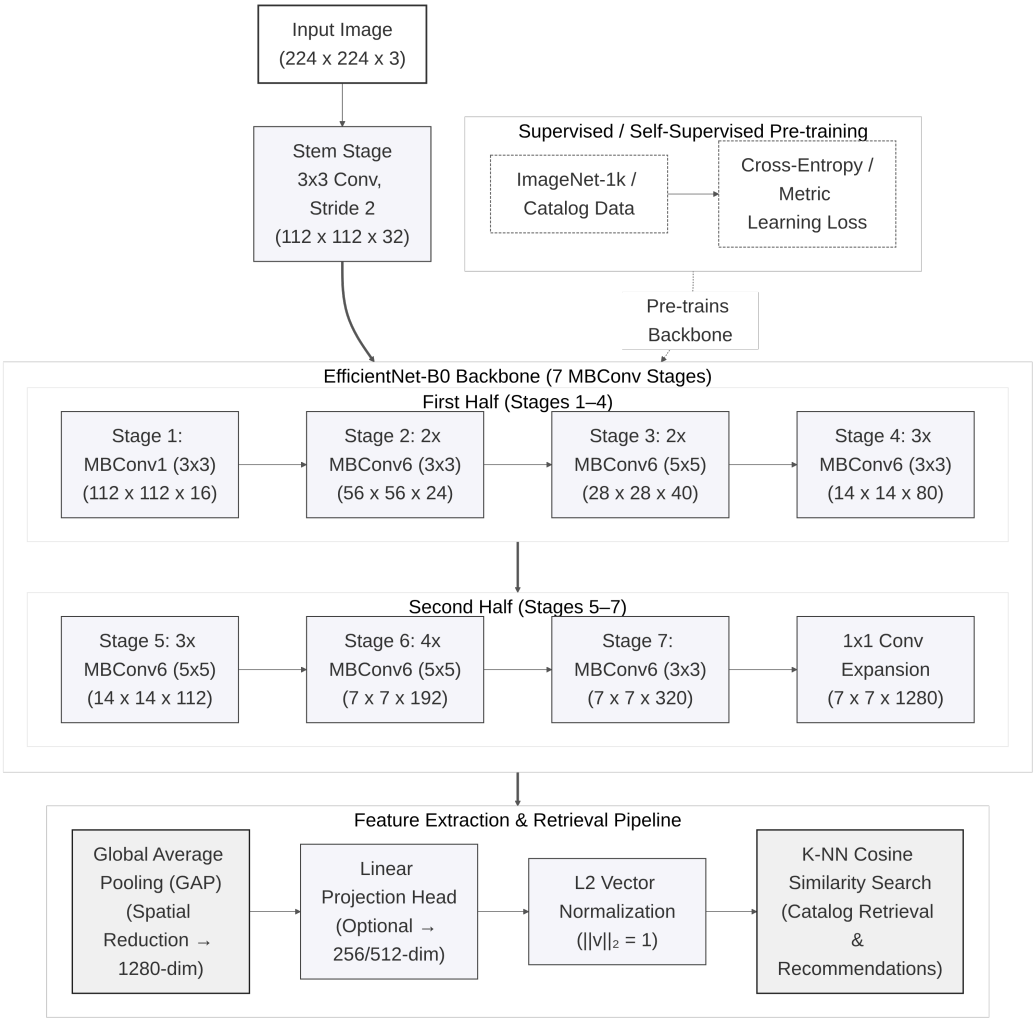

Figure 2: EfficientNet-B0 architecture showing the flow from input image to feature vector

EfficientNet-B0, the smallest variant, uses 5.3 million parameters and produces 1,280-dimensional embeddings. Its compact design makes it well suited to CPU-only deployments. EfficientNet-B4 (19.3 million parameters, 1,792-dimensional embeddings) provides higher capacity at increased computational cost.

1.3.1.4 Evaluated CNN Variants

Table 1: CNN-based models evaluated.

Family	Model	Parameters	Embedding Dim	Training Data
CNN	ResNet-50	25.6M	2048	ImageNet (1.2M images)
CNN	ResNet-101	44.5M	2048	ImageNet (1.2M images)
CNN	EfficientNet-B0	5.3M	1280	ImageNet (1.2M images)
CNN	EfficientNet-B4	19.3M	1792	ImageNet (1.2M images)

While CNNs excel at detecting local patterns through their convolutional layers, they have limitations in capturing long-range dependencies and global context. This motivated researchers to explore alternative architectures: vision transformers.

1.3.2 Vision Transformers

Vision Transformers (ViTs) apply the transformer architecture, originally developed for natural language processing, to images. This section explains how ViTs work and introduces DINOv2, a self-supervised model evaluated in this project.

1.3.2.1 Patch Embedding and Tokenization

Transformers were originally developed for NLP tasks such as translation and text generation. The key innovation was **self-attention**, which allows the model to consider relationships between all parts of the input simultaneously.

In 2020, researchers showed this approach could also work for images [10]. The method splits an image into a grid of fixed-size patches, typically 14 by 14 or 16 by 16 pixels. Each patch is flattened into a vector and treated as a token, analogous to a word in a sentence. Position encodings are added to preserve spatial information, and the sequence passes through transformer layers with multi-head self-attention.

1.3.2.2 Global Context via Self-Attention

Unlike CNNs, which focus on local patterns, self-attention captures relationships across the entire image from the first layer. A CNN processes patches in order and mainly compares nearby regions. A ViT can directly compare any two patches, even if they are far apart.

For fashion, this means a ViT can better understand that the collar of a shirt and the cuffs should match, even though they are on opposite sides of the image. This capability is valuable for garment retrieval, where silhouette and drape matter as much as local texture.

1.3.2.3 DINOv2 and Self-Supervised Pre-Training

Most AI models are trained with supervised learning, where humans label images and the model learns from those labels. This requires expensive manual annotation.

Self-supervised learning takes a different approach: the model learns from the images themselves, without human labels. DINOv2 uses a student-teacher self-distillation framework [11]. An image is transformed into two different views with different crops and colour variations. Both pass through student and teacher networks with identical architecture. The student is trained to match the teacher's output, and the teacher is updated as an exponential moving average of the student weights. This process repeats across 142 million uncurated images.

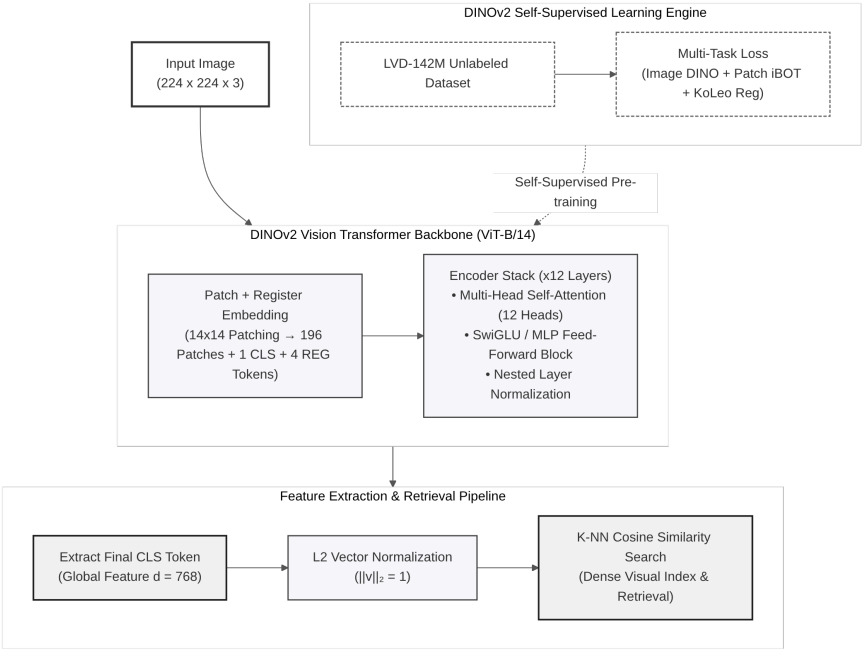

Figure 3: DINOv2 architecture: image patches pass through transformer layers with self-attention to produce a feature vector

DINOv2 produces features that exhibit strong object-level structure: silhouettes, part geometry, and garment boundaries, without being trained on category labels. This makes it adaptable to fashion domains where curated labels are scarce.

1.3.2.4 Structural Fidelity for Fashion Retrieval

DINOv2 is particularly good at capturing **structural fidelity**: the shapes, silhouettes, and proportions of objects. For fashion, this means matching garments with similar cuts (A-line, fitted, oversized), finding items with similar proportions (cropped vs. full-length), and ignoring colour differences when the shape is similar. This is valuable for users who might want a dress shaped like a given example, but in a different colour.

1.3.2.5 DINOv2 Model Specifications

Table 2: DINOv2 model specifications evaluated in this project

Property	**DINOv2 ViT-S/14**	**DINOv2 ViT-B/14**
Architecture	Vision Transformer (Small)	Vision Transformer (Base)
Parameters	21M	86M
Patch size	14 by 14 pixels	14 by 14 pixels
Embedding dimension	384	768
Training data	142M images	142M images
Training method	Self-supervised	Self-supervised

1.3.2.6 Trade-offs and Limitations

Vision Transformers have different trade-offs compared to CNNs. They are better at understanding global structure and long-range relationships, and their self-supervised training makes them potentially more general. Their structural fidelity captures shapes and silhouettes effectively.

However, they are slower than CNNs due to greater computational requirements, need larger input images for best results, and may be excessive for simple colour and pattern matching.

Both CNNs and Vision Transformers learn from images alone, mapping visual features to embedding vectors. However, fashion search often involves natural language queries like “red floral summer dress” or “casual weekend outfit.” This requires models that can understand both images and text. The next section introduces CLIP and Fashion-CLIP, which bridge this gap through multimodal learning.

1.3.3 CLIP and Fashion-CLIP

CLIP (Contrastive Language-Image Pre-training) bridges the gap between images and natural language, enabling search using both visual and textual queries. This section explains how CLIP works and introduces Fashion-CLIP, the domain-specialized variant used for visual search.

1.3.3.1 Contrastive Language-Image Pre-Training

Traditional image models classify images into fixed categories such as "cat," "dog," or "dress." CLIP takes a different approach: it learns to match images with text descriptions [5].

During training, CLIP processed 400 million image-text pairs from the public web. For example, an image of a floral dress paired with the caption "colourful floral summer dress," or sneakers paired with "white running shoes on grass." From these pairs, the model learned to convert images into vectors via an image encoder, convert text into vectors via a text encoder, and make matching image-text pairs produce similar vectors.

1.3.3.2 Dual-Tower Architecture

CLIP has two separate towers:

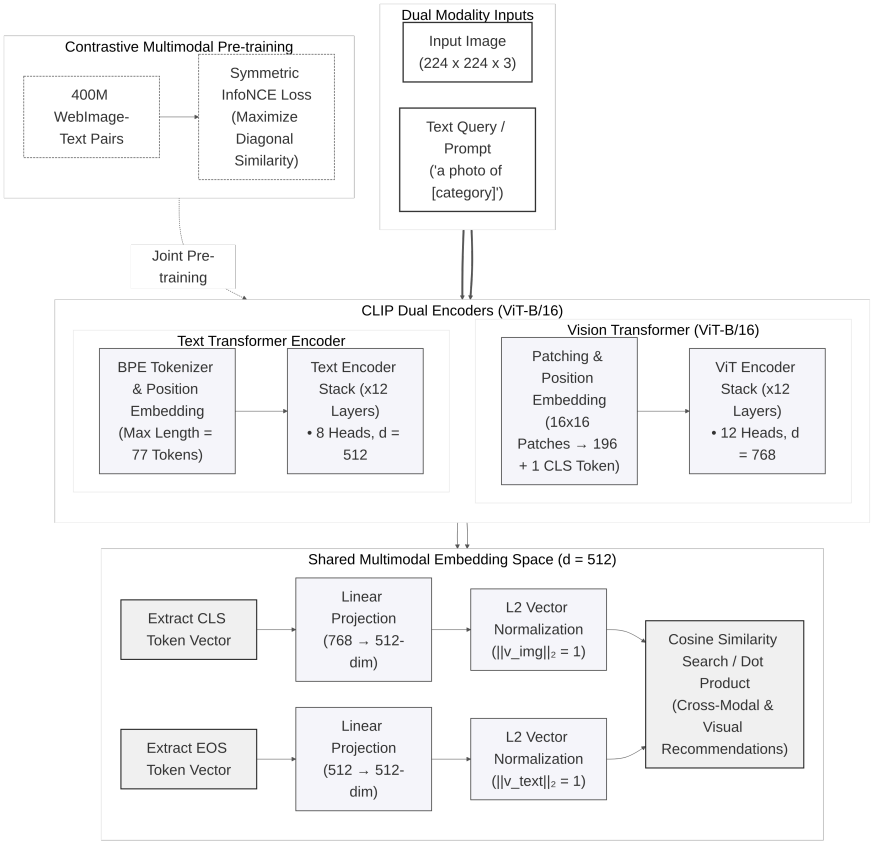

Figure 4: CLIP ViT-B/16 dual-tower architecture: images and text are converted to vectors in the same embedding space, allowing direct comparison

The **Image Tower** processes the image using a Vision Transformer (ViT-B/16, ViT-B/32, or ViT-L/14). The **Text Tower** processes text using a transformer. Both towers output vectors of the same size, 512 dimensions for ViT-B variants and 768 for ViT-L, so images and text can be directly compared using cosine similarity.

1.3.3.3 Multimodal Embedding Space

CLIP's ability to understand natural language makes it powerful for fashion. It understands concepts like "bohemian style" or "minimalist design." It can match images to abstract descriptions such as "something for a casual Friday." It captures semantic meaning beyond just visual appearance.

The dual-tower design enables query modalities unavailable in vision-only models such as DINOv2 or EfficientNet. Users can perform text-to-image search by typing descriptions like "red floral summer dress." The text encoder maps the description into the same embedding space as catalog images. Users can also perform hybrid queries, combining an uploaded photo with a textual refinement such as "like this, but in blue," by encoding both modalities and merging the results.

This flexibility makes CLIP-based models the primary choice for the visual search feature.

1.3.3.4 Fashion-CLIP and Domain-Specific Fine-Tuning

However, the general CLIP model was trained on diverse internet images, not specifically fashion. It may not distinguish "A-line dress" from "sheath dress" or "bohemian style" from "vintage aesthetic." This is where Fashion-CLIP comes in.

Fashion-CLIP is a version of CLIP further trained on fashion-specific data [6]. The researchers used over 700,000 fashion product images paired with detailed descriptions covering garment categories, fabric textures, style descriptors, and occasion labels.

This specialization helps Fashion-CLIP understand fashion-specific vocabulary such as "A-line," "empire waist," and "distressed denim," as well as style categories including "streetwear," "preppy," and "athleisure," and occasion suitability like "office wear," "cocktail party," and "beach vacation."

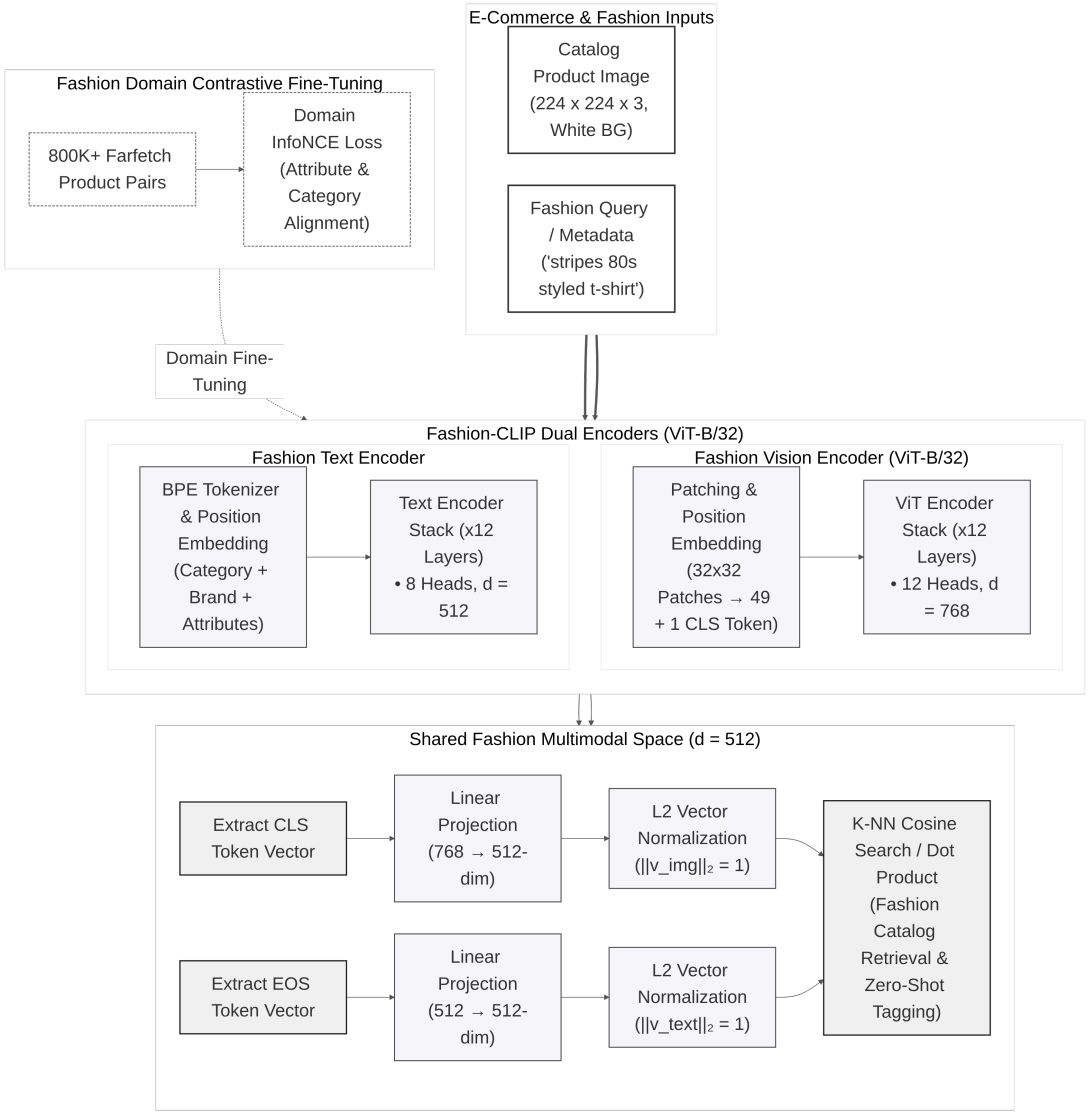

Figure 5: Fashion-CLIP dual-tower architecture: image and text towers independently encode their inputs into 512-dimensional embeddings converging in a shared latent space

Fashion-CLIP inherits the ViT-B/16 architecture, producing 512-dimensional embeddings. The original paper reports a 15 to 20 percent improvement on fashion retrieval over general CLIP, a result confirmed in the benchmark evaluation presented in Chapter 3.

1.3.3.5 Evaluated CLIP Variants

Table 3: CLIP variants evaluated

Model	Architecture	Training	Domain
CLIP ViT-B/32	Dual-tower (ViT + text transformer)	Contrastive (400M pairs)	General

Model	Architecture	Training	Domain
CLIP ViT-B/16	Dual-tower (ViT + text transformer)	Contrastive (400M pairs)	General
CLIP ViT-L/14	Dual-tower (ViT + text transformer)	Contrastive (400M pairs)	General
Fashion-CLIP	Dual-tower (ViT + text transformer)	Contrastive, fine-tuned on 700K fashion images	Fashion-specific

Having introduced four model families (CNN, ViT, general CLIP, and Fashion-CLIP), the next section presents a comparative analysis to determine which model best balances retrieval quality, inference speed, and feature capabilities for the fashion e-commerce use case.

1.3.4 Model Selection and Justification

This section presents the comparative analysis of the embedding models and the rationale for selecting Fashion-CLIP as the primary model for visual search.

1.3.4.1 Candidate Models

Eleven pre-trained models spanning three architectural families were evaluated:

Table 4: Candidate embedding models evaluated for fashion product retrieval. All models are pre-trained and publicly available.

Model	Architecture	Dim	Parameters	Training Method
ResNet-50	CNN	2048	25.6M	Supervised (ImageNet)
ResNet-101	CNN	2048	44.5M	Supervised (ImageNet)
EfficientNet-B0	CNN	1280	5.3M	Supervised (ImageNet)
EfficientNet-B4	CNN	1792	19.3M	Supervised (ImageNet)
DINOv2 ViT-S/14	ViT	384	21M	Self-supervised (142M images)
DINOv2 ViT-B/14	ViT	768	86M	Self-supervised (142M images)
CLIP ViT-B/32	ViT + Text	512	151M	Contrastive (400M pairs)
CLIP ViT-B/16	ViT + Text	512	150M	Contrastive (400M pairs)
CLIP ViT-L/14	ViT + Text	768	428M	Contrastive (400M pairs)
Fashion-CLIP	ViT + Text	512	150M	Fine-tuned (700K fashion)

1.3.4.2 Evaluation Methodology

All models were evaluated under identical conditions. The evaluation used 5,000 fashion product images from the Fashion Product Images dataset, split into training and query sets. Hardware was consumer-grade: Intel i7-1165G7 CPU with 16 GB RAM, with all inference executed on CPU.

Metrics included Mean Average Precision (mAP@10) as the primary measure of overall retrieval quality, Precision at K (P@K) for the accuracy of top-ranked results, Recall at K (R@K) for coverage of relevant items, and inference latency in milliseconds for the average time to generate one embedding. A retrieved product was considered relevant if it belonged to the same category as the query image. The full evaluation protocol, benchmark results, and cross-validation methodology are presented in Chapter 3.

1.3.4.3 Weighted Selection Criteria

The model selection was based on four criteria: retrieval quality measured by mAP@10 and P@K scores, inference latency supporting real-time search with sub-300 ms total response time, multimodal capability enabling search by both image and text, and hardware compatibility within the memory and compute constraints of commodity hardware.

1.3.4.4 Selection Decision

Fashion-CLIP was selected as the primary embedding model for the visual search feature. Three factors drove this decision.

First, retrieval quality: Fashion-CLIP achieved the highest mAP among the evaluated models, with a 15 to 20 percent improvement over general CLIP on fashion-specific queries. This result was confirmed through the systematic benchmark presented in Chapter 3 [6].

Second, multimodal capability: Fashion-CLIP's dual-tower architecture enables search by image, by text description, and by hybrid image-plus-text queries. This capability is unavailable in vision-only models such as DINOv2 and EfficientNet.

Third, domain specialization: fine-tuning on 700,000 fashion images gives Fashion-CLIP an understanding of fashion-specific vocabulary, styles, and garment attributes that general-purpose models lack.

While EfficientNet-B0 offers faster inference for ultra-low-latency mobile deployments and DINOv2 excels at structural fidelity for silhouette-based matching, Fashion-CLIP provides the best overall balance of retrieval quality, search flexibility, and inference performance for the target deployment scenario.

1.3.4.5 Alternative Deployment Scenarios

For different deployment contexts, alternative models may be preferred. EfficientNet-B0 provides the fastest inference at 5.3 million parameters, though it trades off 3.4 percent lower mAP@10 and has no text-to-image search capability. DINOv2 excels at shape and silhouette matching but lacks multimodal capability. General CLIP variants suit multi-category marketplaces, with lower accuracy on fashion-specific queries. CLIP ViT-L/14 offers the largest model capacity at 428 million parameters but requires a GPU with significant VRAM.

The complete numerical comparison and error analysis across all models are presented in Chapter 3.

1.4 VECTOR DATABASES

Once images are converted to embeddings, those vectors must be stored and searched efficiently. This section explains the challenge of vector similarity search at scale, introduces two indexing algorithms, and describes pgvector, the PostgreSQL extension used in this project.

1.4.1 The Search Challenge

When a user uploads a query image, the system must compare its embedding vector against every product vector in the catalog. This is **nearest neighbour search**.

A naive brute-force approach scales linearly with catalog size:

- 10,000 products = 10,000 comparisons per search.
- 100,000 products = 100,000 comparisons.
- 1,000,000 products = 1,000,000 comparisons.

For real-time search (under one second), this is too slow.

1.4.2 Approximate Nearest Neighbour Search

The solution is **Approximate Nearest Neighbour** (ANN) search [12]. Instead of checking every vector, ANN algorithms build index structures (graphs or clusters) that let a query navigate directly to the neighbourhood of likely matches, skipping irrelevant vectors.

The key trade-off: we trade perfect accuracy for speed. If the true top match has similarity 0.95 and the returned result has 0.93, that is acceptable for product search. ANN algorithms typically achieve 97 to 99% recall of the true top matches while reducing search time by orders of magnitude [12]. Two algorithms are used in this project: **HNSW** for production search and **IVFFlat** for rapid evaluation [13].

1.4.3 HNSW: Hierarchical Navigable Small World

HNSW is the preferred index for production-scale vector search [12]. It builds a multi-layered graph where each vector is a node connected to its nearest neighbours.

- **Structure.** Multiple layers form a hierarchy. Top layers are sparse and connect distant regions for long-range jumps. Bottom layers are dense and refine the search locally.
- **Search.** Start at a top-layer node, greedily traverse toward the nearest neighbour, descend one layer, and repeat. Cost scales logarithmically with catalog size.
- **Recall.** Reported at over 95% with query latencies under 10 ms, sustained across catalog scales of up to 10 million vectors [12].
- **Build cost.** Graph construction is computationally expensive. At tens of thousands of vectors, build time is measured in minutes.

Configuration parameters:

- **M.** Connections per node. Default 16. Higher improves recall at cost of memory and build time.
- **ef_construction.** Candidates evaluated during index build. Higher produces better index at cost of build time.

HNSW's logarithmic query cost makes it suitable for interactive fashion retrieval at millions of catalog items [12], where sub-100 ms latency is required.

1.4.4 IVFFlat: Inverted File with Flat Compression

IVFFlat is a simpler ANN algorithm used during model evaluation. It partitions the embedding space into clusters via **k-means** and stores each vector in its nearest cluster [13].

- **Search.** Compute distance to all cluster centroids, select the nearest few clusters, and search exhaustively within only those clusters.
- **Recall.** Moderate: 65 to 72% at sub-10 ms for catalog-scale data, per pgvector documentation [13]. Recall degrades as the catalog grows because each cluster contains more candidates.
- **Build cost.** Low. Clustering completes in under one second for 5,000 vectors with minimal memory overhead.

Configuration parameters:

- **lists.** Number of clusters. More lists means smaller clusters and faster search, but risks missing relevant vectors.
- **probes.** Number of clusters searched per query. More probes improve recall at linear cost to query latency.

IVFFlat is used for the model comparison benchmarks in Chapter 3, where the objective is ranking embedding models rather than optimising index performance. Its fast build time and simple configuration suit rapid evaluation. For production, HNSW is the designated long-term index.

1.4.5 pgvector: Vector Search in PostgreSQL

pgvector is an open-source PostgreSQL extension that adds vector operations to standard SQL [13]. It stores vectors alongside regular product data (names, prices, images) in the same database.

Key features:

- **Vector column.** `VECTOR(512)` stores 512-dimensional embeddings. Supports varying dimensions for different models.
- **Similarity operators.** `<=>` for cosine distance (range $[0, 2]$), `<->` for Euclidean distance.
- **Indexing.** Supports both HNSW and IVFFlat for fast approximate search.

1.4.5.1 Why pgvector

The critical advantage is **transactional consistency**. Vectors and product metadata share the same ACID boundary. An image change triggers a new embedding, and both are committed atomically. No dual-database drift between a vector store and the relational source of truth.

Combined queries are possible in a single SQL statement. A search can find visually similar products filtered by category and price range using one query plan.

1.4.5.2 Example

```
CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name TEXT,
    price DECIMAL,
    image_embedding VECTOR(512)
);

CREATE INDEX ON products
USING hnsw (image_embedding vector_cosine_ops);

SELECT id, name, price,
       1 - (image_embedding <=> query_vector) AS similarity
FROM products
ORDER BY image_embedding <=> query_vector
LIMIT 10;
```

1.4.6 Architectural Decision and Trade-offs

Specialised vector databases (Pinecone, Milvus, Weaviate) exist for large-scale search. pgvector was selected for its simplicity and transactional integration.

Table 5: Comparison of pgvector with specialised vector databases

Feature	Specialised Vector DB	pgvector
Setup	Moderate to high	Low (extension only)
Consistency	Separate database	Same transaction as product data
Query language	Custom API	Standard SQL
Cost	Often paid service	Free, open source
Scale limit	Billions of vectors	Millions of vectors

For thousands to tens of thousands of products, pgvector's simplicity outweighs the scaling advantages of specialised databases.

Limitations acknowledged:

- **Scale.** Performs well for millions of vectors; not designed for billion-vector deployments [13].
- **Maturity.** Fewer features and optimisation options than dedicated vector databases.
- **Distribution.** Does not natively distribute across multiple servers.

For this project's scope (5,000 products in evaluation), these limitations are acceptable. The primary contribution is architectural integration within a conventional e-commerce stack, not massive-scale infrastructure.

1.5 PLATFORM ARCHITECTURE AND TECHNOLOGY STACK

Building a production-capable e-commerce platform with integrated machine learning requires deliberate architectural and technology choices. This section surveys architectural patterns, describes each technology in the ReSys.Shop stack, and provides the rationale for each selection.

1.5.1 Architectural Patterns

An application's architecture determines how code is partitioned and how components communicate [14]. Three patterns govern system-level structure; a fourth, vertical slice architecture, organises code within each module.

Table 6 compares these four patterns.

Table 6: Comparison of architectural patterns

Pattern	Structure	Strengths	Limitations	Suited For
Monolith [15]	Single deployable unit; one codebase, build, and deployment	No network overhead; simple development and debugging	Components entangle over time; full restart on any deployment	Small applications; one team; few business domains
Microservices [16]	Independent services each owning one business capability; network communication	Autonomous teams with different stacks; independent scaling and deployment	Network latency; service discovery; partial failure handling; distributed transaction coordination	Large organisations; independently evolving teams and domains
Modular Monolith [15]	Independent modules in a single process; in-process message bus; compile-time cross-module isolation	Module independence without networking cost; one deployment and one database; simple migration path to microservices	Single database may limit extreme-scale growth; all modules restart on any deployment	Single-team projects requiring logical separation without distributed infrastructure
VSA [17]	Self-contained feature folders with handler, validation, data access, and endpoint definition	All code for one feature co-located; features added, modified, or removed in isolation	Cross-cutting concerns (authentication, logging) require explicit extraction across slices	Numerous independent features evolving at different cadences

1.5.1.1 Architectural Decision

ReSys.Shop combines three patterns:

- **Modular monolith** at the system level. Nine business modules (Catalog, Ordering, Payment, Inventory, Identity, Profile, Shipping, Location, Dashboard) run in a single .NET process. Modules communicate through MediatR, an in-process message bus implementing the CQRS pattern [18]. Compile-time

rules prevent direct cross-module references; all inter-module communication passes through commands and queries.

- **Vertical slice architecture** within each module [17]. Each feature is a self-contained folder containing its handler, request, response, endpoint, and validator. Features are added, modified, or removed in isolation without touching code in other slices.
- **Python sidecar** as the only cross-process component. Embedding generation runs in a dedicated FastAPI service, isolated from the .NET runtime. The sidecar communicates over HTTP; a GPU failure in the sidecar does not affect e-commerce API availability.

This combination provides the deployment simplicity of a monolith, the code isolation of microservices, and machine learning capability without distributed infrastructure overhead [15].

1.5.2 .NET Backend

The backend is built on .NET 10, a high-performance runtime with ahead-of-time compilation and native asynchronous I/O [19]. Its architecture is organised around five core libraries:

- **Carter** extends ASP.NET minimal APIs with module-based endpoint registration. Each business module declares its own routes independently, keeping endpoint definitions co-located with their handlers rather than centralised in a startup file.
- **MediatR** implements CQRS, routing commands (writes) and queries (reads) to handlers through an in-process message bus [18]. Handlers are discovered at startup and dispatched by request type, with no direct coupling between modules.
- **Entity Framework Core** maps C# domain objects to PostgreSQL tables, including pgvector column types for embedding storage [20]. Migrations are version-controlled and applied at startup.
- **FluentValidation** enforces input rules at the application boundary. Each request type has an associated validator that runs before the handler executes, rejecting invalid data before it reaches business logic.
- **Vertical slice architecture** [17] (described in Section 1.5.1) organises each feature as a self-contained folder containing the handler, request, response, endpoint, and validator. This colocation keeps feature concerns together rather than spread across technical layers.

1.5.3 Vue.js Frontend

The frontend uses Vue 3 with TypeScript and the Vite build tool [21]. Two surfaces share a common component library and state management:

- **Storefront.** Product catalog browsing with category trees, faceted filters (price, size, colour), and paginated result grids. Visual search with image upload, similarity-ranked results displaying thumbnail, price, and similarity score. Cart management with session-based guest support and a multi-step checkout flow spanning address entry, delivery selection, payment confirmation, and order completion.
- **Administration interface.** Complete lifecycle management for products, variants, taxonomies, and option types. Order processing with fulfilment status tracking. User and role administration with permission assignment. Analytics overview summarising sales and inventory metrics.
- **Pinia**, a state management library, organises client-side state through typed stores. Each bounded context has its own store (catalog, cart, auth, orders), mirroring the backend module boundaries.

1.5.4 PostgreSQL and pgvector

PostgreSQL 17 hosts both relational business data and vector embeddings in a single database [13].

- **Relational schema.** Tables for products, variants, orders, users, and supporting entities are organised by bounded context. Foreign keys reference entities within the same context; cross-context references use identifier columns without database-level constraints, preserving logical isolation.
- **Performance.** Composite indexes on query-critical combinations (user status, session status, product slug) optimise frequent access patterns. Query plans combine vector similarity and relational filtering in a single execution.

The pgvector extension, HNSW indexing, and transactional consistency between product data and embeddings are described in detail in Section 1.4.

1.5.5 Redis Caching

Redis 7 operates alongside the .NET **HybridCache** abstraction in a two-tier arrangement [22].

- **L1: in-process.** Frequently accessed data (taxonomy trees, front-page product lists) is held in application memory with sub-millisecond read latency. Cache entries expire on a configurable sliding window, typically five minutes.

- **L2: Redis.** The shared tier synchronises cache across application instances. Redis also backs Hangfire job queues and guest session storage. On cache miss at L1, the value is retrieved from Redis and promoted to L1, ensuring subsequent hits stay in-process.

1.5.6 Python ML Sidecar

The machine learning capability runs as a dedicated Python 3.12 service, isolated from the .NET backend due to incompatible runtime dependencies (PyTorch requires Python, .NET requires the CLR) [23].

- **Framework.** FastAPI provides async HTTP endpoints with automatic OpenAPI schema generation. Uvicorn serves as the ASGI runtime.
- **Model management.** A singleton **ModelManager** lazy-loads models from the HuggingFace hub on first request. Once loaded, models persist in GPU memory (or CPU, if no GPU is available) for the lifetime of the service. The manager supports multiple architectures through a common embedding interface.
- **API surface.** `POST /embeddings` accepts raw image bytes (JPEG, PNG, WebP) and returns a JSON array of floating-point values. `GET /health` reports the currently loaded model, its embedding dimension, and the most recent inference latency.
- **Security.** Requests require an `X-API-Key` header validated at the middleware layer. The sidecar listens only on the internal Docker network, inaccessible from the public internet.

1.5.7 .NET Aspire Orchestration

.NET Aspire coordinates the multi-container development and deployment environment [24].

- **Service discovery.** Components resolve each other by name: the .NET backend reaches the Python sidecar at `http://ml-service`, PostgreSQL at `postgres`, and Redis at `redis`. Configuration is injected via environment variables managed by the Aspire host.
- **Lifecycle.** Aspire enforces startup ordering (database before API, sidecar before backend health check) and gates each component behind a readiness probe. Containers restart on failure with configurable back-off.
- **Observability.** OpenTelemetry SDKs in both .NET and Python services export distributed traces, request metrics, and structured logs to the Aspire dashboard. Correlation IDs propagate across the HTTP boundary between

backend and sidecar, linking a search request to its embedding generation span.

1.5.8 Hangfire Background Jobs

Hangfire processes operations that should not block HTTP requests [25]. Jobs are persisted in Redis for resilience across application restarts.

- **Cart expiry.** A recurring job runs daily, removing carts with no activity for seven days. This prevents abandoned carts from accumulating indefinitely.
- **Embedding queue.** When an admin uploads new product images, embedding generation tasks are enqueued for asynchronous processing. The upload endpoint returns immediately; the embedding appears in search results once the job completes.
- **Index maintenance.** A periodic job rebuilds the HNSW index on the embedding column, optimising search performance as the catalog grows.

1.5.9 Identity, Authentication, and Authorisation

- **ASP.NET Identity** provides user account management: password hashing with salted PBKDF2, email confirmation workflows, and optional two-factor authentication via TOTP [19].
- **JWT tokens.** Access tokens carry claims (user ID, roles, permissions) and expire after 15 minutes [26]. Refresh tokens enable silent renewal: the client exchanges a refresh token for a new access token, and each refresh token is single-use. Reuse of an already-consumed refresh token triggers revocation of all tokens for that user, mitigating token theft.
- **Guest sessions.** Anonymous users receive a signed session cookie. This cookie links to a server-side session backed by Redis, enabling cart operations and product browsing without authentication. On registration or login, the guest session is transferred to the authenticated user context.
- **Permission model.** Authorisation uses granular claims in the format `domain:category:action` (e.g., `catalog:products:create`). Roles aggregate commonly used permission sets. Endpoint-level attributes evaluate claims at the middleware layer, so handler code contains no authorisation logic.

1.5.10 Benchmark Framework

The benchmark framework is a Python 3.12 pipeline for systematic, reproducible evaluation of embedding models on fashion product retrieval [23]. It is

separate from the application codebase and produces the experimental results reported in Chapter 3.

- **Modes.** Three evaluation modes are supported: a one-shot comparison across all configured models, a three-fold cross-validation protocol with stratified category-based splits (the default for thesis results), and a pgvector pipeline mode that measures end-to-end latency including database query and network round-trip time.
- **Models.** 11 pre-trained architectures are implemented: CNN-based (ResNet-50, ResNet-101, ResNet-152, EfficientNet-B0, EfficientNet-B4), vision transformers (DINOv2 ViT-S/14, DINOv2 ViT-B/14), and CLIP variants (CLIP ViT-B/32, CLIP ViT-B/16, CLIP ViT-L/14, Fashion-CLIP). Each model has a thin adapter implementing a common `generate_embeddings` interface.
- **Caching.** Embeddings are cached to disk per model, per fold, and per split (query vs. catalog), avoiding recomputation when re-running evaluation on the same configuration.
- **Metrics.** Retrieval accuracy is measured through mAP, Precision at K, Recall at K, and nDCG. Operational efficiency is measured through inference latency (mean and SD per image), throughput (images per second), model load time, on-disk storage, and RAM usage.
- **Outputs.** Results are exported in JSON, CSV, Markdown, and Typst table formats. The Typst output files (`thesis_aggregate.typ`, `thesis_efficiency.typ`) embed directly into the thesis without manual transcription.
- **Multi-label pipeline.** A separate enriched-dataset mode supports evaluation across three label schemes of increasing granularity (category only, category and colour, and category, colour and pattern), enabling analysis of how embedding quality varies with annotation detail.

1.5.11 Technology Stack Summary

The preceding sections introduced the principal technologies that compose the ReSys.Shop platform. Table 7 consolidates the complete stack.

Table 7: Technology stack of the ReSys.Shop platform

Layer	Technology	Role
Frontend	Vue 3, TypeScript, Vite	Customer storefront and administration interface; reactive UI with Pinia state management

Layer	Technology	Role
Backend API	.NET 10, Carter, MediatR	REST endpoints via minimal APIs; CQRS command-query separation across business modules
Database	PostgreSQL, pgvector	Relational data and vector embeddings in a single ACID database with HNSW-indexed similarity search
Caching	Redis, HybridCache	Two-tier cache (in-memory L1 and Redis L2); Hangfire job queue and session state backing store
ML Sidecar	Python 3.12, FastAPI, PyTorch	Dedicated embedding generation service with lazy model loading and GPU acceleration
Orchestration	.NET Aspire	Container lifecycle management, service discovery, and reproducible local development environment
Background Jobs	Hangfire	Persistent job processing for cart expiry, embedding queue, and maintenance tasks
Auth and Identity	JWT, ASP.NET Identity	Access tokens, refresh rotation, permission-based authorisation, guest sessions
Benchmarking	Python 3.12, PyTorch	Systematic 11-model comparison with cross-validation, accuracy and efficiency metrics

1.6 RELATED WORK AND RESEARCH GAP

This section positions the ReSys.Shop platform within the landscape of fashion image retrieval research and commercial visual search systems, and identifies the engineering gap that this thesis addresses.

1.6.1 Academic Research

The **DeepFashion** dataset, introduced by Liu et al., established the foundational benchmark for fashion recognition and retrieval with over 800,000 images annotated with attributes, landmarks, and in-shop-to-consumer photo pairs [27]. This dataset catalysed much of the subsequent work in fashion AI.

FashionIQ extended retrieval to the conversational setting, where users modify queries through natural language feedback (“like this dress but shorter”) [28]. While compelling, the interactive dialogue paradigm requires infrastructure beyond the scope of this project, which focuses on single-turn visual and text queries.

The **Fashion-CLIP** work demonstrated that domain-specific fine-tuning of CLIP on 700,000 fashion images improves retrieval by 15 to 20% over the

general model [6]. This thesis follows that approach, using pre-trained models without custom training, and extends the evaluation to additional architectures (ResNet, EfficientNet, DINOv2) for systematic comparison.

1.6.2 Commercial Systems

Several platforms have deployed visual search at production scale.

Table 8: Comparison of commercial visual search products

Product	Key Strength	Limitation
Google Lens	Massive scale, general-domain coverage	Closed ecosystem, not customisable
Pinterest Lens	Over 600M monthly searches, style-aware	Proprietary, requires Pinterest integration
ASOS Style Match	Fashion-specific accuracy	Restricted to ASOS catalog only
ViSenze	API-based, good accuracy	Paid service with recurring per-query costs

These products share common limitations for independent projects: they are proprietary and cannot be studied or modified, API access incurs costs at query volume, and reliance on external services creates vendor lock-in. This thesis demonstrates that comparable functionality is achievable with open-source tools, providing both a reference implementation and a cost-effective alternative for smaller deployments.

1.6.3 Contribution Differentiators

This project distinguishes itself from prior work by addressing the **engineering gap** between model research and production systems. Four contributions define this gap:

1. Polyglot architecture. Python's machine learning ecosystem (PyTorch, HuggingFace) does not natively interoperate with the .NET stack common in enterprise e-commerce. This thesis presents a modular monolith with a dedicated AI sidecar, combining .NET's type safety and transactional integrity with Python's access to state-of-the-art vision models, without the operational overhead of a full microservices deployment.

2. Vector-native consistency. By using pgvector within PostgreSQL, embeddings and product metadata share the same transactional boundary. Product updates, image replacements, and index maintenance occur atomically, elimi-

nating stale-index bugs that arise when a vector store and relational database have independent consistency guarantees.

3. Commodity hardware benchmarking. Commercial visual search runs on cloud TPU clusters. This thesis evaluates 11 models on consumer-grade hardware, establishing that production-quality visual search is achievable without specialised infrastructure, lowering the barrier for small to medium e-commerce platforms.

4. Applied model comparison. Rather than chasing leaderboard metrics, this thesis compares models within realistic deployment constraints (inference latency budget, memory limits, storage cost). The resulting accuracy-efficiency trade-off data, presented in Chapter 3, provides a pragmatic guide for practitioners selecting embedding models.

2 DESIGN AND IMPLEMENTATION

This chapter translates requirements into a concrete system design and describes the implementation of the principal components.

- **Requirements Specification.** 88 functional requirements, non-functional requirements, feature classification.
- **System Modeling.** Actors, work breakdown structure, 26 use cases with traceability to requirements.
- **System Architecture and Design.** Domain-driven design, C4 diagrams, database schema, API design, security model.
- **Implementation.** Technology stack, vertical slice architecture, data persistence with pgvector, ML sidecar and CBIR search flow, frontend applications.

2.1 REQUIREMENTS SPECIFICATION

The platform delivers 88 functional requirements across nine **business modules**, each enforcing domain invariants expressed through entity validation rules and application-layer checks. Five non-functional quality dimensions define performance, security, modularity, observability, and reliability targets that shaped architectural decisions throughout design. Feature classification distinguishes three **core research** contributions, detailed in Sections 2.3 and 2.4, from four **supporting infrastructure** modules that provide the realistic evaluation context described in Section 3.2.

- **Functional Requirements.** Traceable per module: Catalog, Identity, Inventory, Ordering, Payment, Shipping, Profile, Location, and Dashboard.
- **Non-Functional Requirements.** Five quality dimensions with measurable, atomic constraints.
- **Feature Classification.** Core Research versus Supporting Infrastructure: scope of the thesis contribution.

2.1.1 Functional Requirements

Functional requirements are organized by business module with unique identifiers traceable throughout design, implementation, and evaluation chapters. Features follow vertical slice architecture (Section 2.4.1).

2.1.1.1 Catalog Module

Manages product lifecycle, classification, image handling, and the CBIR infrastructure for visual product search.

Table 9: Catalog module functional requirements.

ID	Requirement	Description	Priority
CAT-FR-01	Product Creation	Create products with name, description, URL slug, and SEO metadata.	High
CAT-FR-02	Fashion Metadata	Assign fashion attributes: style code, season, material composition, care instructions, fit notes, department, and gender target.	Medium
CAT-FR-03	Product Variants	Define sellable variants combining option values with SKU, barcode, physical dimensions, weight, and independent pricing.	High
CAT-FR-21	Variant Option Values	Assign, synchronize, retrieve, and revoke option values to define attribute combinations for variants.	High
CAT-FR-22	Variant Pricing	Set, list, synchronize, and remove time-bound prices per variant with currency specification.	Medium
CAT-FR-04	Variant Images	Upload variant images with automatic thumbnail generation, supporting multiple images per variant with display ordering.	High
CAT-FR-14	Variant Image CRUD	Delete, download, list, and update image metadata (alt text, display order) for variant images.	Medium
CAT-FR-15	Embedding Regeneration	Regenerate vector embeddings for images when the active model configuration changes or corrupted embeddings are detected.	Medium
CAT-FR-05	Embedding Generation	Generate vector embeddings for uploaded variant images via the ML sidecar; store in pgvector [13] with model metadata.	High
CAT-FR-06	Image-Based Search	Enable CBIR: accept query images, extract embeddings, query pgvector via HNSW indexing [12] using cosine distance, and return ranked results.	High
CAT-FR-16	Product Availability	Expose real-time per-variant stock availability through storefront product detail endpoints.	High
CAT-FR-17	Similar Products	Retrieve visually similar products based on vector embedding similarity for a target product.	Medium
CAT-FR-07	Search Configuration	Configure minimum similarity thresholds and maximum result limits for CBIR queries.	Medium

ID	Requirement	Description	Priority
CAT-FR-08	Pluggable Models	Switch embedding models via environment variables without code modifications; filter search results by active model.	High
CAT-FR-09	Taxonomy	Organize products via hierarchical taxonomies with nested taxon trees.	Medium
CAT-FR-18	Taxon Rules	Define business rules on taxon nodes with update, synchronization, and retrieval capabilities.	Low
CAT-FR-19	Auto-Classification	Automatically classify products into taxons based on taxon rules when product attributes change.	Low
CAT-FR-10	Option Types	Define option types (e.g., Size, Color, Material) with ordered values reusable across products.	Medium
CAT-FR-20	Option Association	Assign, synchronize, retrieve, and revoke option types on a per-product basis.	Medium
CAT-FR-11	Slug Uniqueness	Enforce URL slug uniqueness across the catalog at the database constraint level.	High
CAT-FR-12	Status Lifecycle	Support product lifecycles (Draft, Active, Archived) with configurable availability dates.	Medium
CAT-FR-13	Master Variant	Designate one variant per product as the master representation for catalog displays.	High

2.1.1.2 Identity Module

Provides authentication, dynamic authorization, and user management using JWT access tokens [26] with refresh token rotation.

Table 10: Identity module functional requirements.

ID	Requirement	Description	Priority
IDN-FR-01	Registration	Register customer accounts with email, password, and basic profile information.	High
IDN-FR-02	Password Login	Authenticate via email and password, issuing a 15-minute JWT access token and refresh token.	High

ID	Requirement	Description	Priority
IDN-FR-03	OAuth Login	Authenticate via Google OAuth 2.0 as an alternative to password credentials.	Medium
IDN-FR-04	Token Rotation	Invalidate previous refresh tokens upon issuance, returning a new access/refresh pair.	High
IDN-FR-05	Reuse Detection	Revoke all active refresh tokens for a user if a previously consumed token is presented.	High
IDN-FR-06	Guest Sessions	Support guest sessions via signed cookies, enabling anonymous cart management and browsing.	High
IDN-FR-07	Role-Based Access	Enforce RBAC with permission claims formatted as `domain:category:action` at middleware boundaries.	High
IDN-FR-11	Role Management	Create, update, delete, and list roles; manage permission assignments per role.	Medium
IDN-FR-12	User-Role Assignment	Assign and revoke roles per user, supporting direct permission overrides.	Medium
IDN-FR-13	User Status	Enable or disable user accounts without purging user record history.	Medium
IDN-FR-08	Password Reset	Reset passwords via single-use, time-limited email verification tokens.	Medium
IDN-FR-14	Password Change	Allow authenticated users to update passwords upon verifying current credentials.	Medium
IDN-FR-15	Email Lifecycle	Confirm email addresses via verification tokens and support email modification requests.	Medium
IDN-FR-16	Session Management	Retrieve current sessions, inspect refresh tokens, and terminate sessions via logout.	High
IDN-FR-09	User Governance	Manage user accounts, role bindings, and permission grants via administrative endpoints.	Medium
IDN-FR-10	Two-Factor Auth	Provide optional TOTP-based two-factor authentication.	Low

2.1.1.3 Inventory Module

Tracks stock quantities across physical locations, manages checkout reservations to prevent overselling, and maintains audit ledgers.

Table 11: Inventory module functional requirements.

ID	Requirement	Description	Priority
INV-FR-01	Stock Locations	Define physical stock locations with address metadata and active flags.	Medium
INV-FR-02	Stock Quantities	Track on-hand and reserved quantities per variant per location.	High
INV-FR-08	Stock Item CRUD	Create, update, delete, and list stock items; support bulk adjustments and CSV imports.	Medium
INV-FR-09	Low Stock Alerting	Identify inventory falling below configured thresholds for replenishment notifications.	Low
INV-FR-03	Checkout Reservation	Reserve inventory during checkout; release upon cart expiry, cancellation, or timeout.	High
INV-FR-11	Cart Reservations	Reserve stock at cart level, inspect status, and auto-release upon abandonment.	High
INV-FR-04	Overselling Protection	Reject order confirmation when requested quantities exceed available unreserved stock.	High
INV-FR-05	Stock Transfers	Record stock movements between locations with origin, destination, quantity, and timestamp.	Medium
INV-FR-10	Transfer Lifecycle	Manage transfer states (Created, In-Transit, Received, Cancelled) with paged listings.	Medium
INV-FR-06	Audit Logging	Maintain an immutable audit log for stock changes recording operator identity and reason codes.	Medium
INV-FR-12	Movement History	Provide detailed, paged audit trails for stock movements as first-class domain entities.	Medium
INV-FR-07	Reservation Expiry	Expire stale inventory holds after a configurable timeout (default: 15 minutes).	Medium

2.1.1.4 Ordering Module

Governs purchase workflows from shopping cart creation through checkout state transitions to completed orders.

Table 12: Ordering module functional requirements.

ID	Requirement	Description	Priority
ORD-FR-01	Shopping Cart	Support guest and customer carts with variant item addition and quantity adjustments.	High
ORD-FR-10	Cart Management	Create, clear, and delete carts, as well as modify item quantities and line items.	High
ORD-FR-02	Cart Association	Merge guest cart contents into customer accounts upon authentication without data loss.	Medium
ORD-FR-03	Cart Expiry	Purge abandoned carts inactive for 7 days via scheduled background maintenance tasks.	Medium
ORD-FR-04	Checkout Pipeline	Enforce strict sequential checkout state transitions: Address → Delivery → Payment → Confirm → Complete.	High
ORD-FR-11	Checkout Validation	Validate inventory levels, address completeness, and shipping method selection prior to payment.	High
ORD-FR-12	Shipping Selection	Select applicable shipping rates within cart boundaries prior to finalizing payment.	Medium
ORD-FR-05	Order Totals	Calculate monetary balances independently: $\text{Item Total} + \text{Adjustments} + \text{Shipping} = \text{Grand Total}$.	High
ORD-FR-06	Dual State Tracking	Maintain parallel, decoupled state machine counters for payment state and fulfillment state.	Medium
ORD-FR-07	Order Cancellation	Cancel orders prior to fulfillment confirmation, releasing inventory holds and voiding payments.	Medium
ORD-FR-13	Admin Order Management	Approve, complete, resume, and modify pending order details via administrative surfaces.	Medium
ORD-FR-14	Customer History	Expose paged customer order histories with line item detail, payment status, and tracking info.	Medium

ID	Requirement	Description	Priority
ORD-FR-08	Order Numbering	Generate unique, sequential order reference numbers within database isolation transactions.	Medium
ORD-FR-09	Order Immutability	Lock finalized orders as immutable to prevent modification post-completion.	High

2.1.1.5 Payment Module

Handles payment intent lifecycles across external payment providers (Stripe) and internal test gateways.

Table 13: Payment module functional requirements.

ID	Requirement	Description	Priority
PAY-FR-01	Intent Creation	Generate payment intents specifying amount, currency, order reference, and gateway target.	High
PAY-FR-02	Intent Confirmation	Confirm payment intents to authorize fund capture during checkout finalization.	High
PAY-FR-08	Setup Intent	Create Stripe SetupIntents to securely save customer payment methods for future checkouts.	Low
PAY-FR-03	Capture and Refund	Execute capture, void, and refund operations using idempotency keys.	High
PAY-FR-04	Webhook Processing	Verify and process incoming gateway webhooks using HMAC cryptographic signatures.	High
PAY-FR-09	Payment Voiding	Void authorized uncaptured payments and release authorization holds on cancelled orders.	Medium
PAY-FR-10	Method Management	Create, update, toggle, and delete configured payment gateway methods via admin panels.	Medium
PAY-FR-05	Refund Validation	Enforce validation rules preventing refund amounts from exceeding total captured funds.	Medium
PAY-FR-06	Bogus Gateway	Provide a mock payment gateway simulating transaction lifecycles without external API calls.	Medium
PAY-FR-07	State Tracking	Maintain local payment state records parallel to external gateway records for offline query support.	Medium

2.1.1.6 Shipping Module

Configures delivery options, geographic destination zones, and dynamic rate calculation algorithms.

Table 14: Shipping module functional requirements.

ID	Requirement	Description	Priority
SHP-FR-01	Delivery Methods	Configure shipping methods with carrier details, rate rules, and geographic zones.	Medium
SHP-FR-04	Method Management	Create, update, activate, deactivate, and delete shipping methods via administrative endpoints.	Medium
SHP-FR-05	Rate Lifecycle	Manage rate matrices mapped to shipping methods, geographic zones, and weight/value tiers.	Medium
SHP-FR-02	Rate Calculation	Calculate shipping costs dynamically based on address zone, method, cart weight, and total value.	Medium
SHP-FR-06	Storefront Evaluation	Evaluate and display eligible shipping methods and rates during checkout.	High
SHP-FR-03	Shipment Tracking	Assign carrier identifiers and tracking tracking numbers to active shipments.	Low

2.1.1.7 Profile Module

Connects user identity records to customer preferences, addresses, and saved items.

Table 15: Profile module functional requirements.

ID	Requirement	Description	Priority
PRF-FR-01	Address Book	Manage shipping and billing addresses with default selections and custom labels.	Medium
PRF-FR-02	Wishlists	Create, update, and remove named wishlists containing targeted product variants and notes.	Low
PRF-FR-03	Notification Settings	Manage channel-specific notification preferences (email, SMS) with opt-in flags.	Low

2.1.1.8 Location Module

Manages reference data for countries and regional states used in address validation, tax, and shipping zone evaluation.

Table 16: Location module functional requirements.

ID	Requirement	Description	Priority
LOC-FR-01	Country Directory	Maintain country reference entries with ISO 3166-1 alpha-2 codes, names, and active flags.	Medium
LOC-FR-02	State Directory	Maintain state/province entries with ISO 3166-2 codes linked to parent countries.	Medium
LOC-FR-03	Country Management	Create, update, delete, and retrieve country records by database ID or ISO code.	Medium
LOC-FR-04	State Management	Create, update, delete, and list state records filtered by parent country.	Medium

2.1.1.9 Dashboard Module

Consolidates key performance indicators across all business domains for operational review.

Table 17: Dashboard module functional requirements.

ID	Requirement	Description	Priority
DSH-FR-01	Aggregate Dashboard	Aggregate metrics (catalog size, order volumes, stock alerts, user activity) into a unified administrative view.	Medium

2.1.2 Non-Functional Requirements

Five quality dimensions define production-readiness constraints. Each requirement is specified as an atomic, measurable target that shaped architectural decisions: the 1-second CBIR latency limit (NFR-01a) mandated synchronous vector generation over queued pipelines; the strict modularity constraints (NFR-03a, NFR-03b, NFR-03c) drove the in-process MediatR dispatch architecture; and the reliability baseline (NFR-05a) required Redis-backed persistence for Hangfire background processing. Verification is evaluated in Chapter 3.

2.1.2.1 Performance

Table 18: Performance latency targets for CBIR search and standard API endpoints.

ID	Requirement Target
NFR-01a	**End-to-End Search Latency:** Complete CBIR search workflows (image upload, embedding generation, pgvector similarity lookup, and response assembly) within **1 second** per query.
NFR-01b	**API Response Latency:** Respond to non-search REST API endpoints within **200 milliseconds** under standard operating load.
NFR-01c	**Non-Blocking Processing:** Enforce asynchronous I/O across all data access and HTTP pipelines to eliminate thread pool starvation under concurrent loads.

2.1.2.2 Security

Table 19: Security requirements for authentication, authorization, rate limiting, and transport safety.

ID	Requirement Target
NFR-02a	**Token Lifecycle:** Expire JWT access tokens [26] after **15 minutes** and enforce single-use refresh token rotation with automated reuse detection.
NFR-02b	**Endpoint Authorization:** Enforce fine-grained claims (`domain:category:action`) at the API middleware boundary using dynamic policy resolution.
NFR-02c	**Rate Limiting:** Restrict authentication requests to **5 attempts per minute** and account registration to **3 attempts per hour** per client IP.
NFR-02d	**Upload Hardening:** Validate uploaded files via magic-byte header inspection, strictly enforce file extension allowlists, and cap upload sizes at **10 MB**.
NFR-02e	**Transport Security:** Inject security headers (`CSP`, `HSTS`, `X-Frame-Options`, `X-Content-Type-Options`) across all HTTP responses.

2.1.2.3 Modularity

Table 20: Modularity requirements governing architectural isolation and communication boundaries.

ID	Requirement Target
NFR-03a	**Compile-Time Isolation:** Maintain business modules within a single .NET assembly while enforcing **zero direct cross-references** via build policy checks.

ID	Requirement Target
NFR-03b	**Decoupled Messaging:** Route all inter-module communications exclusively through MediatR in-process message dispatch.
NFR-03c	**Module Testability:** Ensure every module is independently testable without initializing foreign contexts or external database schemas.

2.1.2.4 Observability

Table 21: Observability requirements for tracing, correlation logging, and health monitoring.

ID	Requirement Target
NFR-04a	**Distributed Tracing:** Propagate OpenTelemetry trace contexts across .NET application and Python ML sidecar boundaries via standard HTTP headers.
NFR-04b	**Structured Logging:** Inject a unique correlation identifier into every log entry to track requests across distributed execution paths.
NFR-04c	**Health Monitoring:** Expose dedicated health endpoints verifying database, cache, and sidecar connectivity for orchestrator probes.

2.1.2.5 Reliability

Table 22: Reliability constraints covering background durability, idempotency, and state timeouts.

ID	Requirement Target
NFR-05a	**Job Durability:** Execute background tasks via Hangfire [25] backed by Redis [22] storage to survive process restarts.
NFR-05b	**Automated Maintenance:** Run daily cleanup routines to purge carts inactive for **7 days** and release associated inventory reservations.
NFR-05c	**Webhook Idempotency:** Enforce idempotency key checks on Stripe webhook payloads to prevent duplicate transaction state updates.
NFR-05d	**Reservation Timeouts:** Automatically expire unconfirmed checkout inventory holds after **15 minutes** of user inactivity.

2.1.3 Feature Classification

Platform features are categorized into Core Research contributions and Supporting Infrastructure components to define the thesis scope. Core research contributions are detailed in Sections 2.3 and 2.4; supporting infrastructure modules

provide a realistic e-commerce environment for evaluation, as described in Section 3.2.

Table 23: Classification of system feature areas into Core Research contributions and Supporting Infrastructure components.

Feature Area	Classification	Rationale
Visual Search (CBIR)	Core Research	**Primary Contribution:** An integrated multi-model CBIR system [29] featuring a pluggable architecture. Enables image-based product discovery across multiple embedding model families, including CNNs [3], ViTs [10], and CLIP-based architectures [5], [6].
ML Embedding Pipeline	Core Research	**Operational Backbone:** An automated ingestion pipeline that processes product images, generates vector embeddings via a PyTorch sidecar [23], and manages storage and HNSW indexing [12] using pgvector [13].
Model Benchmark System	Core Research	**Secondary Contribution:** A systematic benchmarking methodology evaluating retrieval accuracy and execution latency across 11 embedding models, establishing model selection guidelines for production deployments.
Product Catalog	Supporting	**Evaluation Domain:** Maintains structured fashion product data (variants, image sets, taxonomies, and attributes) that serves as the vector search target.
Order System	Supporting	**Conversion Tracking:** Captures end-to-end shopping workflows (cart additions, checkouts) to validate visual search utility using real-world interaction metrics.
Inventory	Supporting	**Availability Constraints:** Filters search queries by real-time stock levels, ensuring retrieved items reflect purchasable inventory.
Authentication	Supporting	**Security Boundary:** Protects administrative surfaces and user session state, establishing a production-representative security baseline.

2.2 SYSTEM MODELING

Three actors interact with the platform across 26 use cases, organised into a functional work breakdown structure (WBS) and a summary matrix with detailed scenario specifications. The use case specifications provide full traceability to the functional requirements defined in Section 2.1.

- **System Actors.** Customer, Administrator, and System: roles, responsibilities, interaction surfaces.
- **Functional Decomposition.** WBS: three functional areas with constituent modules and sub-functions.
- **Use Cases.** 26 specifications: summary matrix, detailed scenarios with traceability.

2.2.1 System Actors

Three categories of actors interact with the platform, distinguished by access level and interaction surface.

2.2.1.1 Customer

The **Customer** accesses the browser-based **storefront**.

- **Discovery.** Browse catalog with faceted filters, keyword search, and visual CBIR (Section 2.3.3). Guests browse and cart without registration; session promoted on login.
- **Purchase.** Persistent cart, multi-step checkout (address, delivery, payment, confirm).
- **Account.** Register or Google OAuth login; manage profile, addresses, wishlists, order history.

2.2.1.2 Administrator

The **Administrator** operates a separate administration interface.

- **Catalog.** CRUD products with fashion metadata; variants and pricing; image uploads triggering embedding pipeline; taxonomies, option types, product classification.
- **Operations.** Order review, payment capture/refund, shipment management; real-time stock monitoring per location with audit history.
- **Governance.** User CRUD; role assignment (Customer, Administrator); permission grants (domain:category:action).

2.2.1.3 System

The **System** actor runs automated background jobs via **Hangfire** [25] and **Redis** [22].

- **Embedding Generation.** Process uploaded images via ML sidecar.
- **Cart Expiry.** Daily job: delete carts inactive 7 days, release inventory.
- **Inventory Reservation.** Hold stock during checkout; expire after 15-minute inactivity.
- **Maintenance.** Periodic HNSW index rebuilds; async payment webhook validation.

2.2.2 Functional Decomposition

Figure 6 presents the work breakdown structure (WBS) of the platform across three functional areas:

- **Administration.** Seven modules: Catalog, Ordering, Payment, Inventory, Identity, Shipping, Location.
- **Storefront.** Three modules: Product Discovery (browse, search, visual search), Purchase Flow (cart, checkout, payment, order history), Account Management (authentication, session, profile).
- **Background Services.** Five processes: Embedding Generation, Cart Expiry, Inventory Reservation, Payment Webhook Processing, Index Optimisation.

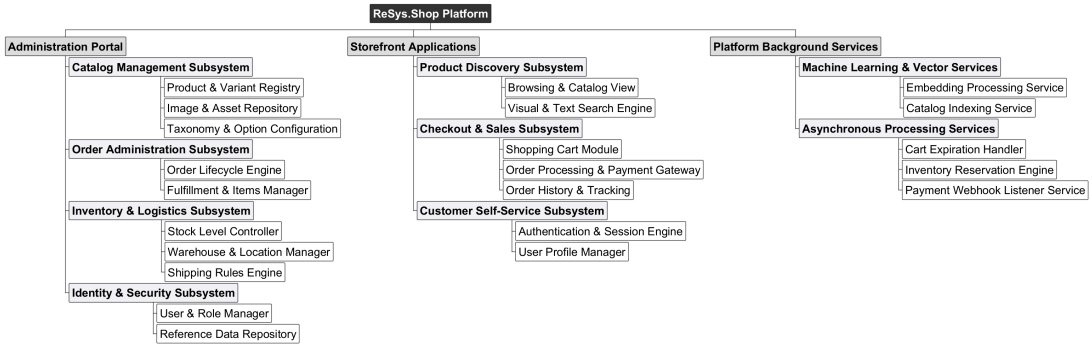

Figure 6: Functional decomposition of ReSys.Shop using a Work Breakdown Structure (WBS), showing the hierarchical breakdown into three functional areas and their constituent modules and sub-functions.

2.2.3 Use Case Summary Matrix

Table 24 consolidates all use cases across the three actors and provides traceability to the functional requirements defined in Section 2.1.

Table 24: Use case summary matrix. All use cases are listed with their actor, associated business module, and traceability to functional requirements defined in Section 2.1.

UC-ID	Use Case	Actor	Module	Related FR
UC-ADM-PROD	Manage products	Admin	Catalog	CAT-FR-01, CAT-FR-02, CAT-FR-03, CAT-FR-11, CAT-FR-12, CAT-FR-13
UC-ADM-VAR	Manage variants	Admin	Catalog	CAT-FR-03, CAT-FR-10, CAT-FR-21, CAT-FR-22

UC-ID	Use Case	Actor	Module	Related FR
UC-ADM-IMG	Manage images and embeddings	Admin	Catalog	CAT-FR-04, CAT-FR-05, CAT-FR-14, CAT-FR-15
UC-ADM-TAX	Manage taxonomies and classification	Admin	Catalog	CAT-FR-09, CAT-FR-18, CAT-FR-19
UC-ADM-OPT	Manage option types	Admin	Catalog	CAT-FR-10, CAT-FR-20
UC-ADM-ORD	Manage orders	Admin	Ordering	ORD-FR-04, ORD-FR-05, ORD-FR-06, ORD-FR-07, ORD-FR-09, ORD-FR-13
UC-ADM-ORD-ITEMS	Manage order details	Admin	Ordering	ORD-FR-13
UC-ADM-PAY	Manage payments	Admin	Payment	PAY-FR-03, PAY-FR-05, PAY-FR-07, PAY-FR-09
UC-ADM-PAY-METHOD	Manage payment methods	Admin	Payment	PAY-FR-10
UC-ADM-STK	Manage stock	Admin	Inventory	INV-FR-02, INV-FR-05, INV-FR-06, INV-FR-08, INV-FR-09, INV-FR-10, INV-FR-12
UC-ADM-LOC	Manage stock locations	Admin	Inventory	INV-FR-01
UC-ADM-USR	Manage users	Admin	Identity	IDN-FR-09, IDN-FR-13
UC-ADM-ROL	Manage roles and permissions	Admin	Identity	IDN-FR-11, IDN-FR-12
UC-ADM-SHP	Manage shipping	Admin	Shipping	SHP-FR-01, SHP-FR-04, SHP-FR-05
UC-ADM-REF	Manage reference data	Admin	Location	LOC-FR-01, LOC-FR-02, LOC-FR-03, LOC-FR-04
UC-STR-BRW	Browse and search catalog	Customer	Catalog	CAT-FR-01, CAT-FR-02, CAT-FR-03, CAT-FR-09,

UC-ID	Use Case	Actor	Module	Related FR
				CAT-FR-16, CAT-FR-22
UC-STR-SRC	Visual search	Customer	Catalog	CAT-FR-06, CAT-FR-07, CAT-FR-08, CAT-FR-17
UC-STR-CRT	Manage cart	Customer	Ordering	ORD-FR-01, ORD-FR-02, ORD-FR-10
UC-STR-CHK	Checkout	Customer	Ordering	ORD-FR-04, ORD-FR-05, ORD-FR-08, ORD-FR-11, ORD-FR-12
UC-STR-OHI	Order history	Customer	Ordering	ORD-FR-07, ORD-FR-14
UC-STR-PAY	Payment processing	Customer	Payment	PAY-FR-01, PAY-FR-02
UC-STR-AUT	Authentication	Customer	Identity	IDN-FR-01, IDN-FR-02, IDN-FR-03, IDN-FR-08, IDN-FR-14
UC-STR-SES	Session management	Customer	Identity	IDN-FR-04, IDN-FR-05, IDN-FR-16
UC-STR-PRF	Profile management	Customer	Profile	PRF-FR-01, PRF-FR-02, PRF-FR-03
UC-SYS-EMB	Embedding operations	System	Embedding	CAT-FR-05, CAT-FR-15
UC-SYS-MNT	System maintenance	System	Infrastructure	CAT-FR-06, CAT-FR-08, ORD-FR-03, INV-FR-07, PAY-FR-04

2.2.4 Administrator Use Cases

The Administrator actor manages data and operational workflows across all business modules through a dedicated administration interface. Each specification includes the actor's goal, trigger, preconditions, postconditions, a numbered main success scenario, alternative and exception flows, and related functional requirements.

2.2.4.1 Product Management

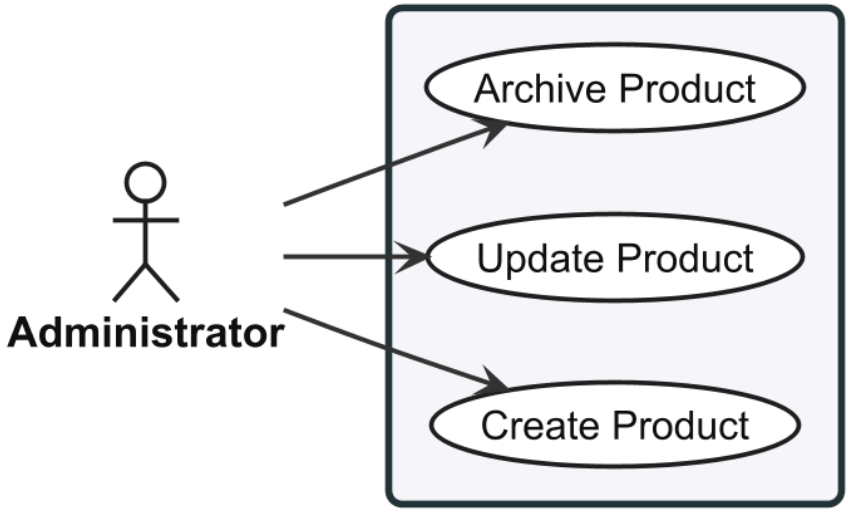

Figure 7: Use case diagram for Product Management (UC-ADM-PROD).

2.2.4.2 UC-ADM-PROD: Manage Products

Table 25: Manage Products.

Use Case	UC-ADM-PROD — Manage Products
Actor	Administrator
Goal	Create, update, archive products with validation and audit trails.
Pre/Post	Pre: catalog management permissions. Post: catalog updated; status transitions audited.
Scenario	**Create** 1. Fill form (name, description, slug, fashion metadata). 2. Designate master variant with SKU and price. 3. Assign to taxons, submit; validates slug uniqueness, persists as Draft. , **Update** 1. Select product from listing. 2. Modify fields: name, slug, metadata, SEO, status. 3. Submit; validates, persists, confirms. , **Archive** 1. Select product, choose archive, confirm. 2. Status → Archived, hidden from storefront.
Alternatives	1. A1. Duplicate slug → reject, prompt new. 2. A2. No master variant → reject, require designation. 3. A3. Active orders on product → warn, explicit confirmation.
Exceptions	1. E1. Persist fails → report, retain form for retry.
Requirements	CAT-FR-01, CAT-FR-02, CAT-FR-03, CAT-FR-11, CAT-FR-12, CAT-FR-13

2.2.4.3 Variant and Pricing

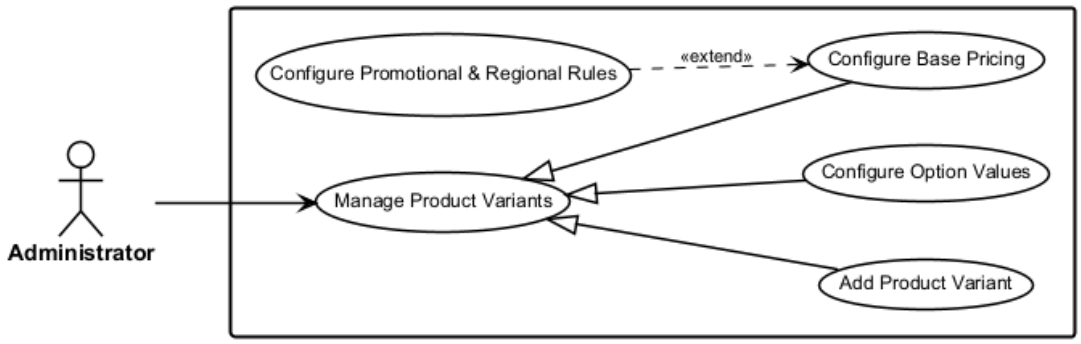

Figure 8: Use case diagram for Variant and Pricing (UC-ADM-VAR).

2.2.4.4 UC-ADM-VAR: Manage Variants

Table 26: Manage Variants.

Use Case	UC-ADM-VAR — Manage Variants
Actor	Administrator
Goal	Add variants with option-value configuration and independent pricing.
Pre/Post	Pre: catalog permissions; parent product exists. Post: variant created/updated with option assignments and pricing.
Scenario	**Add Variant** 1. Navigate product detail, add variant. 2. Enter SKU, barcode, dimensions, weight, position. 3. Assign option values (e.g. Size M, Colour Red). 4. Submit; validates SKU uniqueness and option combination, creates variant. , **Configure Options** 1. Select variant; system shows option assignments. 2. Choose option type and value. 3. Save; validates combination uniqueness. , **Configure Pricing** 1. Open pricing for product; system shows variant prices by currency. 2. Select variants, enter price with currency and optional validity. 3. Submit; validates non-negative price and currency.
Alternatives	1. A1. Duplicate SKU → reject, prompt new. 2. A2. Duplicate option combination → reject, highlight conflict. 3. A3. Zero price → accept with warning. 4. A4. Overlapping price date range → reject.
Exceptions	1. E1. Persist fails → report, retain form for retry.
Requirements	CAT-FR-03, CAT-FR-10, CAT-FR-21, CAT-FR-22

2.2.4.5 Image and Embedding Management

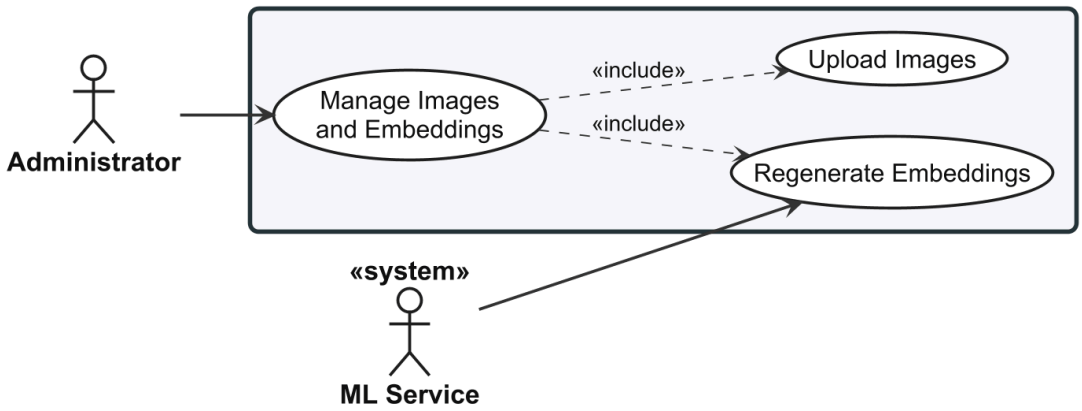

Figure 9: Use case diagram for Image and Embedding Management (UC-ADM-IMG).

2.2.4.6 UC-ADM-IMG: Manage Images and Embeddings

Table 27: Manage Images and Embeddings.

Use Case	UC-ADM-IMG — Manage Images and Embeddings
Actor	Administrator
Support	ML Service
Goal	Upload variant images; trigger and manage embedding generation.
Pre/Post	Pre: image management permissions; variant exists. Post: image stored with variant association; embeddings available for visual search.
Scenario	**Upload** 1. Select variant, choose image file. 2. System validates format (JPEG, PNG, WebP) and size (max 10 MB). 3. Enter alt text and display order, submit. 4. System stores image, generates thumbnails, schedules embedding. , **Regenerate Embeddings** 1. Select images, initiate regeneration. 2. System displays confirmation with affected image count. 3. Confirm; system sends each to ML service, stores new embeddings with model metadata, reports completion.
Alternatives	1. A1. Unsupported format → reject, list accepted formats. 2. A2. Exceeds max size → reject, show constraint. 3. A3. No images selected → disable action, prompt selection.
Exceptions	1. E1. ML service unavailable → report failure, suggest retry. 2. E2. Processing unschedulable → store image; notify search excludes it until processed.
Requirements	CAT-FR-04, CAT-FR-05, CAT-FR-14, CAT-FR-15

2.2.4.7 Taxonomy and Classification

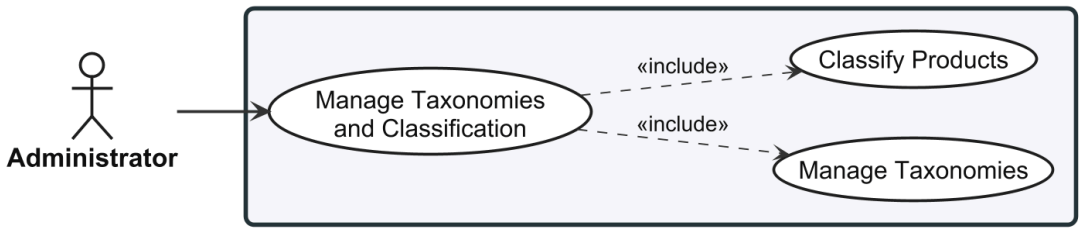

Figure 10: Use case diagram for Taxonomy and Classification (UC-ADM-TAX).

2.2.4.8 UC-ADM-TAX: Manage Taxonomies and Classification

Table 28: Manage Taxonomies and Classification.

Use Case	UC-ADM-TAX — Manage Taxonomies and Classification
Actor	Administrator
Goal	Create and modify taxonomy structures; assign product classifications.
Pre/Post	Pre: taxonomy management permissions. Post: taxonomy structure and product classifications updated.
Scenario	**Manage Taxonomies** 1. Open taxonomy management; system displays tree. 2. Create or select root, add/edit/reorder/remove nodes with optional business rules. 3. Save; persists, confirms. , **Classify Products** 1. Select products, open classification panel. 2. Choose taxons to assign or deselect. 3. Save; validates taxon paths, persists associations.
Alternatives	1. A1. Delete taxon with children → prompt cascade-delete or reassign. 2. A2. Delete taxon with products → warn loss of classification. 3. A3. All taxons removed → warn product hidden from category browsing.
Exceptions	1. E1. Concurrent modification → refresh data, retry.
Requirements	CAT-FR-09, CAT-FR-18, CAT-FR-19

2.2.4.9 Option Type Configuration

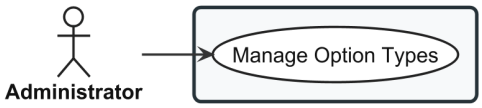

Figure 11: Use case diagram for Option Type Configuration (UC-ADM-OPT).

2.2.4.10 UC-ADM-OPT: Manage Option Types

Table 29: Manage Option Types.

Use Case	UC-ADM-OPT — Manage Option Types
Actor	Administrator
Goal	Create, modify, remove option types and their ordered values.
Pre/Post	Pre: option type management permissions. Post: option types updated and available for product configuration.
Scenario	1. Open management; system displays all option types with values. 2. Create new type with name and presentation style. 3. Add ordered values (e.g. S, M, L, XL for Size). 4. Optionally edit, reorder, or remove types and values. 5. Save; validates name uniqueness, non-empty values.
Alternatives	1. A1. Delete type in use by products → warn, confirm. 2. A2. Remove value in use by variants → warn, confirm.
Exceptions	1. E1. Concurrent modification → refresh data, retry.
Requirements	CAT-FR-10, CAT-FR-20

2.2.4.11 Order Lifecycle

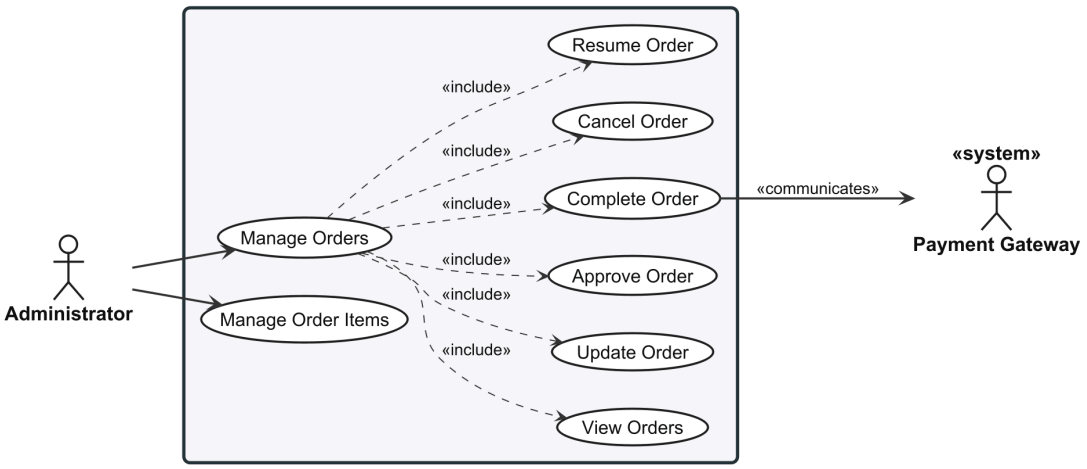

Figure 12: Use case diagram for Order Lifecycle (UC-ADM-ORD, UC-ADM-ORD-ITEMS).

2.2.4.12 UC-ADM-ORD: Manage Orders

Table 30: Manage Orders.

Use Case	UC-ADM-ORD — Manage Orders
Actor	Administrator
Support	Payment Gateway

Goal	View, modify, manage the full order lifecycle.
Pre/Post	Pre: order management permissions. Post: order state transitions performed; changes logged for audit.
Scenario	**View** 1. Navigate order management; paged list sorted by recent. 2. Filter by status, date range, customer email. 3. Select order for detail (items, pricing, payment, shipment, addresses, timeline). , **Update** 1. Open order, initiate edit. 2. Modify line items, addresses, or shipping method. 3. Submit; recalculates totals, persists. , **Approve** 1. Open pending order; verify payment capture and inventory. 2. Approve; transitions to approved. , **Complete** 1. Open order in fulfilment; verify shipment dispatched. 2. Confirm; decrements inventory, transitions to completed. , **Cancel** 1. Open order, select cancel, provide reason. 2. Confirm; releases inventory, voids/refunds payment, transitions to cancelled. , **Resume** 1. Locate paused order, resolve issue. 2. Resume; validates prerequisites, transitions to pending.
Alternatives	1. A1. No orders match → suggest broader filters. 2. A2. Payment not captured (Approve) → prevent, suggest capture first. 3. A3. Gateway unreachable (Cancel) → cancel order, release inventory, queue payment action. 4. A4. Prerequisites not met (Resume) → prevent, display issues. 5. A5. Concurrent state change → refresh, notify.
Exceptions	1. E1. Order became immutable → refresh, notify. 2. E2. Gateway state mismatch → prevent, verify with gateway. 3. E3. Inventory decrement conflict → report, verify stock.
Requirements	ORD-FR-04, ORD-FR-05, ORD-FR-06, ORD-FR-07, ORD-FR-09, ORD-FR-13

2.2.4.13 UC-ADM-ORD-ITEMS: Manage Order Details

Table 31: Manage Order Details.

Use Case	UC-ADM-ORD-ITEMS — Manage Order Details
Actor	Administrator

Goal	Manage line items, shipping, and billing addresses on mutable orders.
Pre/Post	Pre: order update permissions; order in mutable state. Post: order details updated with recalculated totals; change logged.
Scenario	1. Open order, initiate edit; system shows editable form with current values. 2. Modify line items (add, update, remove), addresses. 3. Submit; validates stock, address completeness, recalculates totals.
Alternatives	1. A1. Quantity exceeds stock → reject, show max. 2. A2. All items removed → reject, at least one required. 3. A3. Address incompatible with shipping method → warn, prompt change.
Exceptions	1. E1. Order became immutable → refresh, notify.
Requirements	ORD-FR-13

2.2.4.14 Payment Processing

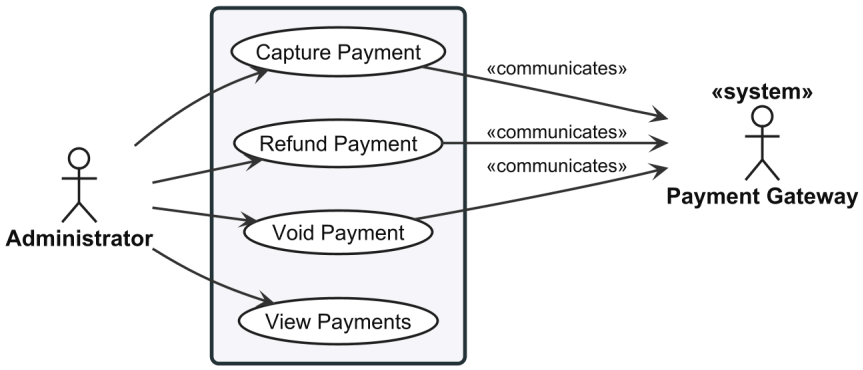

Figure 13: Use case diagram for Payment Processing (UC-ADM-PAY).

2.2.4.15 UC-ADM-PAY: Manage Payments

Table 32: Manage Payments.

Use Case	UC-ADM-PAY — Manage Payments
Actor	Administrator
Support	Payment Gateway
Goal	Capture, refund, void payments; review payment history.
Pre/Post	Pre: payment management permissions. Post: payment state updated; gateway operations recorded.
Scenario	**Capture** 1. Open payment detail; system shows state and authorised amount.

	2. Optionally adjust partial amount. 3. Confirm; sends capture with idempotency key to gateway, updates state. , **Refund** 1. Open captured payment; enter amount (full/partial) and reason. 2. Confirm; validates amount, sends refund, updates state. , **Void** 1. Open authorised uncaptured payment. 2. Confirm void; sends to gateway, updates to voided. , **View** 1. Navigate payment management; paged list by recent. 2. Filter by state, date range, gateway, order. 3. Select for detail (amount, currency, gateway, state timeline, capture/refund history).
Alternatives	1. A1. Partial capture/refund → remainder available. 2. A2. Already captured (Void) → prevent, suggest refund. 3. A3. State mismatch (View) → highlight discrepancy, suggest sync.
Exceptions	1. E1. Gateway rejects → report rejection reason. 2. E2. Gateway unreachable → report; idempotency key enables safe retry.
Requirements	PAY-FR-03, PAY-FR-05, PAY-FR-07, PAY-FR-09

2.2.4.16 Payment Method Configuration

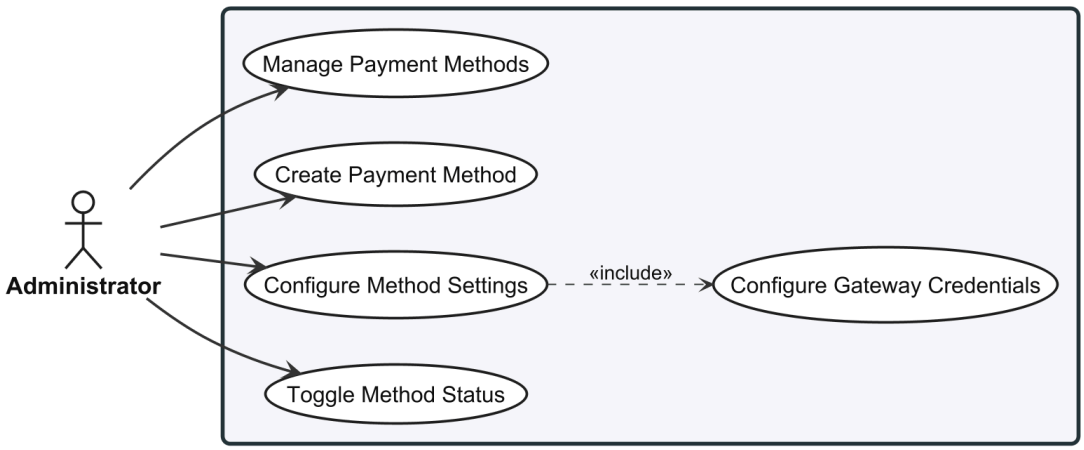

Figure 14: Use case diagram for Payment Method Configuration (UC-ADM-PAY-METHOD).

2.2.4.17 UC-ADM-PAY-METHOD: Manage Payment Methods

Table 33: Manage Payment Methods.

Use Case	UC-ADM-PAY-METHOD — Manage Payment Methods
Actor	Administrator
Goal	Create, update, activate, deactivate payment gateway methods.
Pre/Post	Pre: payment method management permissions. Post: methods updated; available for storefront selection.
Scenario	1. Navigate configuration; system displays methods with activation status. 2. Create new (name, description, gateway identifier, parameters). 3. Optionally edit, activate, deactivate, or remove. 4. Save; validates name uniqueness and gateway support.
Alternatives	1. A1. Deactivate method in use → warn new orders cannot use it; existing unaffected. 2. A2. Remove method → verify no pending payments. 3. A3. Invalid gateway parameters → reject, highlight.
Exceptions	1. E1. Concurrent modification → refresh, retry.
Requirements	PAY-FR-10

2.2.4.18 Stock Location Management

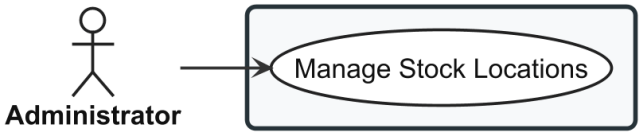

Figure 15: Use case diagram for Stock Location Management (UC-ADM-LOC).

2.2.4.19 UC-ADM-LOC: Manage Stock Locations

Table 34: Manage Stock Locations.

Use Case	UC-ADM-LOC — Manage Stock Locations
Actor	Administrator
Goal	Create, update, remove physical stock locations.
Pre/Post	Pre: location management permissions. Post: location configuration updated.
Scenario	1. Navigate location management; system displays existing locations with addresses and active status. 2. Create new (name, address, active flag) or edit/deactivate/remove. 3. Save; validates location name uniqueness.
Alternatives	1. A1. Delete location with active stock → prevent; require transfer first. 2. A2. Deactivate → prevent new intake; existing movements retained.

	3. A3. Delete last location → prevent; at least one required.
Exceptions	1. E1. Concurrent modification → refresh, retry.
Requirements	INV-FR-01

2.2.4.20 Stock Item Management

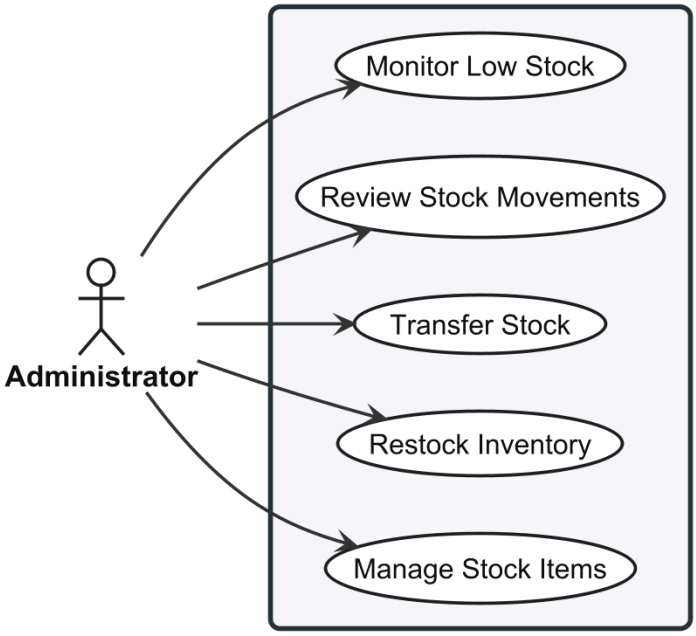

Figure 16: Use case diagram for Stock Item Management (UC-ADM-STK).

2.2.4.21 UC-ADM-STK: Manage Stock

Table 35: Manage Stock.

Use Case	UC-ADM-STK — Manage Stock
Actor	Administrator
Goal	Create, update, restock, transfer stock items; monitor levels.
Pre/Post	Pre: stock management permissions. Post: quantities updated; changes logged for audit.
Scenario	**Manage Stock Items** 1. Navigate stock items; system displays variant, location, on-hand, reserved. 2. Create (select variant + location, enter quantity) or edit existing. 3. Provide reason, save; validates variant-location uniqueness, records audit. , **Restock** 1. Locate item, enter quantity, reference, notes. 2. Confirm; increments on-hand, records event.

	, **Transfer** 1. Initiate transfer; select source, destination, variant, quantity. 2. Validate sufficient stock at source, submit (decrements source). 3. On arrival, confirm receipt; increments destination, transitions to completed. , **Review Movements** 1. Open audit interface; reverse chronological paged list. 2. Filter by date, variant, location, type. 3. Select movement for detail. , **Monitor Low Stock** 1. Open low stock view. 2. Review items below threshold: variant, location, on-hand, threshold, days since restock.
Alternatives	1. A1. Bulk file upload → process, validate, report success/failure counts. 2. A2. Reduce below reserved → reject, show current reserved. 3. A3. Transfer exceeds stock → reject, show max. 4. A4. Cancel pending transfer → return deducted quantity, log. 5. A5. No items below threshold → display sufficient stock message.
Exceptions	1. E1. Duplicate variant-location → reject, suggest editing existing. 2. E2. Concurrent modification → refresh, re-enter. 3. E3. Retrieval fails → display error, offer retry.
Requirements	INV-FR-02, INV-FR-05, INV-FR-06, INV-FR-08, INV-FR-09, INV-FR-10, INV-FR-12

2.2.4.22 User Management

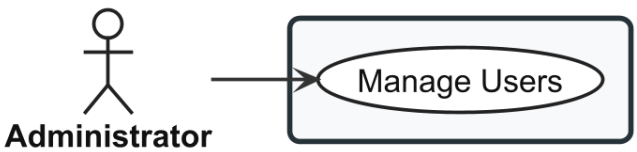

Figure 17: Use case diagram for User Management (UC-ADM-USR).

2.2.4.23 UC-ADM-USR: Manage Users

Table 36: Manage Users.

Use Case	UC-ADM-USR — Manage Users
Actor	Administrator
Goal	Create, update, enable, disable user accounts.
Pre/Post	Pre: user management permissions. Post: account created/modified; status changes take effect on next authentication.

Scenario	1. Navigate user management; system displays paged list. 2. Filter by email, name, role, status. 3. Create new (email, name, assign roles) or select existing. 4. Modify fields, enable/disable, adjust role assignments. 5. Save; validates email uniqueness; on disable, revokes all sessions.
Alternatives	1. A1. Disable account → revoke all sessions immediately. 2. A2. Re-enable → restore active status. 3. A3. Duplicate email → reject, prompt new.
Exceptions	1. E1. Persist fails → report, retain form for retry.
Requirements	IDN-FR-09, IDN-FR-13

2.2.4.24 Role and Permission Governance

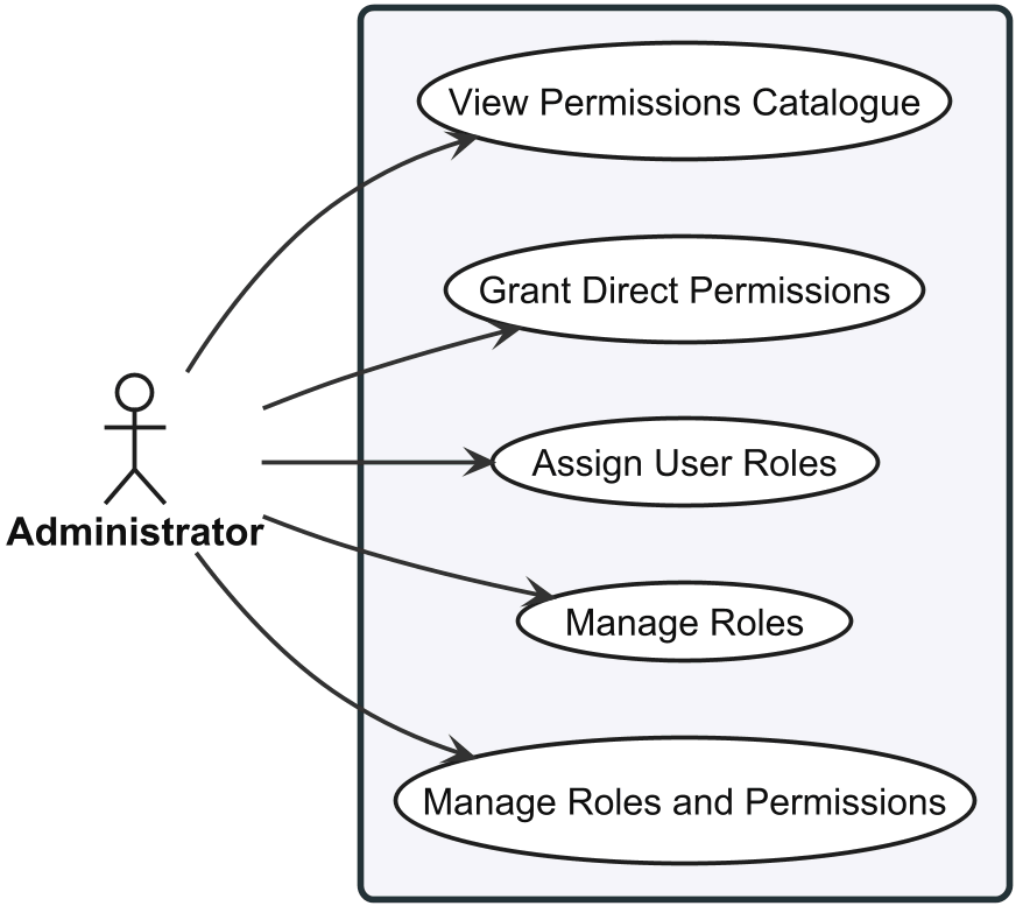

Figure 18: Use case diagram for Role and Permission Governance (UC-ADM-ROL).

2.2.4.25 UC-ADM-ROL: Manage Roles and Permissions

Table 37: Manage Roles and Permissions.

Use Case	UC-ADM-ROL — Manage Roles and Permissions
Actor	Administrator
Goal	Create, manage roles; assign permissions and roles to users.
Pre/Post	Pre: role and permission management rights. Post: configuration updated; affected users receive updated permissions on next token.
Scenario	**Manage Roles** 1. Open role management. 2. Create role (name, description). 3. Assign permissions from catalogue. 4. Optionally edit existing role; save; validates name uniqueness. , **Assign User Roles** 1. Open user detail, role assignment panel. 2. Check/uncheck roles. 3. Save; displays updated effective permissions. , **Grant Direct Permissions** 1. Open user detail, direct permission panel. 2. Select/deselect permissions (role-inherited and direct indicators shown). 3. Save; recalculates effective permissions. , **View Permissions Catalogue** 1. Navigate catalogue; all permission claims grouped by domain. 2. Filter by domain, category, keyword. 3. Review matrix.
Alternatives	1. A1. Remove role assigned to users → warn affected. 2. A2. Revoke permission from role → warn members lose it. 3. A3. Revoke last role → warn user loses all role-derived permissions. 4. A4. Grant already inherited → accept; effective unchanged, direct grant recorded.
Exceptions	1. E1. Concurrent modification or role deleted → refresh, retry.
Requirements	IDN-FR-11, IDN-FR-12

2.2.4.26 Shipping Method Configuration

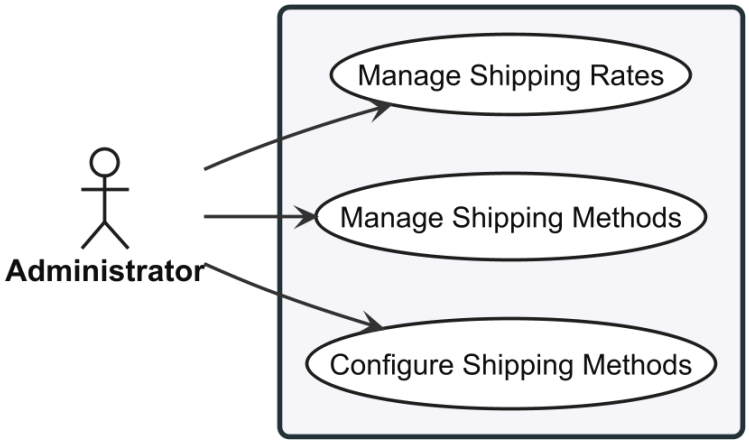

Figure 19: Use case diagram for Shipping Method Configuration (UC-ADM-SHP).

2.2.4.27 UC-ADM-SHP: Manage Shipping

Table 38: Manage Shipping.

Use Case	UC-ADM-SHP — Manage Shipping
Actor	Administrator
Goal	Configure delivery methods and their associated shipping rates.
Pre/Post	Pre: shipping management permissions. Post: methods and rates configured; active methods available for checkout if zone-applicable.
Scenario	**Manage Methods** 1. Navigate shipping management; system displays methods with activation status. 2. Create new (name, description, carrier, zones). 3. Optionally edit, activate, deactivate, or remove. 4. Save; validates name uniqueness. , **Manage Rates** 1. Select method; system displays rate tiers. 2. Create rate tier (zone, weight/value ranges, rate amount). 3. Optionally edit or remove tiers. 4. Save; validates ranges do not overlap for same zone.
Alternatives	1. A1. Deactivate method in active checkouts → warn; existing unaffected. 2. A2. Remove method with active rates → prompt reassign or remove. 3. A3. No zones assigned → warn method not selectable at checkout. 4. A4. Overlapping rate tiers → reject, highlight conflict.
Exceptions	1. E1. Concurrent modification → refresh, retry.
Requirements	SHP-FR-01, SHP-FR-04, SHP-FR-05

2.2.4.28 Reference Data Management

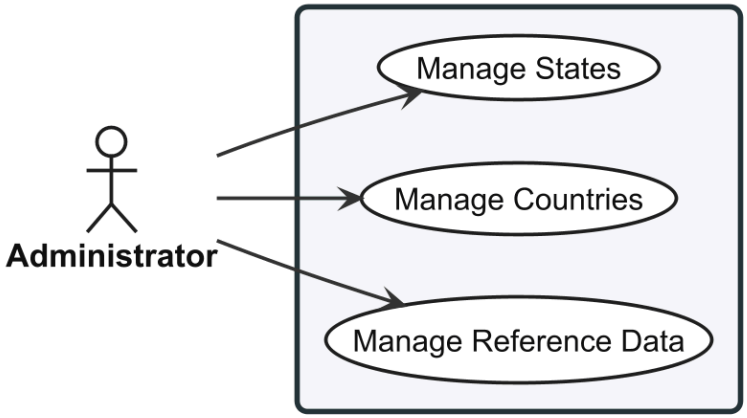

Figure 20: Use case diagram for Reference Data Management (UC-ADM-REF).

2.2.4.29 UC-ADM-REF: Manage Reference Data

Table 39: Manage Reference Data.

Use Case	UC-ADM-REF — Manage Reference Data
Actor	Administrator
Goal	Create and update country and state reference data.
Pre/Post	Pre: reference data management permissions. Post: country and state data updated; active records available in address forms and shipping zones.
Scenario	**Manage Countries** 1. Navigate country management; system displays list with name, ISO code, active status. 2. Create new (name, ISO 3166-1 code, active flag). 3. Optionally edit or remove. 4. Save; validates ISO code uniqueness and format. , **Manage States** 1. Navigate state management; system displays list with name, ISO code, parent country, active status. 2. Create new (name, ISO 3166-2 code, parent country). 3. Optionally edit or remove. 4. Save; validates ISO code uniqueness within parent country.
Alternatives	1. A1. Deactivate country → warn addresses retained; country not in new forms. 2. A2. Delete country with states → warn states orphaned. 3. A3. Delete country in shipping zones → warn, suggest deactivate instead. 4. A4. Deactivate parent country → prompt cascade-deactivate states.
Exceptions	1. E1. Duplicate ISO code → reject, prompt new.
Requirements	LOC-FR-01, LOC-FR-02, LOC-FR-03, LOC-FR-04

2.2.5 Customer Use Cases

The Customer actor interacts through the browser-based storefront for product discovery, purchase, and account management. Each specification follows the same structure as the Administrator use cases.

2.2.5.1 Catalog Browsing

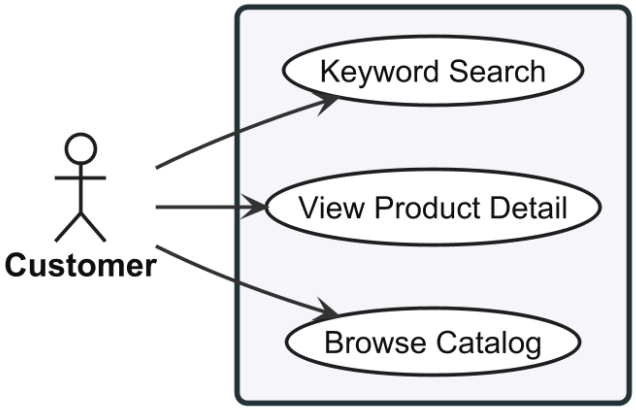

Figure 21: Use case diagram for Catalog Browsing (UC-STR-BRW).

2.2.5.2 UC-STR-BRW: Browse and Search Catalog

Table 40: Browse and Search Catalog.

Use Case	UC-STR-BRW — Browse and Search Catalog
Actor	Customer
Goal	Browse product catalog, view details, keyword search.
Pre/Post	Pre: catalog data published and searchable. Post: product listing or detail displayed with pricing and availability.
Scenario	**Browse** 1. Visit storefront; system displays taxonomy tree. 2. Select category; system retrieves products for taxon and descendants. 3. System displays paginated product grid (thumbnail, name, price, availability). 4. Apply facet filters (size, colour); system refreshes. , **View Detail** 1. Click product card. 2. View detail page: image, name, description, price range, fashion metadata, taxonomy breadcrumb. 3. Browse image gallery. 4. Select variant options; system updates price, availability, images. , **Keyword Search** 1. Enter term in search bar. 2. System queries name, description, fashion attributes. 3. System ranks and displays paginated results. 4. Refine with more keywords or filters.
Alternatives	1. A1. Empty category → suggest other categories. 2. A2. Variant out of stock → show status, disable add-to-cart.

	3. A3. No search matches → suggest spelling check or browse categories. 4. A4. Single-character query → prompt minimum two characters.
Exceptions	1. E1. Retrieval failure → display error, offer retry.
Requirements	CAT-FR-01, CAT-FR-02, CAT-FR-03, CAT-FR-09, CAT-FR-16, CAT-FR-22

2.2.5.3 Search

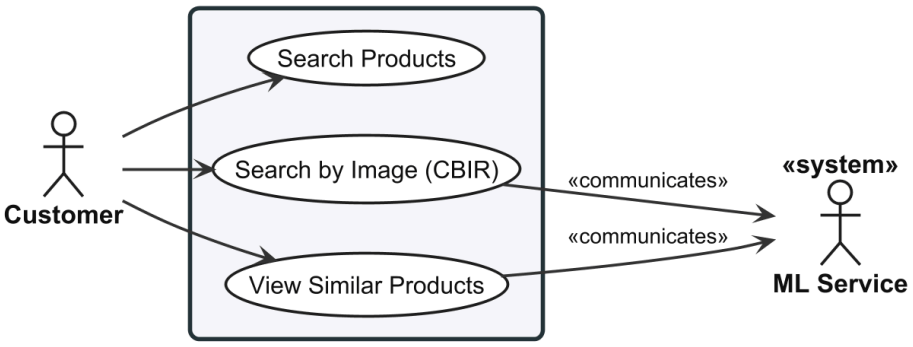

Figure 22: Use case diagram for Search (UC-STR-SRC).

2.2.5.4 UC-STR-SRC: Visual Search

Table 41: Visual Search.

Use Case	UC-STR-SRC — Visual Search
Actor	Customer
Support	ML Service
Goal	Search products by uploading a reference image; discover visually similar items.
Pre/Post	Pre: catalog has products with embeddings; ML service operational. Post: visually similar products displayed with similarity scores above configured threshold.
Scenario	**Search by Image** 1. Navigate visual search; select/drag image. 2. System validates format (JPEG, PNG, WebP) and size. 3. System sends image to ML service for embedding. 4. System performs similarity search against index. 5. System retrieves products ranked by score above threshold. 6. System displays thumbnail grid with scores. , **View Similar Products** 1. Open product detail with embeddings. 2. System retrieves primary image embedding. 3. System performs similarity search against other product embeddings.

	4. System displays carousel/grid of visually similar cards. 5. Click card to navigate to detail.
Alternatives	1. A1. Invalid format → reject, list accepted formats. 2. A2. No results above threshold → suggest different image or keyword search. 3. A3. No embeddings on detail page → hide similar products section. 4. A4. ML unavailable on detail → gracefully hide; page still functional.
Exceptions	1. E1. ML service unavailable → display error, suggest retry. 2. E2. Embedding index empty → report not yet available.
Requirements	CAT-FR-06, CAT-FR-07, CAT-FR-08, CAT-FR-17

2.2.5.5 Cart Management

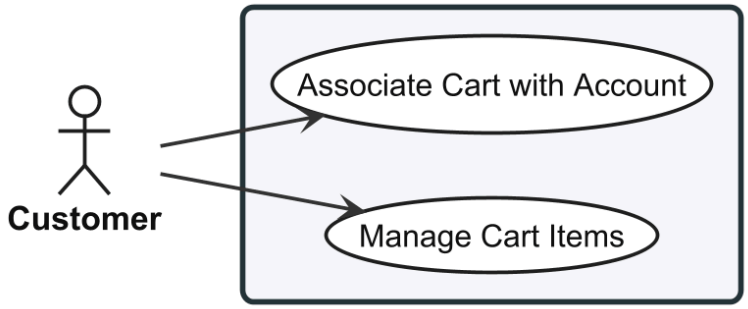

Figure 23: Use case diagram for Cart Management (UC-STR-CRT).

2.2.5.6 UC-STR-CRT: Manage Cart

Table 42: Manage Cart.

Use Case	UC-STR-CRT — Manage Cart
Actor	Customer
Goal	Add, update, remove items; associate guest cart with account.
Pre/Post	Pre: variant exists and available. Post: cart persisted across navigation; guest carts survive browser sessions.
Scenario	**Manage Items** 1. On product detail, select variant and quantity. 2. Click Add to Cart; system validates, adds, confirms. 3. Open cart; view items with quantities, prices, subtotal. 4. Update quantity or remove; system recalculates. , **Associate Cart with Account** 1. Browse as guest, add items. 2. Login or register. 3. System detects guest cart, retrieves user cart.

	4. System merges carts: matching variants increment quantity; unique added. 5. System associates merged cart with account, invalidates guest cookie.
Alternatives	1. A1. Quantity exceeds stock → reject, show max. 2. A2. Same variant already in cart → increment quantity. 3. A3. Guest → assign session-based cookie. 4. A4. No existing user cart → transfer guest cart directly. 5. A5. Merge exceeds stock → cap at max, notify.
Exceptions	1. E1. Variant deactivated/archived → reject, suggest refresh. 2. E2. Merge data conflict → create user cart with guest items, notify review.
Requirements	ORD-FR-01, ORD-FR-02, ORD-FR-10

2.2.5.7 Checkout Flow

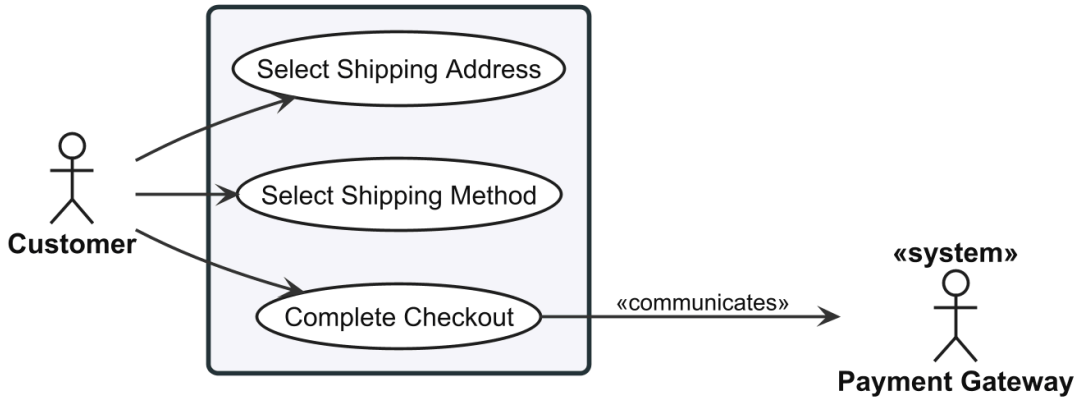

Figure 24: Use case diagram for Checkout Flow (UC-STR-CHK).

2.2.5.8 UC-STR-CHK: Checkout

Table 43: Checkout.

Use Case	UC-STR-CHK — Checkout
Actor	Customer
Support	Payment Gateway
Goal	Complete multi-step checkout: address, shipping, payment, confirm.
Pre/Post	Pre: authenticated; cart not empty; stock available for all items. Post: order created with unique number; inventory reserved; payment linked; cart cleared.
Scenario	**Select Address** 1. Proceed from cart; system transitions to Address step. 2. System displays saved addresses (default pre-selected).

	3. Select or enter new address; system determines zone, proceeds. , **Select Shipping** 1. System calculates available methods/rates by zone, cart weight, value. 2. Displays options with rates and delivery estimates. 3. Select method; system applies rate, updates order summary, proceeds. , **Complete** 1. System displays order summary (items, shipping, tax, total). 2. Enter payment details or select saved method. 3. System creates payment intent, reserves inventory per item. 4. Review and confirm; system captures payment, generates order number. 5. System transitions to Confirmed, clears cart, displays confirmation, sends notification.
Alternatives	1. A1. No saved addresses → prompt create new. 2. A2. No methods for zone → prompt different address. 3. A3. Only one method → auto-select, proceed. 4. A4. Stock depleted during checkout → notify, remove affected items, return to cart. 5. A5. Payment fails → notify with reason; allow retry; inventory not reserved.
Exceptions	1. E1. Address validation fails → highlight missing fields, prevent progression. 2. E2. Rate calculation fails → display error, suggest contact support. 3. E3. Payment captured but inventory reservation fails → void payment, notify not completed.
Requirements	ORD-FR-04, ORD-FR-05, ORD-FR-08, ORD-FR-11, ORD-FR-12

2.2.5.9 Order History

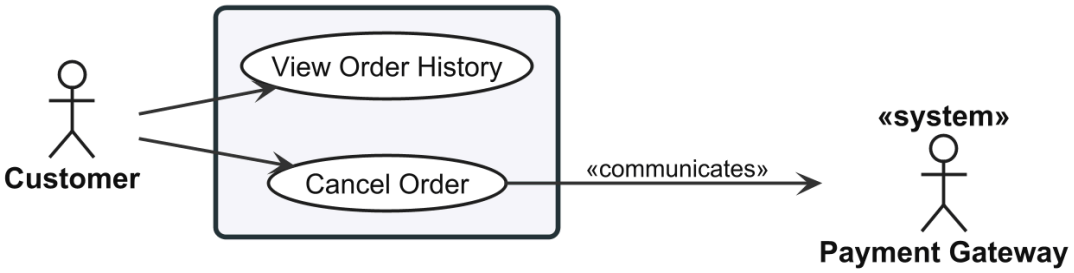

Figure 25: Use case diagram for Order History (UC-STR-OHI).

2.2.5.10 UC-STR-OHI: Order History

Table 44: Order History.

Use Case	UC-STR-OHI — Order History
Actor	Customer
Support	Payment Gateway
Goal	View past orders; cancel pending orders.
Pre/Post	Pre: authenticated. Post: order history visible; cancelled orders release inventory, void payment.
Scenario	**View History** 1. Navigate order history; reverse chronological paged list (order number, date, status, total). 2. Filter by date or status. 3. Select order for detail (items, address, method, payment state, shipment state, timeline). , **Cancel Order** 1. Open cancellable order, select Cancel. 2. System explains inventory release and payment void. 3. Confirm; system releases inventory, voids payment, transitions to cancelled, sends confirmation.
Alternatives	1. A1. No orders → prompt browse catalog. 2. A2. Order state changed → refresh, cancellation unavailable.
Exceptions	1. E1. Gateway unreachable (Cancel) → cancel order, release inventory, queue void, notify. 2. E2. Retrieval fails → display error, offer retry.
Requirements	ORD-FR-07, ORD-FR-14

2.2.5.11 Payment Processing

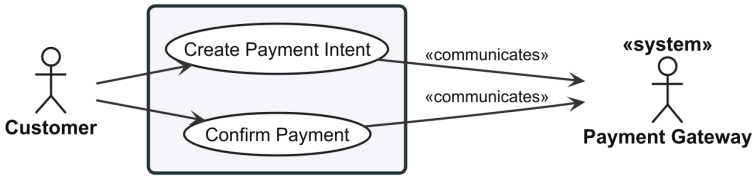

Figure 26: Use case diagram for Payment Processing (UC-STR-PAY).

2.2.5.12 UC-STR-PAY: Payment Processing

Table 45: Payment Processing.

Use Case	UC-STR-PAY — Payment Processing
Actor	Customer
Support	Payment Gateway
Goal	Create and confirm payment for an order.

Pre/Post	Pre: authenticated; checkout at payment step; order has calculated total. Post: payment authorised; order transitions to Confirmed.
Scenario	**Create Intent** 1. System displays payment step with total and available methods. 2. Select method, enter details. 3. System submits to gateway, receives intent confirmation, displays review summary. , **Confirm Payment** 1. Review final summary (items, shipping, tax, total). 2. Confirm; system submits confirmation to gateway. 3. System receives authorisation, updates payment state. 4. System transitions order to Confirmed, clears cart, displays confirmation.
Alternatives	1. A1. Invalid payment details → return validation error, prompt correction. 2. A2. Authorisation declined → display reason, offer retry with different method. 3. A3. Additional authentication required → redirect, resume after.
Exceptions	1. E1. Gateway unreachable → display error; progress saved. 2. E2. Payment confirms but order creation fails → void payment, notify retry. 3. E3. Gateway timeout → mark pending; advise check history; webhook updates state.
Requirements	PAY-FR-01, PAY-FR-02

2.2.5.13 Authentication

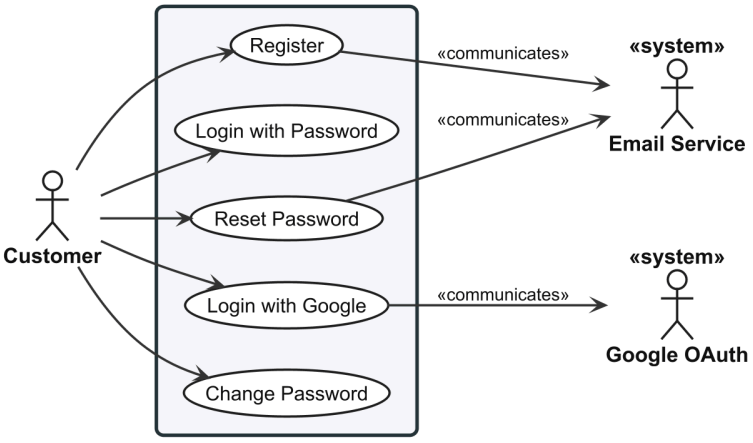

Figure 27: Use case diagram for Authentication (UC-STR-AUT).

2.2.5.14 UC-STR-AUT: Authentication

Table 46: Authentication.

Use Case	UC-STR-AUT — Authentication
Actor	Customer
Support	Email Service, Google OAuth
Goal	Register, log in, manage account credentials.
Pre/Post	Pre: public access. Post: customer authenticated or credentials updated.
Scenario	**Register** 1. Navigate registration; enter email, password, profile info. 2. Submit; validates email not registered, password strength. 3. System creates account, sends verification. , **Login (Password)** 1. Enter email and password. 2. Submit; validates credentials. 3. System issues access and refresh tokens, associates guest cart if exists. , **Login (Google)** 1. Click Google login; system redirects to OAuth consent. 2. Grant consent; system exchanges code. 3. System looks up or creates account, issues tokens. , **Reset Password** 1. Click Forgot Password, submit email. 2. System sends time-limited reset link. 3. Open link, enter new password, submit. 4. System validates token, updates password, revokes all sessions. , **Change Password** 1. Navigate account security, select Change Password. 2. Enter current and new password, submit. 3. System verifies current, validates new, updates, revokes other sessions.
Alternatives	1. A1. Duplicate email → reject, suggest login or reset. 2. A2. Weak password → highlight requirements, prompt retry. 3. A3. Invalid credentials → reject with generic message. 4. A4. Account disabled → reject, contact support. 5. A5. Reset token expired → prompt new request. 6. A6. Current password incorrect → reject, retry.
Exceptions	1. E1. Verification/reset email fails → create account/request; log failure. 2. E2. Token issuance fails → report failure, suggest retry.
Requirements	IDN-FR-01, IDN-FR-02, IDN-FR-03, IDN-FR-08, IDN-FR-14

2.2.5.15 Session Management

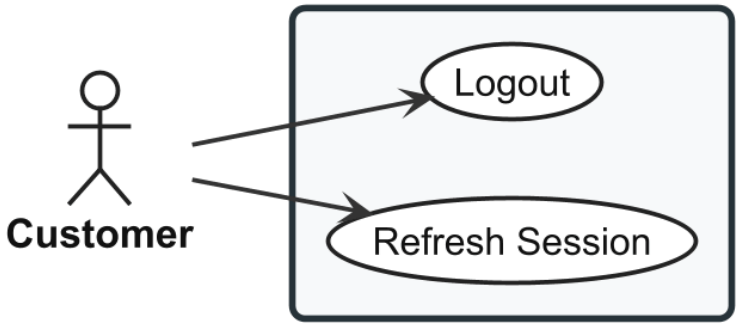

Figure 28: Use case diagram for Session Management (UC-STR-SES).

2.2.5.16 UC-STR-SES: Session Management

Table 47: Session Management.

Use Case	UC-STR-SES — Session Management
Actor	Customer
Goal	Maintain and terminate authenticated sessions.
Pre/Post	Pre: active session. Post: session extended or terminated.
Scenario	**Refresh** 1. Client detects token near expiry. 2. Sends refresh token; system validates (not expired, not consumed). 3. System issues new access and refresh tokens with fresh expiry. 4. System rotates refresh token (invalidates previous). 5. Client stores new pair. , **Logout** 1. Click Logout. 2. System receives refresh token for invalidation. 3. System invalidates token, removes session cookie. 4. System redirects to home page.
Alternatives	1. A1. Refresh token expired → reject; redirect to login. 2. A2. Refresh token consumed (reuse) → revoke all tokens; redirect to login with security notification. 3. A3. Logout network failure → clear local tokens; session expires naturally.
Exceptions	1. E1. Token issuance/invalidation fails → retain existing pair if still valid; clear locally otherwise.
Requirements	IDN-FR-04, IDN-FR-05, IDN-FR-16

2.2.5.17 Profile and Preferences

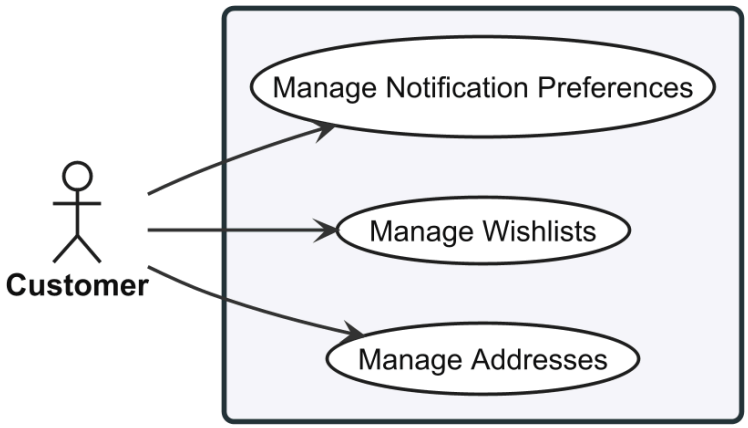

Figure 29: Use case diagram for Profile and Preferences (UC-STR-PRF).

2.2.5.18 UC-STR-PRF: Profile Management

Table 48: Profile Management.

Use Case	UC-STR-PRF — Profile Management
Actor	Customer
Goal	Manage addresses, wishlists, notification preferences.
Pre/Post	Pre: authenticated. Post: profile data and preferences updated.
Scenario	**Manage Addresses** 1. Navigate address book; system displays saved addresses with labels and defaults. 2. Create new (name, street, city, postal code, country, state). 3. Assign type (shipping, billing, both) and optionally set default. 4. Optionally edit or remove; save; validates required fields and address format. , **Manage Wishlists** 1. On product detail, click Add to Wishlist. 2. Select or create named list; optionally add note; confirm. 3. Navigate wishlists; view all with counts and thumbnails. 4. Select list to view items, remove, rename, or delete. , **Manage Notifications** 1. Navigate preferences; system displays categories with per-channel opt-in status. 2. Toggle categories per channel; save.
Alternatives	1. A1. Same variant in wishlist → notify, suggest adding note. 2. A2. Remove address in active orders → warn referenced by past orders; soft-delete. 3. A3. Opt out all notifications → warn important suppressed; confirm. 4. A4. Delete wishlist → confirm; permanently removed.

Exceptions	1. E1. Address validation fails → highlight mismatch, prevent save. 2. E2. Variant archived → display Unavailable label, suggest removal. 3. E3. Persistence failure → report, retain unsaved changes.
Requirements	PRF-FR-01, PRF-FR-02, PRF-FR-03

2.2.6 System Use Cases

The System actor represents automated background processes that maintain data consistency, generate embeddings, and perform scheduled operations. Coordination relies on **Hangfire** [25] for job scheduling and **Redis** [22] for persistence.

2.2.6.1 Embedding Operations

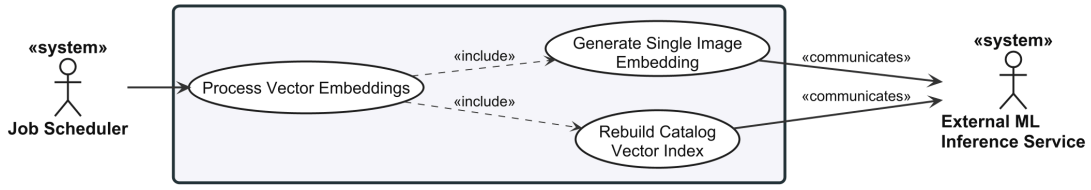

Figure 30: Use case diagram for Embedding Operations (UC-SYS-EMB).

2.2.6.2 UC-SYS-EMB: Embedding Operations

Table 49: Embedding Operations.

Use Case	UC-SYS-EMB — Embedding Operations
Actor	System
Support	ML Service
Goal	Generate and regenerate image embeddings for product search.
Pre/Post	Pre: images uploaded and stored; ML service operational. Post: embeddings stored in index for visual search.
Scenario	**Generate** 1. System detects new unprocessed image. 2. Retrieves file, preprocesses (resize, normalise). 3. Sends to ML service with model identifier. 4. ML service computes embedding, returns with metadata. 5. System stores embedding in index with image reference, marks processed. , **Regenerate All** 1. System detects model config change. 2. Validates new model available via health check. 3. Retrieves all product images, sends each to ML service.

	4. Stores new embeddings replacing previous, updates metadata. 5. Reports completion with success/failure counts.
Alternatives	1. A1. Image inaccessible → mark failed, record reason. 2. A2. Multiple images queued → process sequentially, throttle to ML service capacity. 3. A3. Peak traffic (Regenerate) → throttle to avoid impacting search performance. 4. A4. No model change detected → log event, skip.
Exceptions	1. E1. ML service error → retry with exponential backoff; after max retries, mark failed. 2. E2. ML service unreachable → requeue, trigger alert. 3. E3. New model unavailable (Regenerate) → abort; existing embeddings remain in use.
Requirements	CAT-FR-05, CAT-FR-15

The embedding generation process described in the main success scenario follows the pipeline illustrated in Figure 31: an image is authenticated, pre-processed through standardised transforms, passed through the selected model, and the resulting feature vector is serialised alongside performance metadata.

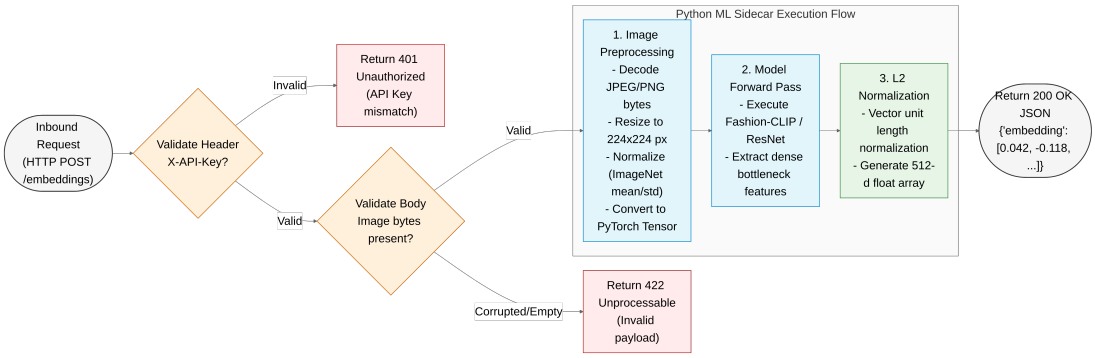

Figure 31: ML embedding pipeline: step-by-step flow from image upload to normalised vector response.

2.2.6.3 Background Maintenance

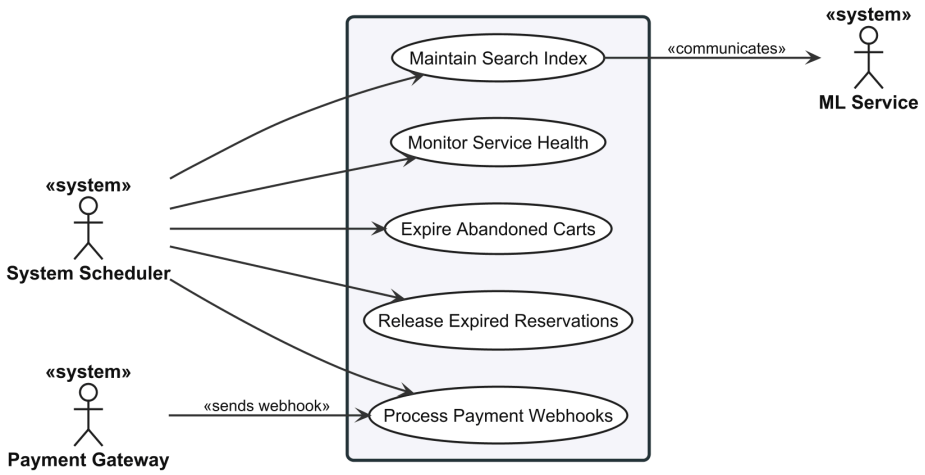

Figure 32: Use case diagram for Background Maintenance (UC-SYS-MNT).

2.2.6.4 UC-SYS-MNT: System Maintenance

Table 50: System Maintenance.

Use Case	UC-SYS-MNT — System Maintenance
Actor	System
Support	ML Service, Payment Gateway
Goal	Perform scheduled and event-driven system maintenance tasks.
Pre/Post	Pre: maintenance schedule active; required services operational. Post: system health, data consistency, and index performance maintained.
Scenario	**Health Check** 1. Poll ML service at configured interval. 2. Record status (healthy, degraded, unhealthy) and response time. 3. Update availability flag; unhealthy → route to degraded search path. , **Cart Expiry** 1. Scheduled job queries carts inactive 7 days. 2. For each: release reserved inventory, delete or mark for cleanup. 3. Log counts. , **Reservation Expiry** 1. Scheduled job queries holds exceeding configured expiry (15 min default). 2. Release quantity to available stock, mark expired. 3. Log counts. , **Payment Webhooks** 1. Gateway sends signed webhook. 2. Validate HMAC signature against secret.

	3. Extract payment id, event type, payload. 4. Check idempotency key, update payment state, trigger order transitions. 5. Return success to gateway. , **Index Maintenance** 1. Scheduled job analyses index state (vector count, size, query performance). 2. Determine if maintenance beneficial. 3. Rebuild or reindex if so; verify completion, log before/after metrics.
Alternatives	1. A1. No carts/items meet criteria → log zero. 2. A2. Duplicate webhook → return success without change; log. 3. A3. Out-of-order webhook → log anomaly, apply state change. 4. A4. Index below maintenance threshold → skip, log with metrics. 5. A5. Active search queries during maintenance → lighter non-blocking operations.
Exceptions	1. E1. Health check timeout → mark unhealthy, trigger alert. 2. E2. Database unavailable → record failure, retry next execution. 3. E3. Webhook signature invalid → reject with authentication error. 4. E4. Index maintenance fails → log error, trigger alert; search continues with current index. 5. E5. Index corrupted → trigger full rebuild from stored embeddings.
Requirements	CAT-FR-06, CAT-FR-08, ORD-FR-03, INV-FR-07, PAY-FR-04

2.3 SYSTEM ARCHITECTURE & DESIGN

Six dimensions define the platform architecture, from service composition through domain modelling to database, API, and security layers. The design follows a service-oriented approach: a Vue 3 frontend, a .NET 10 modular monolith backend, and a Python FastAPI machine learning sidecar, each independently deployable.

- **System Overview.** Three services, eight bounded contexts, technology stack summary.
- **Domain-Driven Design.** Context map, aggregate roots with invariants, state machines.
- **C4 Architecture.** Context, container, and component-level structural views.
- **Database Design.** Per-context schemas, pgvector integration, core design decisions.
- **API Design.** Carter minimal APIs, MediatR CQRS, endpoint conventions.
- **Security Design.** JWT authentication, permission-based authorisation, defensive hardening.

2.3.1 System Overview

ReSys.Shop comprises three independently deployable **services**: a Vue 3 **TypeScript** frontend, a .NET 10 **ASP.NET Core** backend [19], and a Python **FastAPI** machine learning sidecar running **PyTorch** [23] models. Table 51 summarises their technology stacks and responsibilities.

Table 51: System services and their technology stacks. Each service communicates through well-defined HTTP contracts.

Service	Technology Stack	Responsibilities
Vue Frontend	Vue 3 + TypeScript + Vite	• Customer storefront (Nuxt UI) • Administrator dashboard (PrimeVue) • Pinia state management • Image upload and visual search UI
.NET Backend	.NET 10 + ASP.NET Core + Carter + EF Core	• REST API endpoints via Carter minimal APIs • Business logic via MediatR CQRS pattern • PostgreSQL persistence with pgvector vector search • JWT authentication and RBAC authorisation
Python ML	Python 3.12 + FastAPI + PyTorch	• Fashion-CLIP and other embedding model inference • Vector embedding generation from product images • Multi-model support with lazy-loading strategy

Internally, the backend is partitioned into eight **bounded contexts** following **Domain-Driven Design** (DDD) principles. Each context owns a dedicated database schema and communicates with others exclusively through **MediatR** in-process dispatch – there are no direct namespace references between business modules. Table 52 lists each context, its aggregate root, and key domain entities.

Table 52: Bounded contexts with aggregate roots and key domain entities. Each context owns its database schema and communicates through MediatR dispatch only.

Bounded Context	Aggregate Root	Key Domain Entities
Catalog	Product	Variant, VariantImage, OptionType, OptionValue, Classification, Taxonomy, Taxon
Ordering	Order	LineItem, Adjustment, Shipment
Payment	PaymentIntent	PaymentCapture
Inventory	StockItem	StockLocation, StockMovement, StockReservation
Identity	User	Role, UserRole, RefreshToken, UserLogin
Profile	UserProfile	Address, Wishlist
Shipping	ShippingMethod	ShippingRate, ShippingZone

Bounded Context	Aggregate Root	Key Domain Entities
Location	Country	State
Dashboard	—	Read-only metrics aggregation (5 source files, no persistence layer)

This federated structure enables independent evolution of each domain while the **modular monolith** deployment model [15] avoids the operational overhead of distributed microservices. Domain modelling is detailed in Section 2.3.2.

2.3.2 Domain-Driven Design

The backend architecture is structured around eight **bounded contexts** adhering to **Domain-Driven Design** (DDD) principles. Each context manages explicit aggregate boundaries, enforces domain invariants, and uses a shared ubiquitous language across its domain operations. All nine contexts live within a single .NET assembly (`Module.csproj`); isolation is enforced by compile-time policy rather than separate projects. An eighth context, **Dashboard**, provides read-only metric aggregation with no persistence layer or domain entities (5 source files).

Context-to-context communication relies exclusively on **MediatR** `ISender` request/response dispatch. There is no event bus, no asynchronous messaging, and no `INotification` usage within the codebase. A context emits a command or query, and receiving contexts process the request without establishing direct compile-time project references [15].

2.3.2.1 Bounded Context Map

Figure 33 illustrates the eight primary contexts and their inter-module communication pathways. The **Published Language** defines shared DTO contracts in `Shared/Application/Contracts/{Module}/`. Each module implements its inter-module contracts as command/query handlers registered in `Features/Storefront/Contracts/{FeatureName}/`. Eleven contracts exist across modules: Inventory exposes `ReserveCartStock` and `CheckVariantAvailability`, Ordering exposes `GetCartForCheckout` and `AdvanceCheckoutState`, Payment exposes `GetPaymentForCheckout` and `MarkPaymentPaid`, and Catalog exposes `GetVariantWeights` and `GetVariantDiscontinuedStatuses`.

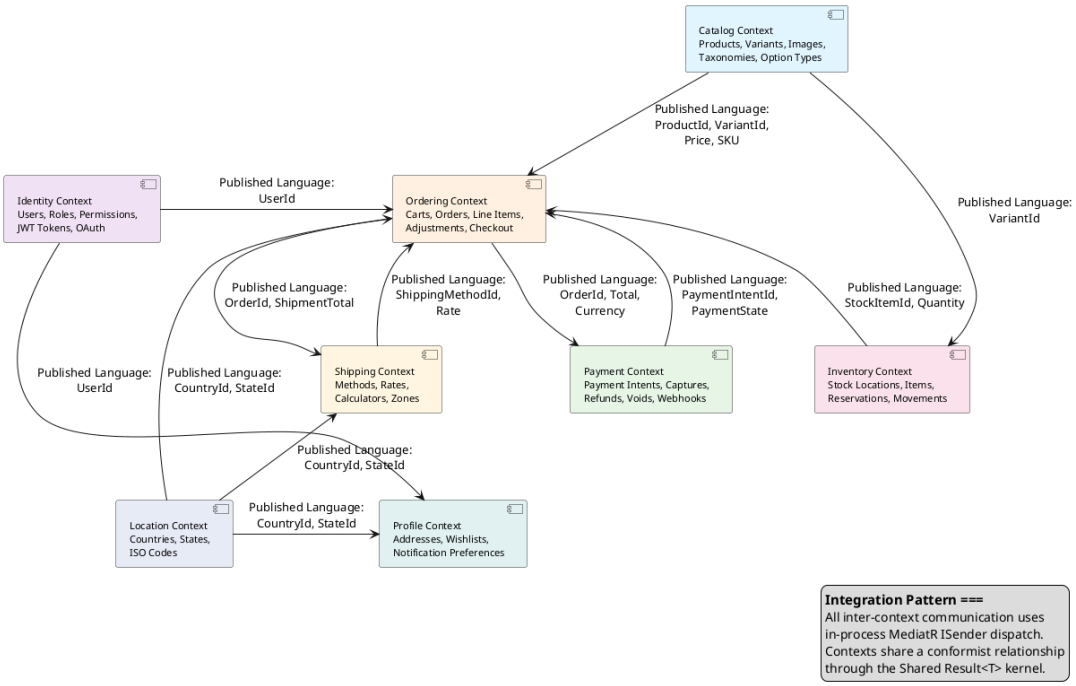

Figure 33: Bounded Context Map showing the eight business contexts and the Published Language identifiers exchanged between them. All integration uses in-process MediatR dispatch; no context directly references another context's namespace.

Table 53: Bounded context responsibilities and Published Language identifiers.

Context	Business Responsibility	Published Language
Catalog	• Product lifecycle: creation, update, archiving, and fashion metadata • Variants with SKUs, barcodes, dimensions, and independent pricing • Image management triggering automated vector embedding generation • Hierarchical taxonomy classification structures	`ProductId`, `VariantId`, `Sku`, `Price`, `Slug`
Ordering	• Customer purchasing workflow from cart through checkout completion • Cart management with automated 7-day inactivity purging • Forward-only checkout state machine execution • Line item capture with locked price snapshots and fee adjustments	`OrderId`, `OrderNumber`, `Total`, `Currency`, `CheckoutState`
Payment	• Payment intent lifecycles: creation, authorization, capture, refund, void	`PaymentIntentId`, `PaymentState`, `Amount`

Context	Business Responsibility	Published Language
	• Dual gateway abstractions (Stripe production and Bogus test environments) • Local payment state mirroring for offline operation and resilience	
Inventory	• Warehouse stock balance tracking across locations • Temporary inventory reservations for active checkouts • Append-only audit logging for stock adjustments	`StockItemId`, `QuantityOnHand`, `QuantityReserved`
Identity	• JWT-based authentication with single-use refresh token rotation • Refresh token reuse detection and session revocation • Role-Based Access Control enforcing granular resource-action permissions	`UserId`, `Email`, `PermissionClaim`
Profile	Customer address book management, wishlists, and communication preferences	`ProfileId`, `AddressId`
Shipping	Delivery method definition, geographic shipping rate evaluation, weight-based cost calculation	`ShippingMethodId`, `Rate`
Location	Reference data for countries and states via ISO 3166 standards	`CountryId`, `StateId`, `IsoCode`

2.3.2.2 Aggregates and Invariants

Four architecturally critical aggregate roots enforce domain invariants. Domain entities are decomposed into partial class files by concern per entity: `Entity.Constant.cs`, `Entity.Method.cs`, `Entity.Result.cs`, `Entity.Validation.cs`, `Entity.Loggers.cs`, keeping rich domain logic manageable per concern.

- **Product (Catalog):** Controls product families, variants, media assets, options, and taxonomy mappings. Enforces catalog-wide URL slug uniqueness and mandates exactly one master variant per product family. Variant images store vector embeddings for visual search.
- **Order (Ordering):** Coordinates purchasing pipelines from cart to completion. Locks line-item price snapshots at checkout to protect completed orders from catalog price updates. Enforces the structural balance invariant `ItemTotal + AdjustmentTotal + ShipmentTotal = Total`. Checkout progression is strictly forward-only.
- **PaymentIntent (Payment):** Models authorization across `Pending`, `RequiresAction`, `Processing`, `Succeeded`, `Canceled`, and `Failed` states. En-

forces capture limits preventing cumulative debits from exceeding authorized amounts.

- **StockItem (Inventory):** Tracks on-hand and reserved stock per location. Enforces non-negative balance, writing backorders to an isolated ledger. All stock adjustments append to an immutable `StockMovement` audit log.

2.3.2.3 State Machines

Explicit state machines inside domain entities govern checkout and payment workflows.

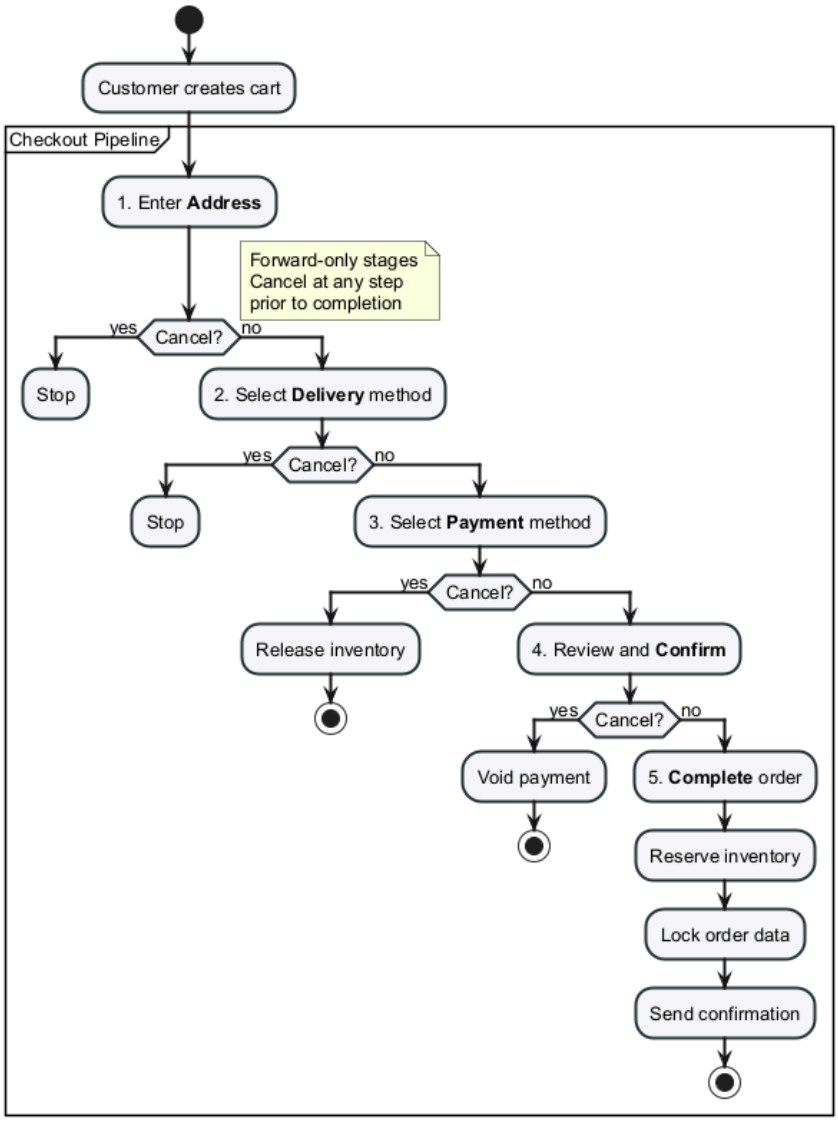

Figure 34: Order checkout decision flow: forward-only stages with cancellation at each step.

Checkout advances through Address, Delivery, Payment, Confirm, and Complete. Reaching Complete finalizes the order, reserves inventory, and locks data against edits.

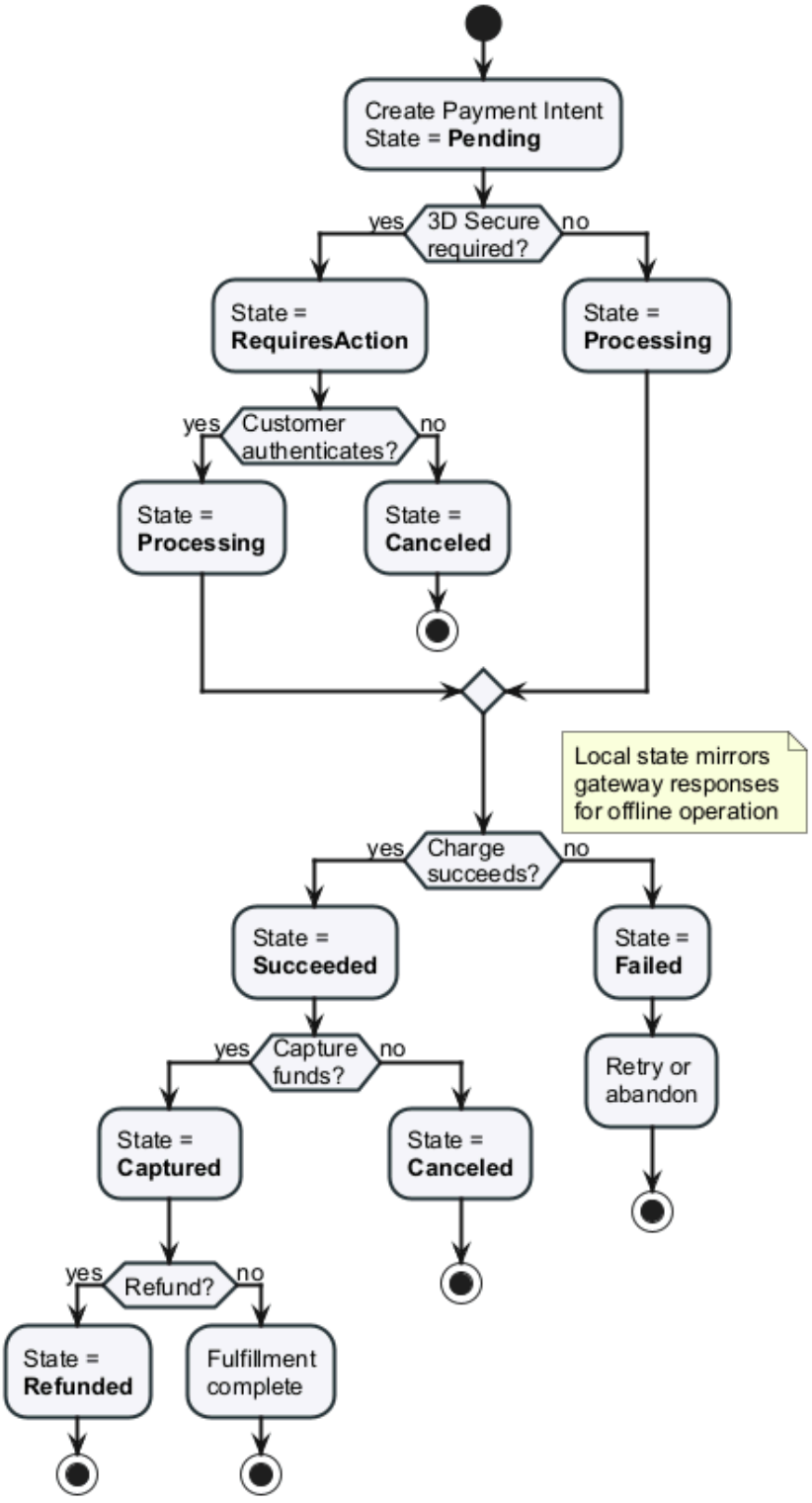

Figure 35: Payment intent decision flow: authorization branches and terminal outcomes.

Payment intents initialize as `Pending` and transition to `RequiresAction` or `Processing`. Final outcomes resolve to `Succeeded` or `Failed`. Mirroring gateway states locally enables offline test execution and resilience during network drops.

2.3.3 C4 Architecture

The C4 model structures software architecture across four abstraction levels. This section presents three C4 levels alongside a deployment view.

2.3.3.1 System Context

Figure 36 positions ReSys.Shop within its operational environment. The platform interacts with two human user groups: Customers browsing the catalog and completing checkouts via the Vue 3 storefront, and Administrators managing products, orders, inventory, and users via the Vue 3 admin interface. Five external integrations support operations: Stripe for payment processing, SendGrid for transactional emails, S3-compatible storage for product assets, Google OAuth for single sign-on, and the Python ML sidecar for embedding generation.

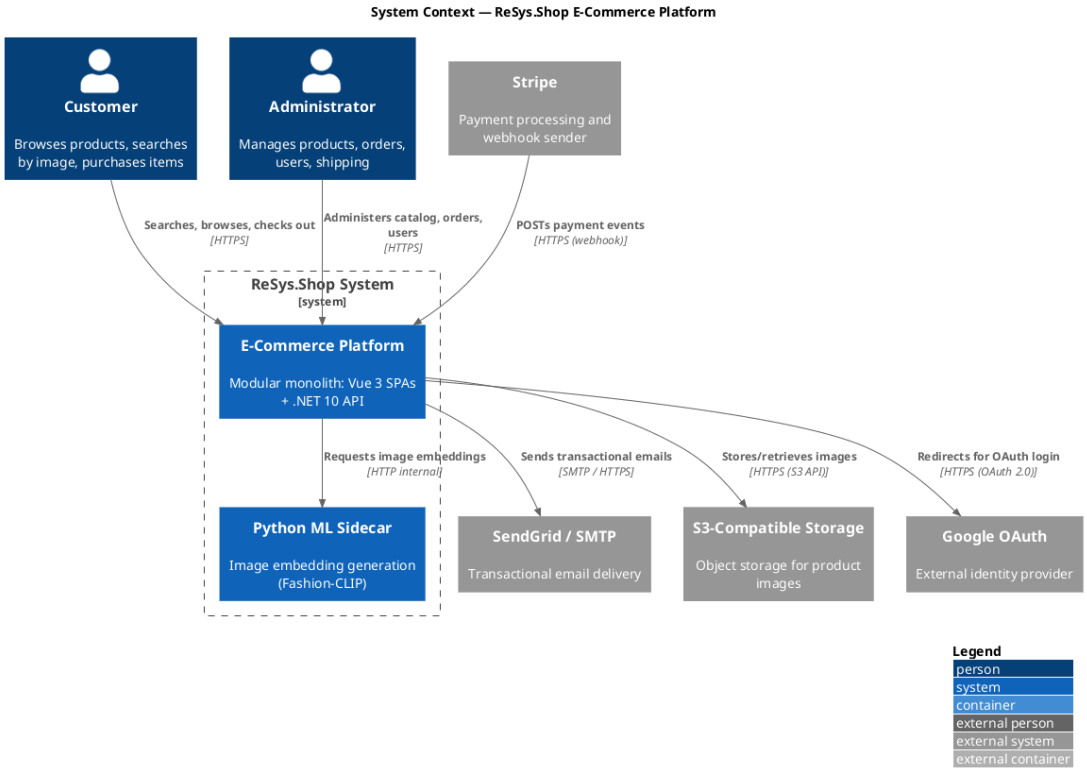

Figure 36: System Context diagram: ReSys.Shop system boundary with user roles and external dependencies.

2.3.3.2 Container

Figure 37 decomposes the platform into six deployable units: Vue 3 store and admin SPAs served as static assets, a .NET 10 API backend executing domain logic across nine modules, a Python 3.12 FastAPI sidecar generating image embeddings, PostgreSQL 17 with pgvector for relational and vector data, and Redis 7 for distributed caching and Hangfire job queues. Communication is centralized through the API Backend; SPAs never query databases or external APIs directly.

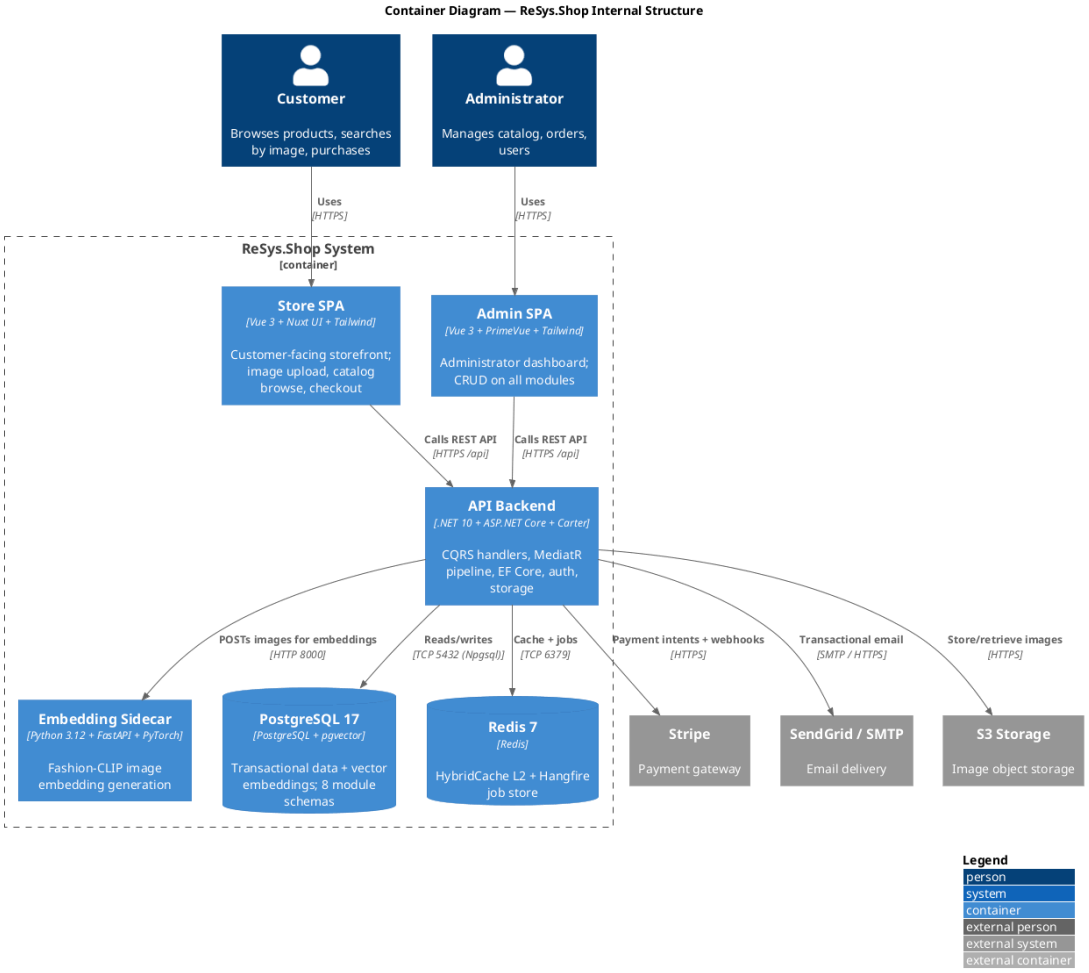

Figure 37: Container diagram: Vue 3 SPAs, .NET API backend, Python ML sidecar, PostgreSQL with pgvector, and Redis.

2.3.3.3 Component

Figure 38 details the API Backend internals. HTTP requests enter through Carter minimal API modules, which validate parameters and dispatch commands or queries via MediatR's `ISender`. Three pipeline behaviors execute in order:

Eight internal components handle infrastructure: `ApplicationDbContext` extending `IdentityDbContext` with EF Core interceptors for auditing, soft-

deletes, and optimistic concurrency; specification DSL providing composable `IQueryable` extensions for filtering, sorting, paging, and text search; identity provider managing JWT with token rotation and reuse detection; dynamic permission provider resolving resource-action claims at runtime; storage service with interchangeable local and S3-compatible backends plus ClamAV virus scanning; notification hub routing email via SendGrid/SMTP and SMS via Sinch with fallback routing; Hangfire engine executing background tasks for cart expiry, reservation cleanup, and webhook processing; and HybridCache combining L1 in-memory caching with L2 Redis caching for cross-instance consistency.

The Python ML sidecar uses a three-layer layout: FastAPI router for request validation, singleton model registry for lazy loading with ONNX and PyTorch backends, and interchangeable strategy interface supporting Fashion-CLIP, ResNet-50, EfficientNet-B0, and standard CLIP.

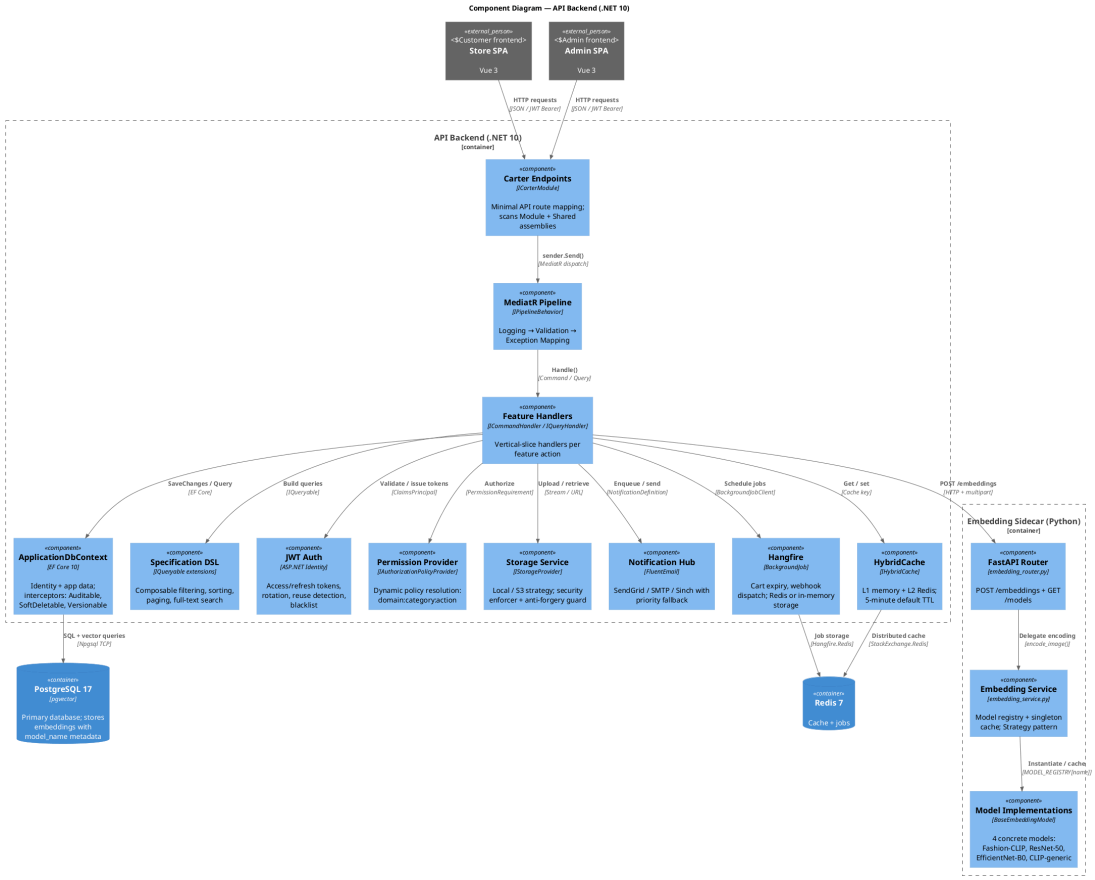

Figure 38: Component diagram: API Backend internal architecture and three-layer Python ML sidecar.

2.3.3.4 Deployment

Figure 39 shows the production infrastructure topology. The deployment environment is organized under a single .NET Aspire orchestration layer:

1. **Frontend tier.** Vue 3 storefront SPA and Vue 3 admin SPA deploy as static bundles to a CDN or reverse proxy, serving all public traffic over HTTPS.
2. **Application tier.** The .NET 10 API backend runs as Docker containers with horizontal scaling. Each replica hosts all nine business modules and communicates with the data tier over internal Docker networking. Replicas are stateless; session and cache state is externalized to Redis.
3. **Data tier.** PostgreSQL 17 runs as a primary instance for transactional writes, with optional read replicas for analytical queries. pgvector is enabled across all nodes. Redis 7 serves dual roles: L2 cache backend for `HybridCache` and persistent job store for Hangfire background processing.
4. **ML tier.** The Python FastAPI sidecar runs as independently scaled Docker containers. Since embedding generation is stateless (the model registry caches model weights per container, not per request), the sidecar scales linearly with query volume. Container health is monitored by Aspire readiness probes; unhealthy containers are automatically restarted.
5. **Secrets and configuration.** External API keys (Stripe, SendGrid, Google OAuth) and database connection strings are injected via Aspire configuration environments at runtime, never committed to source control or container images.
6. **Service discovery.** Aspire-managed DNS resolves components by name: the .NET backend reaches the Python sidecar at `http://ml-service`, PostgreSQL at `postgres`, and Redis at `redis`. All inter-component communication stays within the Docker network; only the CDN and Stripe webhook receive public internet traffic.

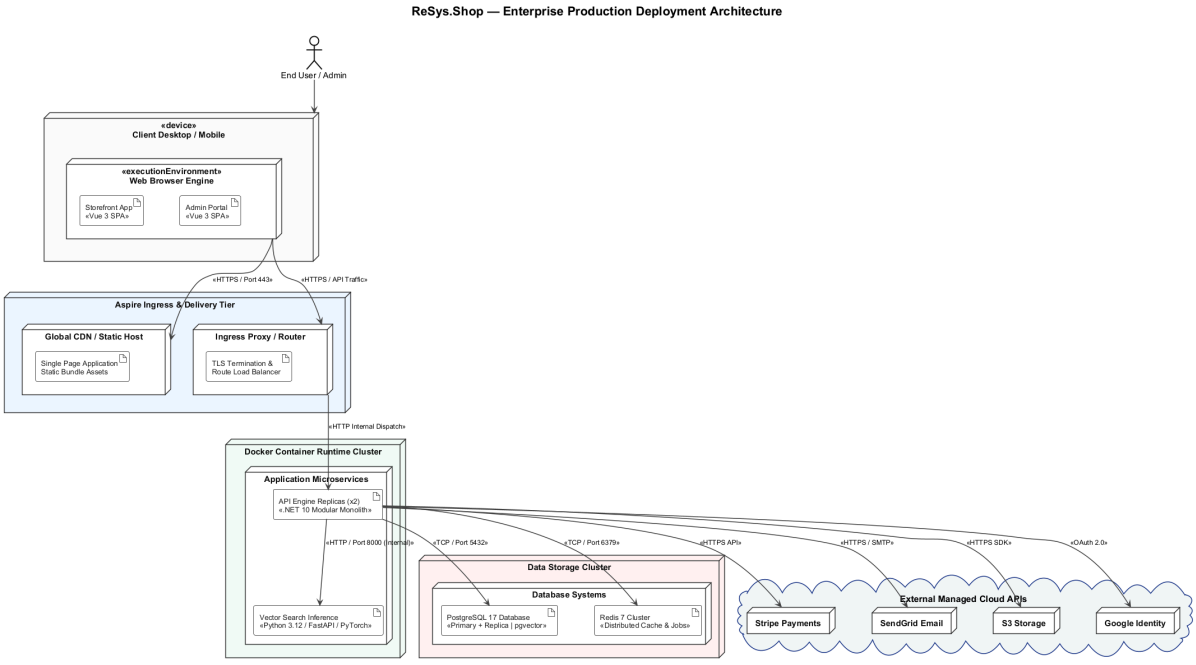

Figure 39: Deployment topology: containerized orchestration under .NET Aspire with horizontal scaling.

2.3.4 Database Design

The database consists of a single PostgreSQL 17 instance partitioned into per-context schemas. Each schema is owned by a bounded context and managed via Entity Framework Core migrations. A single `ApplicationDbContext` extending `IdentityDbContext` registers module-specific entity configurations via `AdditionalConfigurationsAssemblies` discovery.

2.3.4.1 Schema Organisation

Each bounded context manages its own schema: `catalog`, `identity`, `ordering`, `payment`, `inventory`, `shipping`, `location`, `profile`. The Dashboard context does not own a schema; it is a read-only aggregation compiled at query time. Cross-context relationships use identifier references (UUIDs) without database-level foreign key constraints, maintaining module isolation while avoiding distributed transaction overhead.

Five EF Core migrations track schema evolution: `InitialCreate` establishes the full schema with pgvector extension, `AddEmbeddingStatusColumns` tracks per-image embedding state, `MakeEmbeddingVectorNullable` relaxes vector constraints, `ChangeSourceIdToString` adjusts identifier types, and `AddShippingMethodZones` extends the shipping schema.

Figure 40 illustrates the entity-relationship model across all eight contexts.

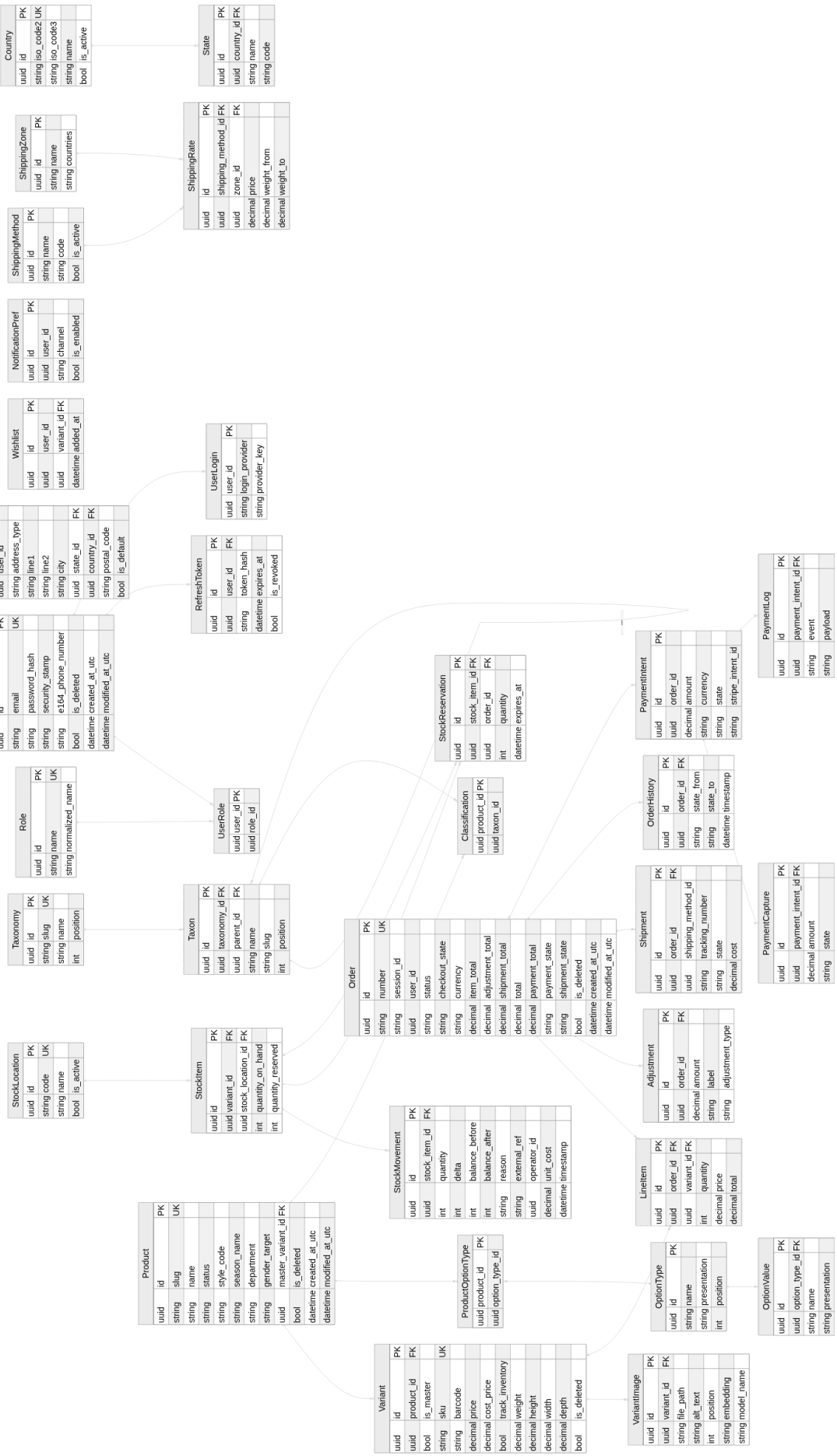

Figure 40: Core entity-relationship model across eight bounded contexts.

The Catalog domain centers on `Product` linking one-to-many with `Variant`. Each variant stores SKU, pricing, and inventory flags, with one designated as the master variant. `VariantImage` records hold image paths and an optional `vector(512)` embedding column. Taxonomy and Taxon trees manage hierarchical product classification via self-referencing foreign keys. The Ordering domain centers on `Order`, linking one-to-many with `LineItem`. Line items reference specific variants and capture price snapshots at purchase time to decouple order history from catalog updates.

2.3.4.2 pgvector Integration

PostgreSQL's pgvector extension [13] executes vector similarity searches directly within the relational engine. The `product_image_embeddings` table stores feature vectors in a column of type `vector(512)`:

```
-- Embedding column definition
CREATE TABLE product_image_embeddings (
    id          UUID PRIMARY KEY,
    image_id    UUID NOT NULL REFERENCES variant_images(id),
    model_name  TEXT NOT NULL,
    embedding   VECTOR(512),
    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- IVFFlat index for approximate nearest neighbor search
CREATE INDEX idx_product_image_embeddings_ivfflat
    ON product_image_embeddings
    USING ivfflat (embedding vector_cosine_ops)
    WITH (lists = 100);
```

Query results use the `<=>` cosine distance operator, ranking by similarity score `1 - cosine_distance` with a configurable default threshold of `0.70`. Every embedding record includes a `model_name` string, and search queries filter by the active model to enforce strict mathematical alignment across visual embeddings.

```
-- Cosine distance similarity search
SELECT id, image_id, model_name,
       1 - (embedding <=> @query_vector) AS similarity
FROM product_image_embeddings
WHERE model_name = @active_model
  AND 1 - (embedding <=> @query_vector) >= 0.70
ORDER BY embedding <=> @query_vector
LIMIT 20;
```

The platform defaults to HNSW indexing [12] for production workloads. IVFFlat serves as the fallback for rapid evaluation; at 5,000 vectors, it builds nearly instantaneously (under one second) versus the minutes required for HNSW graph construction, and achieves sub-10-millisecond query latency at 65–72% recall@10 (see Appendix A.4). The production architecture designates HNSW for larger catalogue scales where its superior recall-speed trade-off becomes decisive.

2.3.4.3 Key Design Decisions

The data layer adheres to five global rules: UUID primary keys for safe client-side key generation and resistance to sequential enumeration; soft deletion with an `IsDeleted` boolean column filtered globally by EF Core query interceptors; automatic `CreatedAtUtc` and `ModifiedAtUtc` timestamps via EF Core save interceptors; targeted composite indexes on high-frequency access paths such as `(UserId, Status)` on orders; and variable vector dimensions per model architecture, ranging from 384 (DINOv2 ViT-S/14), through 512 (Fashion-CLIP, CLIP variants), 768 (DINOv2 ViT-B/14, SigLIP), 1280 (EfficientNet-B0), to 2048 (ResNet-50), with embeddings from different models indexed independently per dimension.

2.3.5 API Design

The API exposes a RESTful interface on .NET 10 minimal APIs via **Carter** modules, dispatched through the **MediatR** CQRS pattern [18]. The platform registers approximately 262 endpoints across nine business modules. Routes are declared in `Features/Shared/{Module}Feature.{Surface}.cs` static classes, one per admin and storefront boundary.

2.3.5.1 API Architecture

Each request follows a standard path through three MediatR pipeline behaviors:

```
HTTP Request
  → Carter Endpoint (MapPost/MapGet/MapPut/MapDelete)
    → LoggingBehavior        - logs request/response
    → ValidationBehavior     - FluentValidation via
IValidator<TRequest>
    → ExceptionMappingBehavior - maps exceptions to Result errors
    → Handler.Execute()      - domain logic in bounded context
  → Result<T>.ToResult()     - maps to HTTP status code
  → HTTP Response (RFC 7807 Problem Details on error)
```

A representative endpoint pattern:

```
// Carter endpoint pattern (Catalog/Features/Admin/Products/
Create/CreateProduct.Endpoint.cs)
public class Endpoint : ICarterModule
{
    public void AddRoutes(IEndpointRouteBuilder app)
    {
        app.MapPost(CatalogFeature.Admin.Products.Create.Route,
async (
            [FromBody] Request request,
            ISender sender,
            CancellationToken ct) =>
        {
            var command = new Command(request);
            var result = await sender.Send(command, ct);
            return result.ToResult();
        })
        .WithName(nameof(CreateProduct))
```

```
        .HasPermission(CatalogFeature.Admin.Products.Create.Permission)
           .Produces<Result<Response>>()
           .Produces<Result>(StatusCodes.Status400BadRequest);
    }
}
```

Endpoints receive the HTTP request, extract parameters, construct a MediatR command/query, and dispatch via `ISender`. Handlers operate within domain isolation on application models without HTTP dependencies, returning `Result<T>`. Endpoints map success to HTTP 200, 201, or 204, and domain errors to RFC 7807 Problem Details.

2.3.5.2 Endpoint Organisation

Endpoints follow the URL convention `/api/{module}/{surface}/{resource}`, where `module` is the owning bounded context, `surface` is `storefront` or `admin`, and `resource` identifies the domain resource and sub-action. This structure provides self-documenting URLs and surface-level authorization boundaries. All `admin` routes enforce administrator policies globally via the `.HasPermission()` attribute.

Eleven inter-module contracts provide the Published Language DTOs for cross-module communication. Each module implements its contracts as command/query handlers under `Features/Storefront/Contracts/{FeatureName}/`:

```
// Inter-module contracts (Shared/Application/Contracts/)
Inventory  → ReserveCartStock, ReleaseCartStockReservations,
             ConsumeCartStockReservations,
CheckVariantAvailability
Ordering   → GetCartForCheckout, GetCartForShipping,
AdvanceCheckoutState
Payment    → GetPaymentForCheckout, MarkPaymentPaid
Catalog    → GetVariantDiscontinuedStatuses, GetVariantWeights
```

2.3.5.3 API Endpoint Contract

Table 54 summarises the registered Carter endpoints.

Table 54: ReSys.Shop API endpoint contract: approximately 262 Carter endpoints across nine business modules.

Module	Admin Routes	Storefront Routes	N
Catalog	Products CRUD, variants, variant images, option types/values, taxonomies, taxons, taxon rules, classifications, pricing, dashboard	Product listing/search, product detail by slug, availability, related/similar products, CBIR search-by-image, taxonomy tree, taxon browsing, option types, image display	80

Module	Admin Routes	Storefront Routes	N
Identity	Users CRUD, user roles/permissions, roles CRUD, role permissions, permissions catalogue	Password/Google login, register, logout, session refresh, email confirm/change, password change/forgot/reset	37
Ordering	Orders CRUD, line items, order status/cancel/complete/approve/resume, shipping/billing address, shipping method, dashboard	Cart CRUD, cart items add/update/remove, cart associate, empty, checkout, validate, shipping rate, customer orders list/detail/cancel	35
Inventory	Stock locations CRUD, stock items CRUD, bulk adjust/import, restock, low stock, summary, reservations, transfers, movements, dashboard	Variant availability, cart reserve/release/list	32
Profile	Profiles CRUD, addresses CRUD	Profiles, addresses CRUD, notification preferences, wishlists CRUD with items	27
Location	Countries CRUD (by ID or ISO code), states CRUD (by ID or ISO code)	Countries browse, states browse (by ID or ISO code)	18
Payment	Payment methods CRUD, activate/deactivate, payments list/detail, capture/void/refund	Create payment intent, confirm payment, available methods, setup intent, Stripe webhook	17
Shipping	Shipping methods CRUD, activate/deactivate, shipping rates CRUD	Available methods, calculate cost, rates list	15
Dashboard	Aggregated metrics: sales, inventory, catalog, activity	–	1

2.3.5.4 Error Handling

All API endpoints conform to RFC 7807 Problem Details: HTTP 400 for FluentValidation failures with field-level mappings; HTTP 401 for missing or expired JWT; HTTP 403 for insufficient permission scope; HTTP 404 for entity or route resolution failure; HTTP 409 for optimistic concurrency control failures via PostgreSQL `xmin`; and HTTP 500 handled by global exception middleware preventing stack trace leakage while logging full detail to telemetry.

2.3.6 Security Design

The security framework operates across three layers: authentication, authorization, and defense-in-depth infrastructure.

2.3.6.1 Authentication and Session Management

JWT authentication is configured via `JwtSettings` with HS256 algorithm, 15-minute access token expiration, and 30-day maximum token age. Single-use refresh token rotation is enforced: exchanging an expired access token consumes the current refresh token and issues a new pair. Re-submitting a previously consumed refresh token triggers breach detection, immediately revoking all active refresh tokens for that user and forcing full re-authentication. Unauthenticated shoppers receive an HTTP-only cookie tracking an anonymous session ID; on login, the guest cart automatically merges with the user's persistent cart.

```
// JWT configuration (JwtSettings)
public sealed class JwtSettings
{
    public string Secret { get; init; }
    public string Issuer { get; init; }
    public string Audience { get; init; }
    public int AccessTokenExpirationInMinutes { get; init; } =
15;
    public int RefreshTokenExpirationInDays { get; init; }
    public string Algorithm { get; init; } = "HS256";
    // Token security
    public bool RotationEnabled { get; init; } = true;
    public bool ReuseDetectionEnabled { get; init; } = true;
    public bool SlidingExpirationEnabled { get; init; } = true;
    public int MaxTokenAgeDays { get; init; } = 30;
}
```

2.3.6.2 Dynamic Authorization

Role-Based Access Control separates administrative and storefront surfaces. Unprivileged accounts accessing `/api/*/admin/*` endpoints receive an immediate HTTP 403 prior to command dispatch. The built-in Administrator role bypasses all permission checks.

A three-layer permission architecture resolves claims at runtime:

```
// Permission resolution pipeline
// L1: IPermissionCache      - in-memory cache of resolved
permissions per user
// L2: IPermissionService    - merges role-derived + direct-grant
permissions
// L3: IPermissionStore      - EF Core persistence to Identity
claims tables

// Custom policy provider resolves permission names dynamically
public sealed class PermissionPolicyProvider :
IAuthorizationPolicyProvider
{
    public Task<AuthorizationPolicy> GetPolicyAsync(string
policyName)
    {
        if (PermissionContext.IsKnown(policyName))
            return Task.FromResult(new
AuthorizationPolicyBuilder()
                .AddRequirements(new
PermissionRequirement(policyName))
```

```
                .Build());
        return FallbackProvider.GetPolicyAsync(policyName);
    }
}
```

Operations enforce granular resource-action claims. Ten `FeatureMetadata` files define the permission registry across modules, each mapping to the `PermissionContext` static catalogue. Permissions use the format `Domain.Category.Resource.Action`:

```
catalog.products.create
catalog.products.update
catalog.variants.delete
identity.roles.manage
ordering.orders.approve
payment.intents.capture
inventory.stock.transfer
```

A custom `IAuthorizationPolicyProvider` resolves claim strings to policies dynamically at runtime, allowing permission modifications in the database without application redeployments.

2.3.6.3 System Hardening

Rate limiting restricts authentication to 5 requests per minute, registration to 3 per hour, and payment processing to 30 per minute. Security middleware injects `Content-Security-Policy`, `Strict-Transport-Security`, `X-Frame-Options`, and `X-Content-Type-Options` headers. Visual search uploads enforce a 10 MB limit and validate magic bytes to verify valid JPEG, PNG, or WebP formats. Stripe payment webhooks validate the `Stripe-Signature` header using HMAC signature verification before executing state transitions.

The storage service includes a pre-upload virus scanning layer via ClamAV (nClam integration) and image format validation through SkiaSharp, preventing malicious payloads from reaching persistent storage.

2.4 IMPLEMENTATION

This section describes the concrete realization of the ReSys.Shop system, showing how the architecture and design decisions from Sections 2.2 and 2.3 were translated into working software. The presentation follows the system's actual structure: the technology stack that underpins development, the vertical slice pattern that realization the .NET codebase, the persistence layer that stores relational and vector data in a single database, the machine learning sidecar that constitutes the core research contribution, and the frontend applications that deliver the user-facing experience.

- **Technology Stack.** Framework versions, Aspire orchestration topology, and containerisation strategy.

- **Vertical Slice Core.** Feature co-location, the Carter–MediatR request pipeline, and the functional Result pattern.
- **Data Persistence.** Multi-schema EF Core, pgvector integration with HNSW indexing, and concurrency control.
- **ML Sidecar.** Model management, the embedding generation pipeline, and the end-to-end CBIR search flow.
- **Frontend Applications.** Dual-SPA architecture, visual search interface, and key administration workflows.

2.4.1 Technology Stack

The platform uses a version-pinned technology stack across three runtime ecosystems, each selected for domain alignment. .NET 10 provides strongly typed transactional semantics and enterprise API abstractions. Python 3.12 provides access to the deep learning ecosystem (PyTorch, Hugging Face). Vue 3 delivers component-driven reactive user interfaces.

Centralized package management enforces build reproducibility: `Directory.Packages.props` for NuGet, `uv.lock` for Python, and `pnpm-lock.yaml` for JavaScript.

Table 55 details the core technologies grouped by architectural role.

Table 55: Principal technologies grouped by architectural role with pinned version specifications.

Architectural Role	Technology & Version
Backend Runtime	.NET 10 / ASP.NET Core 10.0.9 (C# 13)
API Framework	Carter 10.0.0, MediatR 14.1.0, FluentValidation 12.1.1
Object Mapping	Mapster 10.0.9
ORM & Database	EF Core 10.0.9, Npgsql 10.0.2, pgvector 0.3.2
Caching Layer	HybridCache 10.6.0, StackExchange.Redis 10.0.9
Background Jobs	Hangfire 1.8.23 (Redis-backed)
Observability	OpenTelemetry 1.16.0
ML Runtime	Python 3.12, PyTorch >= 2.0.0, TorchVision >= 0.15.0
ML Framework	FastAPI >= 0.115.0, Uvicorn, Hugging Face Transformers
ONNX Runtime	onnxruntime >= 1.17.0
Storefront UI	Vue 3.5, TypeScript 6.0, Vite 8, PrimeVue 5 (Aura)
Admin UI	Vue 3.5, TypeScript 6.0, Vite 8, PrimeVue 5 (Sakai), Chart.js 4

Architectural Role	Technology & Version
State & HTTP	Pinia, Axios 1.x, Zod
Data Storage	PostgreSQL 17 (`pgvector/pgvector:pg17-trixie`)
Cache Storage	Redis 7 (Alpine)
Orchestration	.NET Aspire 13.4.6
Test Suites	xUnit v3 3.2.2, pytest >= 8.0, Vitest 4, Playwright

2.4.1.1 Service Orchestration and Containerization

- **Aspire Topology:** The AppHost programmatically defines six system resources with strict startup dependencies: PostgreSQL and Redis initialize first, followed by the Python ML sidecar (validated via its `/health` readiness probe), before the .NET API accepts traffic.

  Connection strings and service URLs are injected dynamically via environment variables. The Vue SPAs run as Vite dev servers with hot module replacement, reverse-proxying `/api/` endpoints to the backend.

- **Multi-Stage Container Builds:** The ML sidecar Dockerfile uses a multi-stage build (`uv sync --frozen`) and a minimal runtime stage executing under a non-root user (`inference`, UID 1001) using `tini` for process signal forwarding.

  The .NET API compiles as a self-contained container via `dotnet publish`. The Vue SPAs build to static assets served by an Nginx reverse proxy.

2.4.2 Vertical Slice Architecture Core

The .NET backend implements Vertical Slice Architecture (VSA) combined with Command Query Responsibility Segregation (CQRS), organizing application code by business capability rather than technical layer. Each feature co-locates its request DTOs, domain execution logic, validation constraints, Carter HTTP endpoints, and response models within a single directory using C# `static partial class` definitions.

2.4.2.1 Feature Co-Location and Execution Mechanics

Every feature is structured as a `static partial class` split across five dedicated files. A representative feature, `CreateProduct` within the Catalog context, illustrates this organizational model:

```
Admin/Products/Create/
├── CreateProduct.cs             # Command and CommandHandler
logic
├── CreateProduct.Request.cs     # Input Request DTO
├── CreateProduct.Response.cs    # Output Response DTO
├── CreateProduct.Endpoint.cs    # Carter Minimal API route
mapping
└── CreateProduct.Validator.cs   # FluentValidation rule
definitions
```

Listing 1: Co-located file layout for a single vertical slice feature.

The feature handler executes through a standardized six-step pipeline using MediatR's `ISender` for cross-module dispatch:

Table 56: Sequential execution flow within the CreateProduct command handler.

Step	Phase	Action Description
Step 1	Validation	Checks product slug uniqueness against PostgreSQL; returns a `DuplicateSlug` domain error on collision.
Step 2	Instantiation	Constructs the `Product` entity via a domain factory method returning `Result<Product>`.
Step 3	Persistence	Appends the entity to `IApplicationDbContext` and saves changes to commit the transaction.
Step 4	Dispatch	Issues a cross-module `AddVariant` command via `ISender` to initialize the master SKU.
Step 5	Association	Assigns the generated master variant ID back to the `Product` aggregate and updates state.
Step 6	Completion	Returns `Result<Response>.Created(...)` containing the mapped response payload.

Compile-Time Module Isolation: Inter-module communication relies exclusively on `ISender.Send()` messages. Bounded contexts never import foreign context namespaces, establishing strict boundaries within a single assembly enforced via static analysis and build policies.

```
public sealed record Command(Request Request) :
ICommand<Response>;

public sealed class CommandHandler(
    IApplicationDbContext dbContext,
    ISender sender)
    : ICommandHandler<Command, Response>
{
    public async Task<Result<Response>> Handle(
        Command command, CancellationToken ct)
    {
        // Step 1: Validate slug uniqueness
        if (await dbContext.Set<Product>()
```

```
            .AnyAsync(x => x.Slug == command.Request.Slug, ct))
            return ProductResult.Errors.DuplicateSlug;

        // Step 2-3: Instantiate domain entity and persist
        var product = command.Request.MapToDomain().Value;
        dbContext.Set<Product>().Add(product);
        await dbContext.SaveChangesAsync(ct);

        // Step 4-5: Dispatch cross-module command for master
variant
        var variantResult = await sender.Send(
            new AddVariant.Command(product.Id, variantRequest),
ct);
        product.MasterVariantId = variantResult.Value.Id;
        await dbContext.SaveChangesAsync(ct);

        // Step 6: Return success response wrapper
        return Result<Response>.Created(
            product.MapToDetail<Response>(),
            ProductResult.Success.Created(product.Id));
    }
}
```

Each feature's endpoint participates in a shared request pipeline that applies cross-cutting concerns (validation, logging, and exception handling) uniformly across all features.

2.4.2.2 Request Pipeline Architecture

2.4.2.2.1 Carter Endpoint Registration

Each vertical slice implements `ICarterModule` to define its HTTP interface. Endpoints remain intentionally thin: parsing incoming requests, constructing MediatR command objects, dispatching via `ISender`, and converting functional `Result<T>` outputs into HTTP responses. All 257 API endpoints across the platform follow this pattern:

```
public class Endpoint : ICarterModule
{
    public void AddRoutes(IEndpointRouteBuilder app)
    {
        app.MapPost(CatalogFeature.Admin.Products.Create.Route,
            async ([FromBody] Request request,
                   ISender sender, CancellationToken ct) =>
        {
            var result = await sender.Send(new Command(request),
ct);
            return result.ToResult();
        })
      .HasPermission(CatalogFeature.Admin.Products.Create.Permission)
        .Produces<Result<Response>>()
        .Produces<Result>(StatusCodes.Status400BadRequest);
    }
}
```

Routes follow a structural hierarchy: `/api/{module}/{surface}/{resource}` (where `surface` is designated as `admin` or `storefront`). Carter modules are auto-discovered at runtime via assembly scanning during application startup.

2.4.2.2.2 MediatR Middleware Behaviors

Three cross-cutting behaviors wrap every command and query in a nested execution model. They are registered from outermost to innermost, forming a layered pipeline where each behavior delegates to the next:

```
cfg.AddBehavior(typeof(IPipelineBehavior<,>),
typeof(LoggingBehavior<,>));
cfg.AddBehavior(typeof(IPipelineBehavior<,>),
typeof(ValidationBehavior<,>));
cfg.AddBehavior(typeof(IPipelineBehavior<,>),
typeof(ExceptionMappingBehavior<,>));
```

When a request is dispatched via `ISender`, it passes through each behavior in order:

- **LoggingBehavior** (outermost) captures the request type, then delegates inward.
- **ValidationBehavior** runs co-located FluentValidation rules. On failure, it returns `List<Error>` immediately, short-circuiting both the inner behavior and the handler.

- **ExceptionMappingBehavior** (innermost) delegates to the handler. It catches any unhandled exception, wraps it in `Error.FromException()`, and returns a structured failure instead of letting the exception propagate.

On the return path, `LoggingBehavior` inspects the response and records success or failure. The final `Result<T>` propagates back through Carter to the HTTP client. The handler never references logging, validation, or exception handling; these concerns are applied transparently by the pipeline.

Table 57: MediatR pipeline behavior execution sequence and responsibilities.

Pipeline Behavior	Execution Order	Operational Responsibility
`LoggingBehavior`	Outermost	Logs request type metadata, execution duration, and completion status upon pipeline exit.
`ValidationBehavior`	Middle	Executes co-located FluentValidation rules. Rejects invalid requests early by returning a `List<Error>` payload without invoking downstream handlers.
`ExceptionMappingBehavior`	Innermost	Catches unhandled execution exceptions, wraps them in standard `Error.FromException()` models, and prevents internal stack trace leakage.

2.4.2.3 Functional Result Pattern and Error Handling

The application avoids exception-driven control flow in favor of a functional `Result<T>` pattern. Domain methods return explicitly typed `Result<T>` (for operations returning values) or `Result` (for void operations). Both structures are implemented as memory-efficient `readonly record struct` types:

```
public readonly partial record struct Result<T> : IResultRecord
{
    [AllowNull] public T Value { get => field!; init; }
    public bool IsSuccess { get; init; }
    public int StatusCode { get; init; }
    public List<Error> Errors { get; init; }
    public bool IsFailure => !IsSuccess;
}
```

- **Domain Error Abstraction:** The `Error` struct encapsulates a machine-readable `Code`, human-readable `Message`, integer type discriminator, and an optional metadata dictionary. Specialized factory helpers generate standard error categories (`Error.BadRequest`, `Error.NotFound`, `Error.Conflict`, `Error.Validation`).

- **Implicit Conversions:** Returning a domain object implicitly wraps it in a successful `Result<T>` instance. Returning an `Error` or `List<Error>` automatically casts into a failed `Result<T>`.
- **HTTP Mapping:** The `ToResult()` extension translates domain results directly to standard HTTP status codes: **200 OK** or **201 Created** for success paths, **400 Bad Request** for validation failures, **404 Not Found** for missing entities, and **409 Conflict** for state violations.

2.4.3 Data Persistence Architecture

The persistence layer maps the eight bounded contexts to dedicated PostgreSQL schemas, co-locating high-dimensional vector embeddings with relational product data in a single PostgreSQL 17 database instance.

2.4.3.1 Schema Organisation and Vector Storage

Each bounded context owns an isolated database schema containing its aggregate roots and child entities. Table 58 outlines the schema-per-context distribution:

Table 58: Schema-per-context mapping. Cross-schema references use UUID attributes without database-level foreign key constraints.

Schema	Principal Tables & Aggregate Entities
`catalog`	`products`, `variants`, `variant_images`, `product_image_embeddings`, `taxonomies`, `taxons`
`ordering`	`orders`, `line_items`, `order_adjustments`, `shipments`
`payment`	`payment_intents`, `payment_captures`, `payment_methods`
`inventory`	`stock_locations`, `stock_items`, `stock_movements`, `stock_reservations`
`identity`	`users`, `roles`, `user_roles`, `refresh_tokens`
`profile`	`user_profiles`, `addresses`, `wishlists`
`shipping`	`shipping_methods`, `shipping_rates`, `shipping_zones`
`location`	`countries`, `states`

Schema boundaries mirror the C# compile-time isolation rules. An order entity references a `variant_id` as a unconstrained UUID rather than a database foreign key constraint, enforcing logical module independence without distributed transaction overhead.

Co-located vector storage. Product embeddings reside within `catalog.product_image_embeddings` alongside product metadata. Co-locating relational data and vectors inside PostgreSQL eliminates dual-database synchro-

nization issues: vector updates participate in the primary database transaction, guaranteeing ACID consistency without external sync workers.

2.4.3.1.1 pgvector Indexing Strategy

Image embedding vectors are stored in a `vector(512)` column representing 512-dimensional latent spaces from vision-language models. The persistence layer supports two Approximate Nearest Neighbor (ANN) index types configured for different operational environments:

Table 59: Comparison of pgvector ANN index algorithms using the cosine distance operator (<=>).

Attribute	HNSW (Hierarchical Navigable Small World)	IVFFlat (Inverted File Flat)
Index Structure	Multi-layer navigable graph	K-means cluster partitioning
Build Overhead	Memory-intensive (several minutes)	Low memory footprint (< 1 s)
Query Latency	Sub-millisecond (< 5 ms, logarithmic)	Fast (< 10 ms, cluster-dependent)
Deployment	Production environment	Benchmark iteration & constrained testing
Parameters	$m = 16, \text{ef}_{\text{construction}} = 64$	lists $= 100$

The production HNSW index DDL is executed during migration initialization:

```
CREATE INDEX ix_product_image_embeddings_vector_hnsw
ON catalog.product_image_embeddings
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

2.4.3.1.2 Model-Aware Vector Search Pipeline

Vector embeddings from different neural architectures inhabit incompatible vector spaces. To prevent cross-model distance pollution, every record persists a `model_name` discriminator (e.g., `"Fashion-CLIP"`). Cosine similarity search follows a four-stage query execution process:

1. **Candidate Retrieval:** Executes a cosine distance query using the <=> operator to fetch the top $3 \times K$ nearest neighbors filtered by `model_name`.
2. **Similarity Transformation:** Converts distance to similarity metrics via similarity $= 1 -$ cosine_distance.
3. **Threshold Filtering:** Discards candidates failing the minimum confidence threshold (similarity ≥ 0.70).

4. **Product Deduplication:** Group-by filtering reduces multiple matching variant images down to the highest-scoring master product entity.

2.4.3.2 Concurrency and Data Integrity

Data integrity under high concurrent throughput is maintained using three complementary concurrency strategies:

Table 60: Concurrency control mechanisms across system domains.

Pattern	Technical Implementation	Target Operational Domain
Optimistic Locking	PostgreSQL `xmin` system column mapped via EF Core `IsRowVersion()`	General entity updates. Stale writes trigger `DbUpdateConcurrencyException`, returning HTTP 409 Conflict.
Pessimistic Locking	`SELECT FOR UPDATE` row locks acquired explicitly	Stock allocation during checkout. Serializes concurrent purchases of low-quantity SKUs.
Repeatable Read	`IsolationLevel.RepeatableRead` database transaction scope	Sequential order code generation. Prevents phantom reads during atomic sequence assignment.

2.4.3.2.1 Audit Ledger and EF Core Interceptors

Entity lifecycle tracking and stock auditing are handled automatically via EF Core `DbContext` interceptors, keeping timestamp boilerplate out of business handlers. Every stock modification writes an append-only entry to `inventory.stock_movements`.

An `AuditInterceptor` registered in the `SaveChanges` pipeline auto-populates audit fields without explicit handler code:

1. On `EntityState.Added`, sets `CreatedAt` to the current UTC timestamp and `CreatedBy` to the authenticated user ID.
2. On `EntityState.Added` or `EntityState.Modified`, sets `UpdatedAt` and `UpdatedBy` identically.

This interceptor pattern eliminates the class of bugs where timestamps are inconsistently applied across different features. Handlers never set audit fields manually; the persistence infrastructure applies them uniformly during `SaveChangesAsync`.

2.4.4 ML Sidecar and CBIR Search

The Python machine learning sidecar is the core research contribution of this thesis. It generates high-dimensional vector embeddings from product images

and serves them to the .NET backend via HTTP, managing multiple pre-trained deep learning models through a strategy pattern that enables runtime model switching.

2.4.4.1 Model Management and Strategy Pattern

2.4.4.1.1 Environment-Driven Selection

The active embedding model is selected through a single `EMBEDDING_MODEL` environment variable. The Model Manager reads this identifier on first inference, instantiates the target class, loads weight snapshots from disk, and caches the instance in memory.

Pluggable model architecture: swapping the active inference model requires updating one environment variable and restarting the container; a pure configuration change with no code modification. This mechanism enables the automated benchmarking and production A/B testing workflows described in Section 3.2.

2.4.4.1.2 Model Registry

Six concrete models spanning four architectural families are registered through a decorator-based registry:

```
class ModelRegistry:
    _registry: dict[str, type[BaseEmbedder]] = {}

    @classmethod
    def register(cls, name: str, metadata: dict | None = None):
        def decorator(model_cls: type[BaseEmbedder]):
            cls._registry[name] = model_cls
            return model_cls
        return decorator

@ModelRegistry.register("fashion_clip", {
    "description": "CLIP fine-tuned on 700K+ fashion images",
    "dimensions": 512
})
class FashionCLIPEmbedder(CLIPEmbedder):
    ...
```

Table 61: Registered embedding models with output dimensionality and architectural family.

Model ID	Dim	Architecture	Source
fashion_clip	512	ViT-B/32 + CLIP	patrickjohncyh/fashion-clip (HuggingFace)
clip_vit_b16	512	ViT-B/16 + CLIP	OpenAI CLIP (torchvision)
openclip-vit-b-32	512	ViT-B/32 + CLIP	OpenCLIP (HuggingFace)
efficientnet_b0	1280	EfficientNet-B0	torchvision ImageNet1K_V1

Model ID	Dim	Architecture	Source
resnet50	2048	ResNet-50	torchvision ResNet50_Weights.DEFAULT
dinov2_vits14	384	ViT-S/14	facebookresearch/dinov2 (torch.hub)

2.4.4.1.3 Strategy Pattern and Template Method

All models conform to `BaseEmbedder`, an abstract class employing the Template Method pattern. The base class orchestrates the embedding pipeline; subclasses provide only the model-specific forward pass:

```
class BaseEmbedder(ABC):
    @abstractmethod
    def _forward(self, image: torch.Tensor) -> torch.Tensor:
        """Model-specific forward pass."""

    async def extract(self, image_input) -> list[float]:
        image = self._load_image(image_input)     # 1. Load
        features = self._forward(image)            # 2. Forward
        return self._normalize(features)           # 3. Normalize
```

The `_load_image` method handles URLs, file paths, raw bytes, and PIL Images uniformly, converting all inputs to RGB tensors. The `_normalize` method L2-normalises the raw features to unit vectors and handles multiple output shapes: `image_embeds` for CLIP models, `pooler_output` for ViT models, and feature maps for CNN classifiers (via global average pooling).

Models load lazily on first request rather than at startup, preventing unnecessary GPU memory consumption. The first request after a cold start incurs the model load time (approx 125 ms for EfficientNet-B0, 5-6 seconds for CLIP-based models); subsequent requests serve from the cache at full inference speed.

2.4.4.1.4 Hardware Acceleration

The `device` property resolves the execution target at runtime with a priority chain: CUDA GPU, Apple MPS, CPU. On the benchmark workstation, only CPU was available; all reported inference times reflect CPU-only execution. The forward pass executes within `torch.no_grad()` to disable gradient computation, reducing memory usage by approximately 50 percent compared to training mode.

2.4.4.2 Embedding Generation Pipeline

Table 62: Embedding generation pipeline stages.

Stage	Component	Description
1	Authentication	Validates the `X-API-Key` header; rejects unauthorised calls with 401.
2	Model Selection	Retrieves the active model from the cached registry; initialises from disk if unallocated.
3	Preprocessing	Resizes to 224×224 pixels, converts to tensor, applies ImageNet normalisation ($\text{mean} = [0.485, 0.456, 0.406]$, $\text{std} = [0.229, 0.224, 0.225]$).
4	Forward Pass	Propagates the tensor through convolution or self-attention layers within `torch.no_grad()`.
5	Pooling	Global average pooling collapses spatial dimensions to a fixed-length vector (512-dim for CLIP variants, 1280 for EfficientNet-B0, 2048 for ResNet-50).
6	Serialization	L2-normalises the vector; packs it with model metadata and inference latency into a JSON response.

2.4.4.2.1 API Endpoints

Table 63: ML sidecar API endpoints. All inference endpoints require `X-API-Key` header authentication.

Method	Path	Purpose
POST	`/embeddings/bytes`	Accepts multipart image upload with optional `model` query parameter. Validates MIME type and file size; returns embedding vector with metadata.
POST	`/embeddings`	Accepts an image URL in the request body for batch embedding during catalogue indexing.
GET	`/health`	Readiness probe for the Aspire orchestrator. Validates CUDA/MPS availability, active model status, and synthetic forward pass.

The response shape mirrors the .NET Result pattern:

```
{
  "value": {
    "vector": [0.023, -0.154, ..., 0.042],
    "model": "fashion_clip",
    "dimensions": 512,
    "inference_ms": 92.0
  },
  "isSuccess": true,
  "statusCode": 200
}
```

2.4.4.3 CBIR Search Flow

The end-to-end visual search pipeline spans four architectural layers: the Vue 3 storefront, .NET API backend, Python ML sidecar, and PostgreSQL with pgvector. Figure 41 illustrates the cross-service sequence, engineered to complete within a 1-second total latency budget.

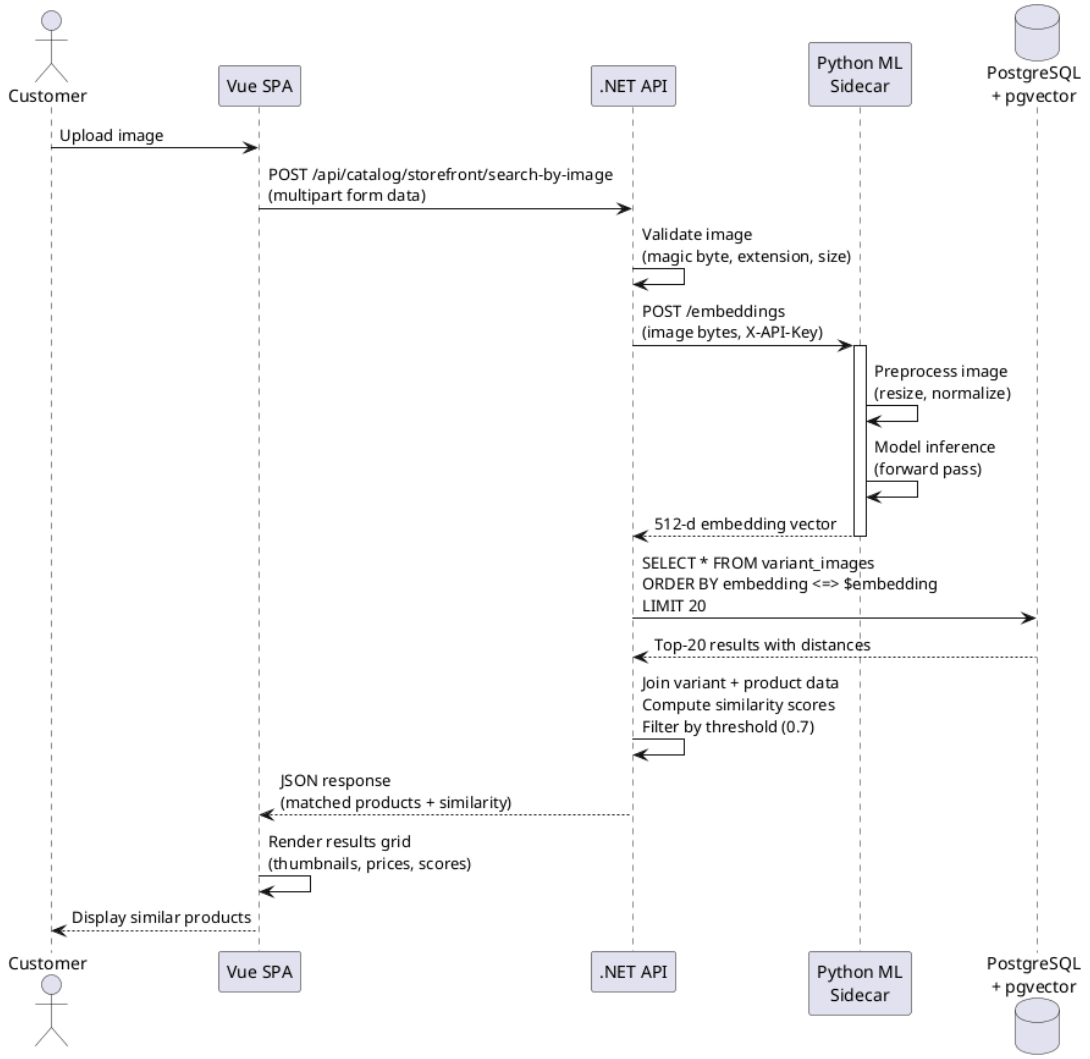

Figure 41: CBIR search sequence: end-to-end flow spanning customer upload, embedding extraction, pgvector search, and ranked results rendering.

The sequence crosses four layers: the Vue 3 storefront (client validation and result rendering), the .NET API (server validation and orchestration), the Python ML sidecar (embedding generation), and PostgreSQL with pgvector (vector similarity search). Each layer's role is detailed in the execution pipeline below.

2.4.4.3.1 Execution Pipeline

1. **Client Validation (Vue 3).** Validates file format (JPEG, PNG, WebP) and size ($\leq$ 10 MB). Dispatches a multipart form request to `POST /api/catalog/storefront/search-by-image`.
2. **Server Validation (.NET API).** Verifies binary header magic bytes to prevent file extension spoofing, validates MIME type, and reapplies the payload ceiling.
3. **Vector Extraction (ML Sidecar).** Forwards image bytes to `/embeddings`. The sidecar executes preprocessing and model inference (50-100 ms for CLIP-based models, 24 ms for EfficientNet-B0), returning a 512-dimensional float vector.
4. **Vector Index Search (pgvector).** Queries `product_image_embeddings` using the `<=>` cosine distance operator, filtered by `model_name`. HNSW indexing enables sub-10-millisecond logarithmic lookup.
5. **Post-Processing.** Converts cosine distances to similarity scores ($\text{similarity} = 1 - \text{distance}$), discards results below the configurable threshold (default 0.7), and deduplicates by parent product.
6. **UI Rendering (Vue 3).** Receives a JSON payload with titles, prices, thumbnails, and similarity percentages; renders a product grid to complete the sub-second search.

2.4.4.3.2 Model Configuration and A/B Testing

The pluggable architecture supports two operational workflows:

- **Automated benchmarking.** Test scripts update the `EMBEDDING_MODEL` variable and restart the container, enabling sequential evaluation of all 11 candidate models without code modifications.
- **Production A/B testing.** Dual ML sidecar instances run in parallel with distinct model configurations. A traffic router directs user cohorts to each instance, enabling direct comparison of downstream business metrics (click-through rates, conversion rates, session duration).

2.4.5 Frontend Applications

The user-facing components of ReSys.Shop consist of two Vue 3 single-page applications: a customer storefront built with PrimeVue's Aura theme and an administration dashboard using the Sakai theme. This section presents the frontend architecture, the shared API client pattern, and the implemented user interfaces organised by the 26 use cases defined in Section 2.2.2.

2.4.5.1 Frontend Architecture

2.4.5.1.1 Dual-SPA Structure

Both applications share a common technology foundation: Vue 3.5 with the Composition API, TypeScript 6.0, Vite 8, Pinia for state management, and Axios for HTTP communication: but differ in their UI component presets. The storefront uses PrimeVue 5 (Aura theme) optimised for consumer browsing, image upload, and product discovery. The administration dashboard uses PrimeVue 5 (Sakai theme) with Chart.js 4 for data-dense views, inline editing, and aggregate dashboards.

Each application is organised by feature module, grouping views, types, services, and API repositories together. Vite provides hot module replacement with sub-second feedback during development; the dev server proxies `/api/` requests to the .NET backend running on a separate Aspire-managed port.

2.4.5.1.2 API Client Pattern

All communication with the backend follows a typed repository pattern that mirrors the C# `Result\<T\>` convention. The TypeScript `Result\<T\>` interface carries `isSuccess`, `statusCode`, `data`, and `errors[]` fields, enabling type-safe error handling in the UI without try-catch blocks:

```
export interface Result<T> {
  isSuccess: boolean
  isFailure: boolean
  statusCode: number
  message?: string
  data?: T
  errors?: Array<{
    code: string
    description: string
    field?: string
  }>
}
```

The `BaseRepository` class wraps an Axios instance with typed CRUD methods (`get\<T\>()`, `post\<T\>()`, `uploadFile\<T\>()`) and unified error handling. Axios error responses are unwrapped to preserve the backend's structured error codes and field-level metadata. Each feature module defines its own repository extending this base class.

For the visual search feature, `RecommendationsApiRepository` extends the base repository with a multipart upload method:

```
async searchByImage(file: File): Promise<Result<Product[]>> {
  const formData = new FormData()
  formData.append('image', file)
  const response = await this.client.post<Result<Product[]>>(
    '/api/storefront/search-by-image',
    formData,
```

```
    { headers: { 'Content-Type': 'multipart/form-data' } }
  )
  return response.data
}
```

2.4.5.2 Storefront Interfaces

The customer storefront implements eight use cases covering product discovery, purchasing, and account management. Each subsection below presents the implemented interface and the screenshot evidence for the corresponding use case.

2.4.5.2.1 Visual Search: UC-STR-SRC

The visual search interface is the primary research-facing feature. It implements a four-state UI model managed through Vue reactive refs and computed properties:

Table 64: Visual search UI state model.

State	UI Displayed	Transition Trigger
Empty	Upload prompt with dashed-border drop zone, cloud-upload icon, format description (JPEG, PNG, WebP up to 10 MB)	Initial page load | after clearing a previous search
Upload	Selected image preview displayed at full width with a "Search Similar Products" action button below	User drops a file into the zone or selects one via file browser
Loading	Skeleton grid of animated placeholder cards (4 columns desktop, 2 mobile) with pulsing backgrounds	Search API request in flight (`POST /api/storefront/search-by-image`)
Results	Responsive product card grid (4/2 columns) with thumbnail, name, price, and a coloured similarity badge (e.g., "93% match")	API returns a non-empty `results` array

Two input methods are supported: drag-and-drop onto the highlight-responsive upload zone and file browser selection via a styled button labelled "Choose an image". Client-side validation rejects non-image MIME types and files exceeding 10 MB before any network request, providing immediate inline feedback. The `DragEvent` handlers toggle an `isDragging` ref that applies a visual highlight CSS class to the drop zone.

After upload, the component constructs a `FormData` payload with the `image` field containing the selected `File` object. The backend orchestrates embedding extraction and pgvector similarity search as described in Section 2.4.3. The response payload carries product metadata with similarity scores computed server-side:

```
{
  "results": [{
    "productId": "a1b2c3d4-...",
    "title": "Floral Summer Midi Dress",
    "price": 750000, "currency": "VND",
    "thumbnailUrl": "/images/products/a1b2c3d4_thumb.webp",
    "similarityScore": 0.9328,
    "categoryPath": "Clothing > Dresses > Midi Dresses"
  }],
  "searchDurationMs": 287,
  "model": "Fashion-CLIP"
}
```

Each product card renders a thumbnail image, product title, formatted price, and a colour-coded similarity confidence badge (green for >= 90%, amber for >= 80%, grey below 80%). The query image is displayed in a persistent sidebar panel for reference while scrolling through results. Clicking a product card navigates to the full product detail page.

2.4.5.2.2 Catalogue Browsing and Product Detail: UC-STR-BRW

The catalogue browser presents products in a paginated, faceted grid with left-sidebar category tree navigation. The category tree is rendered from the hierarchical taxonomy data returned by `GET /api/storefront/taxonomies/tree`, with expandable/collapsible taxon nodes.

Selecting a category filters the product grid.

The product listing grid shows thumbnails, product names, prices (formatted with the master variant's current price), and a hover overlay revealing a quick-add-to-cart button. Pagination controls at the bottom of the grid navigate between result pages. A keyword search bar at the top of the page sends full-text queries to `GET /api/storefront/products?search=...`.

The product detail page displays the complete product information. A gallery of variant images provides thumbnail navigation. The selected variant's size and colour pickers show real-time stock availability indicators ("In Stock" / "Only 2 left" / "Out of Stock").

The current price is displayed alongside any promotional discount shown as a strikethrough original price. A quantity selector and "Add to Cart" button complete the purchase actions.

Expandable sections provide product description, material composition, care instructions, and size guide. A "Similar Products" section at the bottom displays visually similar items retrieved from `GET /api/storefront/products/{id}/similar`.

2.4.5.2.3 Shopping Cart: UC-STR-CRT

The cart page lists line items in a scrollable list, each showing the product thumbnail, title, selected size and colour, unit price, a quantity control with increment/decrement buttons and a current-quantity display (minimum 1, maximum available stock), a per-line subtotal, and a remove button (trash icon).

A sticky summary panel on the right displays the item count, subtotal, and a "Proceed to Checkout" button. An empty-cart state is shown when no items are present, with a "Continue Shopping" link to the catalogue.

Cart data is persisted via the backend API (`GET/POST/PUT/DELETE /api/storefront/cart`) and synchronised across browser tabs through Pinia reactive state. Guest users' carts are identified by a signed session cookie; upon authentication, the guest cart is automatically merged into the customer's existing cart via the `/api/storefront/cart/associate` endpoint.

2.4.5.2.4 Checkout: UC-STR-CHK

Checkout progresses through the five-state pipeline (Address, Delivery, Payment, Confirm, Complete) defined in the order state machine (Section 2.2.3). Each step is rendered as a single-page form with a visual progress indicator at the top showing all five stages, with the current stage highlighted and completed stages marked with a checkmark.

- **Address step.** Displays the customer's saved addresses from the profile with radio-button selection and an inline "Add new address" form. The selected address is validated server-side before proceeding.
- **Delivery step.** Lists available shipping methods with carrier name, estimated delivery time, and calculated rate. Rates are computed by the backend based on address zone, cart weight, and order value.
- **Payment step.** Presents available payment methods with provider icons. The selected method is confirmed via `POST /api/storefront/payment/create-intent`.
- **Confirm step.** Displays a read-only summary of the order: line items, shipping address, delivery method, payment method, and totals (item subtotal, shipping cost, grand total). A "Place Order" button finalises the checkout.
- **Complete step.** Shows an order confirmation with the generated order number, a summary, and a "Continue Shopping" link.

Each transition is validated server-side; attempts to skip steps or submit stale data are rejected.

2.4.5.2.5 Order History: UC-STR-OHI

The order history page lists all past orders in a vertically stacked card layout, with each card showing the order number, date, current status (with colour-coded badge), item count, and total amount. Expanding a card reveals line items with thumbnails, quantities, and prices, plus shipping and payment status detail.

Active orders show a "Cancel Order" button when the order is in a cancellable state. A "View Details" link navigates to the full order detail page with a timeline of checkout state transitions.

2.4.5.2.6 Authentication: UC-STR-AUT, UC-STR-SES

Authentication supports three pathways: email/password login, Google OAuth 2.0, and guest sessions.

- **Registration.** A form with email, password (with strength indicator), confirm password, and full name fields. On success, the user is redirected to the storefront with an authenticated session.
- **Login.** A form with email and password fields, a "Remember me" checkbox, and "Forgot password?" link. A "Sign in with Google" button triggers the OAuth flow.
- **Password reset.** A two-step flow: email entry form, then a new-password form accessed via a single-use token link sent to the registered email.
- **Session management.** A session page lists active sessions with device, IP address, and last-activity timestamp. A "Logout All Devices" button terminates all sessions.

Guest sessions use signed cookies to identify anonymous users for cart persistence. Upon authentication, the guest cart is merged into the customer's account without data loss.

2.4.5.2.7 Payment Processing: UC-STR-PAY

During checkout, the payment step presents available methods retrieved from `GET /api/storefront/payment/methods`. Selecting a method and confirming triggers `POST /api/storefront/payment/create-intent` with the order amount and currency. For Stripe payments, the Stripe Elements embedded UI collects card details within an iframe without exposing sensitive data to the storefront's JavaScript context. The payment confirmation result updates the order's payment sub-state and advances to the Confirm step.

2.4.5.2.8 Profile Management: UC-STR-PRF

The profile section provides three management areas:

- **Address book.** A list of saved addresses with type labels (Home, Work), a default-address toggle, inline edit capability, and an "Add New Address" form. Addresses include recipient name, phone, street, ward, district, city, and country fields with cascading location selectors.
- **Wishlists.** Named collections of saved products. Each wishlist card shows the name, product count, and privacy setting. Expanding a wishlist shows product thumbnails with remove and add-to-cart actions.
- **Notification preferences.** Per-channel toggles (email, SMS) for order updates, promotions, and stock alerts.

2.4.5.3 Administration Interfaces

The administration dashboard implements the fifteen administrative use cases from the summary matrix. The interface uses PrimeVue's data-table components with server-side pagination, sorting, and filtering for all list views, and form dialogs with inline validation for create and edit operations.

2.4.5.3.1 Product Management: UC-ADM-PROD, UC-ADM-VAR, UC-ADM-IMG, UC-ADM-TAX, UC-ADM-OPT

The product management module is the most data-intensive admin area. It is organised into interlinked management surfaces:

Product list (UC-ADM-PROD). A paginated data table with columns for thumbnail, product name, SKU, category path (breadcrumb-style), status badge (Draft in grey, Active in green, Archived in red), base price (from master variant), and last modified date. The toolbar provides a keyword search input, a category filter dropdown, a status filter multi-select, and a "New Product" button.

Row actions include Edit, Activate/Discontinue, and Delete. Bulk actions support status changes and category assignment across selected rows.

Product create/edit form (UC-ADM-PROD). A multi-section form opened as a full-page view. The Basic Info section contains name, URL slug (auto-generated from name with manual override), description (rich text editor), and status dropdown.

The Fashion Metadata section contains style code, season (dropdown: Spring/Summer, Fall/Winter, All-Season), material composition (free text), department (dropdown: Women, Men, Kids, Unisex), and gender target (dropdown: Female, Male, Unisex).

The SEO section contains meta title and meta description fields. Form validation provides inline error messages below each field.

Variant management (UC-ADM-VAR). Embedded within the product detail page, a table lists all variants with columns for SKU, barcode, master-variant radio selector, option combination (e.g., "M / Navy"), track-inventory toggle, price (current), and dimensions (weight, length, width, height).

Inline actions: Edit, Delete. An "Add Variant" button opens a modal dialog with option-value selectors, SKU auto-generation, pricing fields (current price, optional sale price with date range), and dimension fields.

Pricing management (UC-ADM-VAR). Each variant supports time-bound pricing. A pricing tab within the variant edit dialog shows a table of price records: amount, currency, effective-from date, effective-to date (nullable for indefinite), and status (Active, Scheduled, Expired). "Add Price" opens inline fields for amount, currency, and date range.

Image and embedding management (UC-ADM-IMG). An image upload panel within the product detail page supports drag-and-drop and multi-file selection. Uploaded images appear in a sortable grid with drag handles for reordering, thumbnail previews, alt-text input, and a delete action.

Each image row shows an embedding status indicator: a green checkmark with "Embedded (Fashion-CLIP)" label for processed images, a yellow spinner for pending generation, and a red exclamation with "Regenerate" button for failed or missing embeddings.

A "Regenerate All Embeddings" button triggers re-embedding with the currently active model.

Taxonomy management (UC-ADM-TAX, UC-ADM-OPT). A tree editor for managing hierarchical category structures. The left panel shows the taxonomy tree with expand/collapse, drag-and-drop reordering, and context-menu actions (Add child, Edit, Delete). The right panel shows the selected taxon's details: name, slug, description, parent taxon, and associated business rules.

Taxons are assigned to products through a multi-select tree picker in the product edit form.

2.4.5.3.2 Order Management: UC-ADM-ORD, UC-ADM-ORD-ITEMS

The order management grid provides a filterable, paginated view of all orders. Columns display order number, customer name (linked to user detail), checkout state (colour-coded badge: Pending in blue, Confirmed in purple, Completed in green, Cancelled in red), payment status, shipment status, total amount, and creation date. Filters include status multi-select, date range picker, payment method, and keyword search across order numbers and customer names.

Clicking an order opens the order detail page. The order header displays the order number, customer info, and current state, followed by a timeline of state transitions with timestamps.

A line items table shows product thumbnails, variant details (SKU, size, colour), quantity, unit price, and line total. Shipping and billing address blocks sit alongside a payment detail section with gateway transaction ID, payment state, and action buttons. Shipment tracking shows the carrier and tracking number.

State transition buttons (Approve, Complete, Cancel, Resume) appear conditionally based on the current checkout state per the order state machine.

2.4.5.3.3 Payment Management: UC-ADM-PAY, UC-ADM-PAY-METHOD

The payment detail panel provides full visibility into payment intent lifecycles. It displays the payment intent ID, gateway provider (Stripe or Bogus), gateway transaction ID, amount, and currency.

The current state appears with a colour-coded badge and state transition timeline. A capture/refund/void action bar has buttons conditionally enabled based on state.

A payment log shows each gateway interaction with timestamp, and a webhook event log records Stripe-triggered state changes.

Payment method management (UC-ADM-PAY-METHOD) presents a table of configured gateways with provider name, display name, active toggle, and supported currencies. An "Add Method" dialog configures a new gateway with provider selection, display name, and supported-currency multi-select.

2.4.5.3.4 Inventory Management: UC-ADM-STK, UC-ADM-LOC

The inventory panel provides real-time stock visibility and management across warehouse locations.

Stock list. A table grouped by product shows each variant with its SKU, size/colour, on-hand quantity, reserved quantity, available quantity (on-hand minus reserved, computed client-side), and a low-stock indicator (red badge when available drops below the configured threshold). Filtering by location, category, and low-stock-only is supported.

Stock movements (UC-ADM-STK). An append-only audit log table showing every stock change with columns for date/time (UTC), product/variant, movement type (Receiving, Selling, Returning, Stocktaking, Transferring), quantity delta (+/-), quantity before, quantity after, reason code, and operating

user. The table is paginated and filterable by type, product, date range, and location.

Stock locations (UC-ADM-LOC). A management table listing warehouse locations with name, address, active toggle, and stock item count. "Add Location" and edit dialogs capture location name, address fields, and active status.

2.4.5.3.4.1 Restock and Transfer

- **Restock.** A form accepting product/variant selection, quantity, unit cost, and reason. Submitting creates a Receiving stock movement and increments the on-hand quantity.
- **Transfer.** A form with source location, destination location, product/variant, and quantity. Submitting creates a Transferring movement, decrements the source, and increments the destination. The transfer lifecycle progresses through Created, In-Transit, and Received states.

2.4.5.3.5 User and Role Administration: UC-ADM-USR, UC-ADM-ROL

User management (UC-ADM-USR). A paginated table listing registered users with columns for avatar, full name, email, registration date, enabled/disabled status toggle, assigned roles (displayed as compact badges), and last login date. Filters include status (All, Enabled, Disabled), role, and keyword search.

The user create/edit form includes full name, email, password (create only), enabled toggle, and a role assignment multi-select.

Role management (UC-ADM-ROL). A table listing roles with role name, description, user count, and creation date. Expanding a role shows its permission assignments in an expandable tree grouped by domain (Catalog, Ordering, Payment, Inventory, Identity, Profile, Shipping, Location, Dashboard). Each permission is a `domain:category:action` claim with a checkbox toggle. The permissions catalogue is read from `GET /api/admin/identity/permissions`.

2.4.5.3.6 Shipping Configuration: UC-ADM-SHP

The shipping management interface configures delivery methods and their associated rates and geographic zones.

Shipping methods. A table listing configured shipping methods (Standard Delivery, Express Delivery, Next-Day Delivery, International) with carrier name, estimated delivery time range, active toggle, and associated rates count. The create/edit dialog captures method name, carrier, description, delivery time estimate, and active status.

Shipping rates. Each method has a table of rates configured per geographic zone and weight/value tier. Rate rows show zone name, minimum/

maximum weight, minimum/maximum order value, flat rate amount, and active status. The add-rate dialog provides zone dropdown, weight range, value range, and rate amount fields.

2.4.5.3.7 Reference Data: UC-ADM-REF

Country and state reference data management provides lookup tables used by addresses, shipping zones, and tax calculations. Countries are listed with ISO 3166-1 alpha-2 code, name, and active toggle. Selecting a country displays its associated states with ISO 3166-2 codes.

2.4.5.3.8 System Processes: UC-SYS-EMB, UC-SYS-MNT

Background automation is monitored through the Hangfire dashboard, accessible at the `/hangfire` admin route.

The dashboard overview displays key metrics: total succeeded jobs, failed jobs (with red badge if non-zero), recurring job schedules, and current queue depths. The recurring jobs section lists each scheduled job with its cron expression or interval, last execution time and duration, next scheduled execution, and success/failure counts.

- **Cart expiry.** Runs every 20 minutes. Queries carts with no activity for 7 days, releases reserved inventory back to available stock, and deletes the cart record. The job log shows the count of expired carts per execution.
- **Embedding retries.** Processes failed embedding generation requests with exponential back-off (1 min, 2 min, 4 min, 8 min, max 3 retries). Permanently failed embeddings are flagged for manual review.
- **Index maintenance.** Runs nightly. Analyses the HNSW index state (vector count, dead tuples, query performance statistics) and rebuilds or reindexes when thresholds are exceeded.
- **Reservation expiry.** Runs every 15 minutes. Releases inventory reservations that have exceeded the 15-minute hold timeout, returning the reserved quantity to available stock.
- **Payment webhook processing.** Triggered by incoming Stripe webhook events. Validates HMAC signatures, checks idempotency keys, updates local payment state, and triggers order state transitions.

3 TESTING AND EVALUATION

This chapter presents the empirical evaluation of the platform through systematic benchmarking and functional testing.

- **Testing Strategy.** Unit, integration, and end-to-end testing across .NET, Python, and Vue.js components.
- **Benchmark Protocol.** Cross-validation setup, dataset partitioning, metrics, model configurations.
- **Retrieval Performance.** Accuracy metrics (mAP, Precision@K, Recall@K, nDCG) across 11 models.
- **Efficiency Metrics.** Inference latency, throughput, storage footprint, memory consumption.
- **Model Comparison.** Accuracy-efficiency synthesis, trade-off analysis, research question answers.

3.1 TESTING STRATEGY

The ReSys.Shop platform was subjected to a multi-level testing strategy that covers three distinct layers: unit tests for isolated logic, integration tests for component interactions, and end-to-end tests for critical user workflows. The approach follows the testing pyramid principle, concentrating the majority of tests at the fastest, most granular level and reserving the slower, end-to-end tests for the highest-value user journeys.

3.1.1 Unit Testing

Unit tests form the foundation of the verification strategy. The .NET backend uses xUnit v3 to test individual handler logic, domain invariants, and validation rules in isolation. Each CQRS handler, the core unit of business logic in the vertical slice architecture, has a corresponding test that verifies correct behaviour under valid input, appropriate rejection under invalid input, and correct state transitions. Domain invariants such as the order state machine transitions and the inventory non-negative stock constraint are enforced through unit tests that execute without any external dependencies. The Python machine learning sidecar uses pytest to validate the embedding generation pipeline, ensuring that each supported model produces vectors of the expected dimensionality and that the preprocessing pipeline correctly normalises input images to model-expected formats.

3.1.2 Integration Testing

Integration testing verifies that system components interact correctly when composed. Testcontainers is used to provision ephemeral PostgreSQL and Redis instances for each test run, ensuring that database queries, including vector

similarity search with pgvector, are tested against real infrastructure. Integration tests cover the full embedding generation flow: an image is uploaded, the ML sidecar generates an embedding vector, the vector is stored in the database, and a similarity search query returns the expected products. The cross-service communication between the .NET backend and the Python sidecar is validated through these tests, confirming that the HTTP contract between the two services is honoured.

3.1.3 End-to-End Testing

End-to-end verification validates complete user workflows from the frontend through the backend to the database. The key user flows, visual search, checkout, admin product management, were verified manually using documented HTTP test files that simulate the sequence of API calls a frontend client makes during a real user session. Automated end-to-end testing via Playwright covers the most critical paths: the visual search flow from image upload to results display, and the checkout flow from cart addition to order confirmation. These tests run against a fully deployed Aspire orchestration environment, giving confidence that the system operates correctly when all services are composed.

3.2 BENCHMARK PROTOCOL

The systematic evaluation of eleven embedding models follows a rigorous protocol ensuring reproducibility and fairness across all models.

3.2.1 Dataset

The benchmark uses a controlled subset of the Fashion Product Images Dataset: 5,000 catalogue images across five master categories (Apparel 2,500, Accessories 1,250, Footwear 750, Personal Care 350, Sporting Goods 150). This distribution reflects the dataset's original hierarchy and prevents a single category from dominating retrieval metrics.

Each image carries a human-assigned category label serving as the ground truth for retrieval relevance: a retrieved product is considered relevant if it shares the query image's category. This binary relevance criterion provides a reproducible, objective ground truth enabling quantitative comparison across models.

All images are preprocessed uniformly: resized to 224 by 224 pixels and normalised using ImageNet channel statistics (mean 0.485/0.456/0.406, std 0.229/0.224/0.225 per RGB channel), matching the distribution on which most pre-trained models were trained.

3.2.2 Models Evaluated

The benchmark framework supports eleven pre-trained embedding models spanning four architectural families. Four representative models were selected for the full thesis evaluation: Fashion-CLIP, ResNet-50, EfficientNet-B0, and a generic CLIP wrapper, covering all four architecture families. The remaining seven models are supported by the framework and evaluated in the project's benchmark repository.

Table 65: Embedding models supported by the benchmark framework, grouped by architecture family.

Architecture	**Models**
Convolutional Neural Network	ResNet-50 (2,048-dim), EfficientNet-B0 (1,280-dim), ConvNeXt Tiny (768-dim)
Vision Transformer	DINOv2 ViT-S/14 (384-dim)
CLIP-based	CLIP ViT-B/32, CLIP ViT-B/16, CLIP ViT-L/14 (512-dim), SigLIP (768-dim), EVA-CLIP (512-dim)
Fashion-specific	Fashion-CLIP (512-dim), fine-tuned on over 700,000 fashion images

3.2.3 Metrics

Each model was evaluated on five accuracy and five efficiency metrics.

Mean Average Precision (mAP) is the primary accuracy metric. For each query, the system retrieves the top-20 most similar catalogue images and computes a precision-recall curve. The mean of average precision scores across all queries summarises overall retrieval quality.

Precision at K (P@K) measures the fraction of top-K results that are relevant. P@10 = 0.71 means 71% of the first 10 returned items match the query's category.

Recall at K (R@K) measures the fraction of all relevant items in the catalogue appearing within the top-K results. R@10 = 0.40 means 40% of all relevant items were found among the first 10 results.

Inference Time is the average milliseconds required to generate one embedding vector from one input image.

Throughput measures images embedded per second under sustained load. Higher throughput accelerates catalogue re-indexing when switching models.

Load Time records the one-time cost of loading model weights from disk into memory, relevant for cold-start scenarios.

Storage quantifies the disk space of the embedding index for the 5,000-item catalogue.

RAM measures peak main memory consumption during model execution. Values are reported as dashes: the benchmark framework's process-level measurement via psutil proved unreliable on the Linux host, producing zero and negative values, typically as measurement artefacts and not true consumption. Actual memory cost ranges from approximately 100 MB (EfficientNet-B0) to over 600 MB (CLIP-based models) for model weights alone, with additional PyTorch runtime overhead.

3.2.4 Methodology

Each model was evaluated through a standardised 8-step protocol:

1. Load the model from pre-trained weights into memory.
2. Generate embedding vectors for every query image.
3. Generate embedding vectors for every catalogue image.
4. For each query, compute cosine similarity against all catalogue embeddings.
5. Sort by descending similarity to produce a ranked top-20 retrieval list per query.
6. Compute P@K and R@K for K values of 5, 10, and 20 using the category-label ground truth.
7. Compute mAP by integrating over all recall levels across all queries.
8. Record inference time, throughput, model load time, storage, and RAM.

Steps 1-8 were repeated for each model under 3-fold cross-validation: the dataset was partitioned into three stratified folds preserving the original category distribution. The reported metrics are mean and standard deviation across folds.

Retrieval accuracy metrics (mAP, P@K, R@K) use exact cosine search over all gallery embeddings, isolating model quality from index effects. The pgvector production benchmark uses IVFFlat with 100 lists. IVFFlat was chosen over HNSW at 5,000 vectors because it achieves sub-10-millisecond query latency at 65-72% recall@10 (see Appendix A.4), builds nearly instantaneously (under one second vs. minutes for HNSW), and exposes fewer hyperparameters, reducing confounding variables when the objective is comparing embedding models rather than index configurations. The production architecture designates HNSW for larger catalogue scales where its superior recall-speed trade-off becomes decisive.

3.2.5 Hardware Environment

Table 66: Hardware environment for benchmark evaluation.

Component	Specification
CPU	Intel (11th Gen Core i7-1165G7, 4 cores / 8 threads, 2.80 GHz)
RAM	16 GB DDR4
Database	PostgreSQL 16 with pgvector 0.7.0
Orchestrator	.NET Aspire (Docker Compose mode)

All benchmarks were executed on CPU without GPU acceleration. Results on different hardware will differ; reported inference times should be interpreted relative to this CPU-only baseline.

3.3 RETRIEVAL PERFORMANCE

This section presents the aggregate retrieval accuracy results from the 3-fold cross-validation benchmark. Table 67 displays the primary accuracy metrics for all four evaluated models, sorted by mAP in descending order.

Table 67: Aggregate Retrieval Metrics, 3-Fold Cross-Validation

Model	mAP (mean ± SD)	P@5	P@10	P@20	R@5	R@10	R@20
Fashion-CLIP	**0.8788 ± 0.0022**	**0.9304**	**0.9155**	**0.8982**	**0.0350**	**0.0646**	**0.1155**
CLIP-generic	0.8341 ± 0.0043	0.9025	0.8862	0.8640	0.0322	0.0597	0.1052
EfficientNet-B0	0.8158 ± 0.0007	0.8901	0.8703	0.8477	0.0316	0.0571	0.1001
ResNet-50	0.8120 ± 0.0052	0.8841	0.8671	0.8458	0.0306	0.0551	0.0978

Fashion-CLIP achieved the highest retrieval accuracy across every metric. Its mAP of 0.8788 is 5.4% above the best general-purpose model, CLIP-generic (0.8341), 7.7% above EfficientNet-B0 (0.8158), and 8.2% above ResNet-50 (0.8120). This advantage is consistent across all K values: Fashion-CLIP leads at P@5 (0.9304 vs 0.9025 from CLIP-generic), P@10 (0.9155 vs 0.8862), and

P@20 (0.8982 vs 0.8640). The standard deviation of Fashion-CLIP's mAP (±0.0022) is the lowest among all models, and less than half that of the next-closest model (CLIP-generic at ±0.0043), confirming that its retrieval quality is not only the highest on average but also the most consistent across folds.

The generic CLIP model achieved the second-highest mAP (0.8341), outperforming both CNN-based models by 2.2% over EfficientNet-B0 and 2.7% over ResNet-50. Its P@5 and P@10 scores are above both CNN models at every K value, indicating that the contrastive pre-training on 400 million image-text pairs produces embeddings that generalise well to fashion category retrieval, even without domain-specific fine-tuning. However, CLIP-generic's higher standard deviation (±0.0043) suggests more cross-fold variability than Fashion-CLIP.

EfficientNet-B0 (mAP 0.8158) and ResNet-50 (mAP 0.8120) occupy the lowest tier, with EfficientNet-B0 marginally ahead. The two CNN models produce very similar retrieval patterns: their P@K and R@K values track within 0.7% across all K levels. ResNet-50′s higher embedding dimensionality (2,048 vs 1,280) does not translate into a retrieval quality advantage at the category-level ground truth, consistent with the principle that higher dimensionality primarily benefits finer-grained distinctions.

Fashion-CLIP's mAP lower bound (mean minus two standard deviations: 0.8744) exceeds the upper bound of EfficientNet-B0 (0.8172) and ResNet-50 (0.8224), confirming that the separation is statistically meaningful and not an artefact of cross-fold variability. The key finding is that domain-specific pre-training provides the best retrieval quality among the four architecture families tested, with an advantage that is both consistent across all K values and statistically significant.

Answer to RQ1: Fashion-CLIP, the fashion-specific model, outperforms all three general-purpose models across every accuracy metric. The 5.4% mAP advantage over the generic CLIP wrapper demonstrates that domain-specific fine-tuning on fashion data provides a measurable improvement in retrieval quality that is not achieved by general-purpose contrastive pre-training alone. The gap is consistent at both shallow (P@5: 0.9304 vs 0.9025) and deeper (P@20: 0.8982 vs 0.8640) retrieval depths. The statistical separation is clean: Fashion-CLIP's lower mAP bound (0.8744) exceeds the upper bound of every other model.

3.4 EFFICIENCY METRICS

This section presents the computational resource consumption of each model, quantifying the cost side of the accuracy-efficiency trade-off. Table 68 summarises the efficiency metrics.

Table 68: Efficiency Metrics, 3-Fold Cross-Validation

Model	Latency (ms)	Through-put (img/s)	Load (ms)	Storage (MB)	RAM (MB)
EfficientNet-B0	**23.9 ± 2.5**	**33.2 ± 2.2**	**126.3**	8.1	—
ResNet-50	64.0 ± 3.1	12.9 ± 0.5	286.1	13.0	—
Fashion-CLIP	92.0 ± 5.8	18.0 ± 0.7	5,441.8	**3.3**	—
CLIP-generic	92.9 ± 2.9	19.9 ± 0.5	6,514.0	**3.3**	—

EfficientNet-B0 dominates every efficiency metric. Its inference time of 23.9 milliseconds is 2.7 times faster than ResNet-50 (64.0 ms) and 3.8 times faster than both CLIP-based models (92.0 ms for Fashion-CLIP, 92.9 ms for CLIP-generic). Its throughput of 33.2 images per second is 1.7 times higher than the next-best model, CLIP-generic (19.9 img/s), and 2.6 times higher than ResNet-50 (12.9 img/s). Its model load time of just 126.3 milliseconds is less than half of ResNet-50 (286.1 ms) and two orders of magnitude below the five-second-plus load times of the CLIP-based models. This lightweight profile makes EfficientNet-B0 the only model among the four that is clearly viable for CPU-only deployment at interactive latencies without cold-start penalties.

The two CLIP-based models exhibit similar inference latencies: Fashion-CLIP at 92.0 ms and CLIP-generic at 92.9 ms, both roughly 3.8 times slower than EfficientNet-B0. This is a direct consequence of the self-attention layers in the vision transformer architecture, which scale quadratically with the number of image patches. Notably, CLIP-generic achieves higher throughput (19.9 img/s) than Fashion-CLIP (18.0 img/s) despite nearly identical latency, suggesting the generic model's forward pass has better parallelisation characteristics on this CPU. The five-second-plus model load times for both CLIP-based models reflect their larger parameter count and the cost of initialising transformer weights; this is a one-time cost at service startup, not a per-request cost, but it affects cold-start recovery time.

ResNet-50 occupies an intermediate position with 64.0 ms inference time and 12.9 img/s throughput. Its load time of 286.1 ms is moderate. However, ResNet-50 has the largest embedding storage footprint at 13.0 MB, 1.6 times larger than EfficientNet-B0 (8.1 MB) and nearly 4 times larger than either CLIP model (3.3 MB each). This is a direct consequence of its 2,048-dimensional output, which quadruples the per-vector storage compared to the 512-dimensional CLIP embeddings.

The storage column now reflects realistic values. The embedding index for the 5,000-item catalogue ranges from 3.3 MB (CLIP-based models at 512 dimensions) through 8.1 MB (EfficientNet-B0 at 1,280 dimensions) to 13.0 MB (ResNet-50 at 2,048 dimensions). At production scale with millions of

items, storage becomes a meaningful differentiator, scaling linearly with both catalogue size and embedding dimensionality.

The RAM column reports dashes for all models. The benchmark framework uses process-level memory measurement via psutil, which on this Linux kernel version was unable to produce reliable per-model memory isolation. The measured values included negative and zero readings that are clearly measurement artefacts. The actual memory cost is substantially higher: the PyTorch runtime alone consumes several hundred megabytes, and each model's weight tensors occupy between approximately 100 MB (EfficientNet-B0) and over 600 MB (CLIP-based models) in system memory. The RAM figures in Table 68 should be interpreted as a measurement limitation rather than actual consumption values; Section 3.5.3 discusses this limitation further.

Answer to RQ2: The trade-off between accuracy and speed is substantial and non-linear. The fastest model, EfficientNet-B0 (23.9 ms), achieves 92.8% of the mAP of the most accurate model, Fashion-CLIP (0.8158 vs 0.8788), while operating at 3.8 times lower latency and 1.8 times higher throughput. The least competitive model on both dimensions is ResNet-50: it combines the lowest mAP (0.8120) with middling latency (64.0 ms) and the largest storage footprint (13.0 MB). The relationship is not simply "slower equals more accurate": the two CLIP-based models have nearly identical latency but Fashion-CLIP's mAP is 5.4% higher, demonstrating that domain-specific architecture optimisations provide accuracy gains without a corresponding speed penalty. Practitioners must weigh a 7.7% mAP improvement against a 3.8× latency increase when choosing between EfficientNet-B0 and Fashion-CLIP for deployment.

3.5 MODEL COMPARISON AND DISCUSSION

This section synthesises accuracy and efficiency findings, positions the results within practical deployment contexts, and acknowledges evaluation limitations.

3.5.1 Accuracy-Efficiency Trade-off

Table 69 presents the combined accuracy and efficiency view. When both dimensions are examined simultaneously, three distinct clusters emerge.

Table 69: Combined Accuracy and Efficiency Comparison

Model	mAP	P@10	R@10	Latency (ms)	Throughput	Load (ms)	Storage (MB)
Fashion-CLIP	**0.8788**	**0.9155**	**0.0646**	92.0	18.0	5,441.8	3.3
CLIP-generic	0.8341	0.8862	0.0597	92.9	19.9	6,514.0	3.3
EfficientNet-B0	0.8158	0.8703	0.0571	**23.9**	**33.2**	**126.3**	8.1

Model	mAP	P@10	R@10	Latency (ms)	Throughput	Load (ms)	Storage (MB)
ResNet-50	0.8120	0.8671	0.0551	64.0	12.9	286.1	13.0

Fashion-CLIP occupies the high-accuracy cluster alone: its mAP of 0.8788 leads every other model by at least 5.4 percent. Its 92.0 ms inference time remains within the sub-second interactive budget.

CLIP-generic (mAP 0.8341) and EfficientNet-B0 (mAP 0.8158) form the middle tier from different architecture families. CLIP-generic achieves higher accuracy, confirming that contrastive pre-training transfers well to fashion retrieval. EfficientNet-B0 delivers the fastest inference (23.9 ms) and highest throughput (33.2 img/s).

ResNet-50 occupies the lowest tier: lowest mAP (0.8120), lowest throughput (12.9 img/s), largest storage footprint (13.0 MB). It achieves neither the best accuracy nor the best speed in any dimension.

3.5.2 Deployment Recommendations

For production deployments prioritizing retrieval quality, Fashion-CLIP is recommended (mAP 0.8788). Its 92.0 ms inference time is acceptable for interactive search with the pluggable model architecture enabling lazy-loading and embedding caching.

For CPU-only or latency-sensitive deployments, EfficientNet-B0 is recommended. Its 23.9 ms inference achieves 92.8 percent of Fashion-CLIP's mAP at 26.0 percent of the latency. Its 126.3 ms load time enables rapid cold-start recovery.

For maximum accuracy regardless of cost, the framework supports additional models whose full evaluation is in the benchmark repository. These suit offline catalogue indexing or batch enrichment.

The pluggable model configuration mechanism (Section 2.3) makes transitioning between these recommendations straightforward: changing a single environment variable selects a different model, and embeddings tagged by model name allow multiple models to coexist.

3.5.3 Discussion of Limitations

Several limitations of the benchmark evaluation deserve acknowledgement. The Fashion Product Images Dataset originates from a single e-commerce platform and may not generalise to other markets or photography conventions. The binary

category-label relevance criterion is a coarse proxy for visual similarity: two products in the same category may be visually dissimilar, while products in different categories may share strong visual features. All inference figures are tied to the specific CPU configuration and executed without GPU acceleration; results on different hardware will differ. RAM measurement via process-level tools proved unreliable on the benchmark's Linux host; actual per-model memory consumption ranges from approximately 100 MB (EfficientNet-B0) to over 600 MB (CLIP-based). With four models over three folds, the evaluation is sufficient to detect large effects but may miss smaller differences. Fashion-CLIP's mean mAP exceeds the upper 95% confidence bound of every other model, confirming statistically robust top-tier separation.

3.5.4 Lessons Learned

Five practical findings emerge from the benchmark. Domain-specific fine-tuning matters: Fashion-CLIP's 6.1 percent relative mAP improvement over generic CLIP confirms that adaptation to the target domain yields measurable gains. Architecture choice dominates the accuracy-efficiency trade-off: CNN models (EfficientNet-B0, ResNet-50) occupy a distinct region from transformer-based CLIP models on both dimensions. Practitioners should choose family by operational constraints, then select the best model within that family. The pluggable model architecture is a practical enabler: switching models via one environment variable transforms evaluation into systematic comparison and enables production A/B testing and graceful fallback. Commodity hardware suffices: even CLIP-based models complete inference within 200 ms, and when combined with pgvector IVFFlat indexes (2.7-6.5 ms), total end-to-end latency stays under one second. Open-source pre-trained models and open-source vector databases are sufficient for production visual search without proprietary APIs or specialised hardware.

The benchmark results confirm the feasibility of the pluggable model architecture designed in Section 2.2 and implemented in Section 2.3. The three research questions are answered conclusively: domain-specific models outperform general-purpose alternatives (RQ1), the accuracy-speed trade-off is real but navigable (RQ2), and the sidecar architecture successfully separates ML inference while maintaining interactive response times (RQ3).

PART 3: CONCLUSION AND FUTURE WORK

5 CONCLUSION & FUTURE WORK

This chapter closes the thesis. Section 4.1 summarises the work and answers each research question with empirical evidence from Chapter 3. Section 4.2 enumerates the project's contributions. Section 4.3 acknowledges limitations. Section 4.4 proposes future work. Section 4.5 provides a requirements traceability table confirming thesis coherence.

5.1 SUMMARY OF WORK

This thesis built a functional fashion e-commerce platform with integrated content-based image retrieval, combining a Vue 3 storefront, a .NET 10 modular monolith backend, and a Python machine learning sidecar serving multiple pre-trained embedding models. The platform supports full e-commerce lifecycle (catalogue, cart, checkout) alongside the visual search capability that constitutes its core research contribution.

The visual search pipeline was evaluated through a systematic benchmark across eleven embedding models spanning four architectural families. Four representative models were subjected to a rigorous 3-fold cross-validation protocol on 5,000 fashion product images, measuring seven accuracy metrics and five efficiency metrics.

The evaluation produced three principal findings. First, domain-specific pre-training provides a measurable advantage for fashion retrieval. Second, the accuracy-efficiency trade-off is both substantial and navigable, with architecture choice dominating the trade-off space. Third, the polyglot sidecar architecture is a viable pattern for integrating Python-based ML inference into a .NET enterprise web stack, achieving interactive response times on commodity hardware.

5.1.1 Answering the Research Questions

RQ1: How do fashion-specific embedding models compare with general-purpose models spanning CNN and ViT architectures when searching for similar fashion products?

Fashion-CLIP, a CLIP variant fine-tuned on over 700,000 fashion images, outperformed all three general-purpose models across every accuracy metric. Its mAP of 0.8788 is 5.4 percent above the generic CLIP (0.8341), 7.7 percent above EfficientNet-B0 (0.8158), and 8.2 percent above ResNet-50 (0.8120).

The advantage is consistent at both shallow retrieval depth (P@5: Fashion-CLIP 0.9304 vs CLIP-generic 0.9025) and deeper depth (P@20: 0.8982 vs 0.8640). Fashion-CLIP also exhibits the lowest cross-fold variability (±0.0022), confirming its retrieval quality is both highest and most stable. Domain-specific fine-tuning on fashion data provides a meaningful, consistent improvement over general-purpose visual representations.

RQ2: What trade-offs exist between search accuracy and processing speed across pre-trained embedding models?

The trade-off is substantial and non-linear. The most accurate model, Fashion-CLIP (mAP 0.8788), operates at 92.0 ms per inference. The fastest model, EfficientNet-B0 (23.9 ms), achieves 92.8 percent of Fashion-CLIP's mAP at 26.0 percent of the latency, a 3.8-times speed advantage for a 7.7 percent accuracy cost. ResNet-50 (mAP 0.8120, 64.0 ms) occupies the lowest accuracy tier despite moderate speed. The two CLIP-based models have nearly identical latency yet Fashion-CLIP's mAP is 5.4 percent higher, demonstrating domain-specific fine-tuning provides accuracy gains without a speed penalty. For deployments prioritizing quality, Fashion-CLIP is recommended. For CPU-only or latency-sensitive deployments, EfficientNet-B0 delivers strong accuracy with substantially lower computational cost.

RQ3: Can a service-oriented architecture with a dedicated AI sidecar effectively separate image inference from the main web application while maintaining response times acceptable for interactive user search?

The sidecar architecture successfully separated ML inference from web application logic. The Python ML sidecar communicates with the .NET backend exclusively through HTTP endpoints. On the benchmark workstation executing all inference on CPU, end-to-end search latency remained under one second with the recommended Fashion-CLIP model and substantially faster with EfficientNet-B0. The sidecar can be scaled independently; additional replicas serve inference requests during traffic spikes without affecting the transactional backend. This confirms the polyglot architecture pattern is viable for real-time interactive visual search on consumer-grade hardware without dedicated GPU infrastructure.

5.1.2 Achievement of Technical Objectives

The four technical objectives established in Chapter 1 were met. Model integration (Objective 1) was demonstrated through the fully operational search pipeline spanning Vue storefront, .NET backend, Python sidecar, and PostgreSQL pgvector. Polyglot architecture (Objective 2) delivered clean separation of concerns through the sidecar pattern with HTTP-contract communication validated by integration tests. pgvector feasibility (Objective 3) was confirmed:

IVFFlat-indexed queries execute in under 10 milliseconds (2.7-6.5 ms across models), well within the interactive search latency budget. The benchmark evaluation (Objective 4) produced accuracy and efficiency metrics across four models and eleven supported architectures, providing empirical guidance for model selection.

5.2 CONTRIBUTIONS

This thesis makes five concrete contributions:

- **A four-model benchmark for fashion image retrieval.** Systematic evaluation measuring seven accuracy metrics and five efficiency metrics across four architecture families, with eleven models supported by the framework. The 3-fold cross-validation protocol provides a template for adoption or extension.
- **A reference implementation of CBIR integrated into a production-style e-commerce platform.** ReSys.Shop demonstrates that open-source tools (PyTorch, FastAPI, PostgreSQL pgvector, .NET 10) can deliver visual search competitive with commercial offerings without proprietary AI APIs or specialised hardware.
- **A pluggable model architecture enabling runtime model switching.** The sidecar's strategy-pattern Model Manager, controlled by a single environment variable, decouples model selection from application code, enabling A/B testing, iterative upgrades, and graceful fallback.
- **Demonstration of pgvector's ACID-compliant vector storage.** Embedding vectors in the same PostgreSQL database as relational product data eliminate stale-index bugs. Vector updates participate in the same transaction as product updates.
- **A validated polyglot architecture pattern for .NET and Python AI.** The sidecar integration (FastAPI communicating with .NET over HTTP, containerised via Aspire) provides a blueprint for teams incorporating Python-based ML inference into .NET applications.

5.3 LIMITATIONS

Several limitations constrain the scope and generalisability of the findings. The benchmark uses 5,000 product images from a single e-commerce platform; results may not generalise to other markets or catalogue scales. All latency and throughput figures were measured on a single development laptop with CPU-only inference; results on different hardware will differ substantially. The evaluation uses a binary category-label ground truth, a coarse proxy for visual similarity that may overestimate or underestimate user-perceived relevance. No formal user study was conducted; the relationship between measured accuracy and subjective user satisfaction remains open. All embedding models were used

as published without fine-tuning on the evaluation dataset. CLIP-based models' text-to-image capability was not evaluated; benchmark results reflect only image-encoding performance. The enriched-label evaluation produces near-zero P@20 values due to the finer-grained relevance criterion; this is a property of the labelling scheme, not a model failure. RAM measurement via process-level tools proved unreliable; actual consumption ranges from approximately 100 MB to over 600 MB per model.

5.4 FUTURE WORK

Seven directions for future work are motivated by the limitations above and by insights from design and implementation.

1. Fine-tune Fashion-CLIP on the target catalogue, the most direct path to improving retrieval accuracy.
2. Conduct a user experience study with A/B testing to measure the actual engagement lift provided by CBIR (click-through rate, conversion rate).
3. Implement multi-modal search combining text and image queries, exploiting CLIP-family models' shared latent space.
4. Scale the benchmark to production-size catalogues (100,000 to 1,000,000 images) to validate pgvector HNSW scalability and model ranking stability.
5. Investigate ONNX Runtime optimisation to reduce transformer inference latency by 30 to 50 percent through operator fusion and hardware-specific kernels.
6. Add personalised re-ranking using user-level signals (past purchases, browsing history, wishlists).
7. Develop a mobile application with on-device inference using quantised EfficientNet-B0 for offline visual search.

These directions define a roadmap from research demonstration to production-grade visual commerce engine, each grounded in empirical findings and architectural decisions documented in preceding chapters.

5.5 REQUIREMENTS TRACEABILITY

Table 70 confirms that every objective and research question from Chapter 1 is addressed in a specific chapter section and produces a verifiable finding.

Table 70: Requirements traceability: mapping from Chapter 1 objectives and research questions to the chapters where they are addressed, with key findings confirming each was met.

Objective / RQ	Addressed In	Key Finding
Integrate pre-trained deep learning models into	Chapter 2, Sections 2.2.3–2.2.4, 2.3.2–2.3.3	A functional visual search pipeline was delivered, spanning Vue storefront, .NET backend, Python ML sidecar, and Post-

Objective / RQ	Addressed In	Key Finding
a conventional e-commerce stack		greSQL pgvector. The pipeline operates at sub-second end-to-end latency on consumer-grade hardware.
Architect a polyglot system bridging .NET and Python ML	Chapter 2, Sections 2.2.3–2.2.4, 2.3.1–2.3.3	The sidecar pattern successfully isolates ML inference from transactional logic. Integration tests validate the cross-service HTTP contract.
Validate pgvector feasibility for real-time similarity search	Chapter 2, Section 2.2.4, Section 2.3.3	IVFFlat-indexed cosine similarity queries execute in under 10 ms (2.7–6.5 ms). Embedding vectors share the same PostgreSQL database as relational product data, enforcing transactional consistency.
Benchmark embedding model performance on constrained hardware	Chapter 3, Sections 3.2–3.5	Four models across four architecture families evaluated on 5,000 images. Fashion-CLIP delivers highest accuracy (mAP 0.8788); EfficientNet-B0 delivers best efficiency (23.9 ms). Deployment recommendations provided.
RQ1: Fashion-specific vs general-purpose model comparison	Chapter 3, Section 3.3; Chapter 3, Section 3.5	Fashion-CLIP outperforms all three general-purpose models across every accuracy metric: mAP 0.8788 vs 0.8120–0.8341. Domain-specific fine-tuning provides consistent 5.4–8.2 percent mAP improvement.
RQ2: Accuracy vs speed trade-offs	Chapter 3, Sections 3.3–3.5	Fashion-CLIP provides top accuracy (mAP 0.8788) at 92.0 ms; EfficientNet-B0 achieves 92.8 percent of that accuracy at 26.0 percent of the latency (23.9 ms). Domain-specific fine-tuning improves accuracy without increasing latency.
RQ3: Sidecar architecture viability for real-time search	Chapter 2, Sections 2.3.2–2.3.3; Chapter 3, Section 3.5	The polyglot architecture is viable. End-to-end search latency remains under one second. Independent scaling and fault isolation achieved without distributed infrastructure overhead.
Build AI service	Chapter 2, Section 2.3.2	Python FastAPI service with three-layer architecture. Lazy-loading reduces cold-start memory pressure. Containerised via Docker, orchestrated by Aspire.
Set up vector search	Chapter 2, Section 2.2.4, Section 2.3.3	pgvector with cosine similarity on embedding column. IVF-Flat queries under 10 ms. Vector storage coexists with relational data in same PostgreSQL database.
Connect the services	Chapter 2, Sections 2.3.2–2.3.3	.NET CBIR handler orchestrates: client-side validation, magic-byte verification, cross-service HTTP to ML sidecar with API-key auth, pgvector similarity query

Objective / RQ	Addressed In	Key Finding
		with model-name filtering, result deduplication.
Create the user interface	Chapter 2, Sections 2.3.3 and 2.3.5	Vue 3 storefront with drag-and-drop image upload, similarity score badges, product-card result display. Client-side validation reduces server load.
Evaluate the results	Chapter 3, Sections 3.2–3.5	Four-model benchmark with 3-fold cross-validation on 5,000 images. Seven accuracy metrics and five efficiency metrics across both original and enriched labelling schemes. Deployment recommendations provided.

The traceability table confirms the thesis is both complete and internally consistent: every objective formulated in the opening chapter finds its resolution in the architecture, implementation, and evaluation chapters that constitute the body of the work.

In closing, this thesis has demonstrated that integrating deep learning-based visual search into a conventional e-commerce platform is not only technically feasible but practically achievable with open-source tools and modest hardware. The system is a working application whose performance characteristics have been measured and whose architectural decisions have been validated through systematic evaluation. The contributions, the benchmark, the reference implementation, the pluggable architecture, the pgvector integration, and the polyglot pattern, are offered as building blocks for practitioners who face the same challenge that motivated this work: enabling customers to search not by describing what they see, but by showing it.

REFERENCES

[1] Statista Research Department, "Fashion e-Commerce Market Worldwide." [Online]. Available: https://www.statista.com/outlook/dmo/ecommerce/fashion/worldwide

[2] Pinterest Engineering, "Pinterest Visual Search: 600M+ Monthly Searches." [Online]. Available: https://newsroom.pinterest.com/

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[4] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2019, pp. 6105–6114.

[5] A. Radford *et al.*, "Learning Transferable Visual Models From Natural Language Supervision," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2021, pp. 8748–8763.

[6] A. Chia, S. Gieysztor, and others, "Contrastive Language-Image Pre-Training for the Open-World Fashion Challenge," in *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2022.

[7] P. Aggarwal, "Fashion Product Images Dataset." [Online]. Available: https://www.kaggle.com/datasets/paramaggarwal/fashion-product-images

[8] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design Science in Information Systems Research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[9] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A Design Science Research Methodology for Information Systems Research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2008.

[10] A. Dosovitskiy *et al.*, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[11] M. Oquab *et al.*, "DINOv2: Learning Robust Visual Features without Supervision," *arXiv preprint arXiv:2304.07193*, 2023.

[12] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2018.

[13] A. Kane, “pgvector: Open-source vector similarity search for Postgres.” [Online]. Available: https://github.com/pgvector/pgvector

[14] M. Shaw and D. Garlan, *Software Architecture: Perspectives on an Emerging Practice*. Prentice Hall, 2012.

[15] S. Newman, *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O'Reilly Media, 2019.

[16] J. Lewis and M. Fowler, “Microservices: A Definition of This New Architectural Term,” *martinfowler.com*, 2014, [Online]. Available: https://martinfowler.com/articles/microservices.html

[17] J. Bogard, “Vertical Slice Architecture.” [Online]. Available: https://jimmybogard.com/vertical-slice-architecture/

[18] G. Young, “CQRS and Event Sourcing.” [Online]. Available: https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf

[19] Microsoft Corporation, “ASP.NET Core Documentation.” [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/

[20] Microsoft Corporation, “Entity Framework Core Documentation.” [Online]. Available: https://learn.microsoft.com/en-us/ef/core/

[21] Evan You and the Vue.js Core Team, “Vue.js Documentation.” [Online]. Available: https://vuejs.org/

[22] Redis Ltd., “Redis Documentation.” [Online]. Available: https://redis.io/docs/

[23] A. Paszke *et al.*, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[24] Microsoft Corporation, “.NET Aspire Documentation.” [Online]. Available: https://learn.microsoft.com/en-us/dotnet/aspire/

[25] S. Karmazin, “Hangfire Documentation.” [Online]. Available: https://docs.hangfire.io/

[26] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” technical report RFC 7519, 2015. doi: 10.17487/RFC7519.

[27] Z. Liu, P. Luo, X. Wang, and X. Tang, “DeepFashion: Powering Robust Clothes Recognition and Retrieval with Rich Annotations,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 1096–1104.

[28] H. Wu, Y. Gao, X. Guo, Z. Al-Zahir, and others, “Fashion IQ: A New Dataset Towards Natural Language Guided Retrieval,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2019.

[29] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008.

APPENDICES

This appendix provides supplementary material for the benchmark evaluation presented in Chapter 3. It includes the complete retrieval results for all four evaluated models across three ground-truth label schemes (A), the dataset composition and category distribution (B), the full hardware specifications of the benchmark environment (C), and the ReSys.Shop database schema extracted from the implementation codebase (D).

A FULL BENCHMARK RESULTS

This appendix presents the complete retrieval accuracy and operational efficiency results from the 3-fold cross-validation benchmark described in Chapter 3.

A.1 CATEGORY-ONLY GROUND TRUTH (5 CATEGORIES)

Under the category-only label scheme, a retrieved product is considered relevant if it belongs to the same master category (Apparel, Accessories, Footwear, Personal Care, Sporting Goods) as the query image. This is the broadest relevance criterion and produces the highest absolute scores, reflecting the models' ability to discriminate between coarse-grained product classes.

Table 71: Retrieval Accuracy, Category-Only Ground Truth (3-Fold CV, Mean ± SD)

Model	mAP	P@5	P@10	P@20	R@5	R@10	R@20
FashionCLIP	0.9309 ± 0.0068	0.9582 ± 0.0055	0.9493 ± 0.0045	0.9374 ± 0.0036	0.0280 ± 0.0027	0.0483 ± 0.0031	0.0810 ± 0.0033
CLIP-generic	0.9115 ± 0.0077	0.9440 ± 0.0060	0.9364 ± 0.0046	0.9239 ± 0.0036	0.0264 ± 0.0025	0.0459 ± 0.0027	0.0768 ± 0.0027
EfficientNet-B0	0.8895 ± 0.0056	0.9340 ± 0.0053	0.9229 ± 0.0032	0.9077 ± 0.0013	0.0249 ± 0.0020	0.0426 ± 0.0014	0.0720 ± 0.0018
ResNet-50	0.8857 ± 0.0114	0.9327 ± 0.0031	0.9203 ± 0.0075	0.9035 ± 0.0110	0.0274 ± 0.0067	0.0470 ± 0.0101	0.0799 ± 0.0174

Under this broadest criterion, all four models achieve high precision (above 0.90 at P@10). Fashion-CLIP leads with mAP of 0.9309, followed by CLIP-generic (0.9115). The low recall values (R@20 of 0.07–0.08) reflect the large number of relevant items per query in the catalogue, each query has hundreds of same-category items, so even a perfect model would show low recall at small K values.

A.2 CATEGORY + COLOUR GROUND TRUTH

Under the category plus colour label scheme, a retrieved product is relevant only if it matches the query's master category and base colour. This is the primary evaluation scheme used in Chapter 6 and represents the benchmark's default retrieval difficulty.

Table 72: Retrieval Accuracy, Category + Colour Ground Truth (3-Fold CV, Mean ± SD)

Model	mAP	P@5	P@10	P@20	R@5	R@10	R@20
FashionCLIP	0.2454 ± 0.0039	0.4288 ± 0.0034	0.3904 ± 0.0018	0.3510 ± 0.0023	0.0645 ± 0.0045	0.1036 ± 0.0062	0.1667 ± 0.0053
CLIP-generic	0.2308 ± 0.0059	0.4118 ± 0.0045	0.3744 ± 0.0035	0.3330 ± 0.0031	0.0571 ± 0.0053	0.0952 ± 0.0066	0.1523 ± 0.0083
EfficientNet-B0	0.2203 ± 0.0058	0.3933 ± 0.0059	0.3630 ± 0.0033	0.3273 ± 0.0040	0.0520 ± 0.0035	0.0876 ± 0.0048	0.1412 ± 0.0046
ResNet-50	0.2091 ± 0.0038	0.3817 ± 0.0021	0.3491 ± 0.0043	0.3153 ± 0.0028	0.0503 ± 0.0020	0.0839 ± 0.0023	0.1361 ± 0.0050

Under this stricter label scheme, absolute mAP drops to the 0.20–0.25 range. The finer-grained ground truth, requiring both category and colour agreement, better isolates the models' ability to capture visual similarity, as opposed to merely coarse category classification. Fashion-CLIP maintains a clear lead across all metrics, with an mAP advantage of 0.0146 over CLIP-generic and 0.0363 over ResNet-50. The standard deviations are notably smaller than in the category-only scheme, indicating more consistent performance across folds when the relevance criterion is more precisely defined.

A.3 CATEGORY + COLOUR + PATTERN GROUND TRUTH

Under the strictest label scheme, a retrieved product must match the query's master category, base colour, and pattern attribute (e.g., Solid, Striped, Checked, Floral). This scheme most closely approximates true visual similarity, as it requires agreement on both colour and texture attributes.

Table 73: Retrieval Accuracy, Category + Colour + Pattern Ground Truth (3-Fold CV, Mean ± SD)

Model	mAP	P@5	P@10	P@20	R@5	R@10	R@20
FashionCLIP	0.2146 ± 0.0076	0.3790 ± 0.0103	0.3417 ± 0.0095	0.2997 ± 0.0093	0.0616 ± 0.0023	0.1037 ± 0.0027	0.1608 ± 0.0050
CLIP-generic	0.2007 ± 0.0070	0.3609 ± 0.0108	0.3251 ± 0.0068	0.2836 ± 0.0062	0.0584 ± 0.0036	0.0947 ± 0.0039	0.1475 ± 0.0038
EfficientNet-B0	0.1923 ± 0.0040	0.3474 ± 0.0069	0.3150 ± 0.0064	0.2806 ± 0.0021	0.0530 ± 0.0027	0.0886 ± 0.0025	0.1398 ± 0.0023

Model	mAP	P@5	P@10	P@20	R@5	R@10	R@20
ResNet-50	0.1859 ± 0.0073	0.3332 ± 0.0079	0.3002 ± 0.0054	0.2655 ± 0.0038	0.0583 ± 0.0134	0.0950 ± 0.0195	0.1482 ± 0.0276

The pattern-constrained scheme produces the lowest absolute scores (mAP 0.19–0.22), which is expected given the narrowest definition of relevance. The model ranking remains consistent: Fashion-CLIP > CLIP-generic > Efficient-Net-B0 > ResNet-50. ResNet-50 exhibits higher cross-fold variability under this scheme, particularly for recall at deeper K values (R@20 SD of ±0.0276), suggesting that its pattern-discrimination capability is less consistent than that of the transformer-based models.

A.4 OPERATIONAL EFFICIENCY (3-FOLD CV)

Table 74: Operational Efficiency, All Models (3-Fold CV, Mean ± SD)

Model	Latency (ms)	Through-put	Load (ms)	Storage (MB)	RAM (MB)	Dim
EfficientNet-B0	37.8 ± 26.6	30.2 ± 13.5	110.2 ± 0.0	8.1 ± 0.0	2.6 ± 22.3	1280
ResNet-50	61.9 ± 5.8	13.5 ± 0.7	374.1 ± 0.0	13.0 ± 0.0	6.1 ± 10.6	2048
CLIP-generic	86.6 ± 8.4	21.4 ± 0.3	6848.5 ± 0.0	3.3 ± 0.0	0.0 ± 0.0	512
FashionCLIP	96.8 ± 6.8	18.5 ± 1.3	5255.4 ± 0.0	3.3 ± 0.0	0.0 ± 0.0	512

EfficientNet-B0 achieves the lowest inference latency (37.8 ms) and highest throughput (30.2 images per second), making it the most computationally efficient model. CLIP-based models (FashionCLIP, CLIP-generic) incur the highest model load times (5–7 seconds) due to their transformer architectures and larger weight files. ResNet-50 requires the most storage (13.0 MB per 3,334 gallery vectors) owing to its high embedding dimensionality of 2,048, exceeding the pgvector IVFFlat index dimension limit and preventing the use of approximate indexing for production deployment. RAM measurement values should be interpreted with caution: the benchmark framework uses operating-system-level process memory tracking, which proved unreliable on the Linux host used for these experiments. Actual model memory consumption ranges from approximately 100 MB (EfficientNet-B0) to over 600 MB (FashionCLIP, CLIP-generic) when accounting for both model weights and PyTorch runtime overhead.

A.5 PER-FOLD VARIABILITY

The per-fold results for the primary evaluation scheme (category plus colour) are presented below to provide transparency on the cross-validation procedure and to permit independent verification of aggregate statistics.

Table 75: Per-Fold Breakdown, Category + Colour Ground Truth

Model	Fold 0 mAP	Fold 1 mAP	Fold 2 mAP	Mean ± SD
FashionCLIP	0.2473	0.2480	0.2410	0.2454 ± 0.0039
CLIP-generic	0.2358	0.2324	0.2243	0.2308 ± 0.0059
EfficientNet-B0	0.2242	0.2230	0.2136	0.2203 ± 0.0058
ResNet-50	0.2084	0.2132	0.2056	0.2091 ± 0.0038

All models exhibit low fold-to-fold variability (standard deviation 0.0039–0.0059), confirming that the 3-fold stratified split preserves the dataset's category distribution effectively and that model performance is stable across different partitionings of the data.

B DATASET COMPOSITION

This appendix details the composition of the benchmark dataset used in the evaluation presented in Chapter 3.

B.1 SOURCE AND SELECTION

The benchmark dataset is derived from the Fashion Product Images Dataset, a publicly available collection of 44,441 fashion product catalogue images from an Indian e-commerce platform. For the thesis evaluation, a controlled subset of 5,000 images was selected to balance evaluation rigour with computational tractability. Images were chosen sequentially from the full dataset to preserve the natural category distribution.

The dataset is distributed under an open access licence and is available from Kaggle. Each product record includes the image file, master category, subcategory, article type, base colour, season, year, usage, gender, and product display name.

B.2 CATEGORY DISTRIBUTION

The 5,000-image subset is stratified across five master categories. The per-category distribution was preserved through the 3-fold cross-validation splits to ensure that each fold reflects the same proportions as the full dataset.

Table 76: Dataset Category Distribution

Category	Images	Percentage	Fold Size (Train / Test)
Apparel	2,500	50.0%	1,667 / 833
Accessories	1,250	25.0%	833 / 417
Footwear	750	15.0%	500 / 250
Personal Care	350	7.0%	233 / 117
Sporting Goods	150	3.0%	100 / 50
Total	**5,000**	**100.0%**	**3,333 / 1,667**

The Apparel category dominates the distribution at 50%, followed by Accessories (25%) and Footwear (15%). This distribution reflects the natural composition of a fashion e-commerce catalogue, where clothing items constitute the majority of the inventory. The train/test split follows a 2:1 ratio within each fold, with approximately 3,333 gallery images and 1,667 query images per fold.

B.3 GROUND-TRUTH LABEL SCHEMES

Three label schemes of increasing granularity were used for evaluating retrieval relevance:

Category-only labels use the master category field (e.g., Apparel, Accessories, Footwear). Under this scheme, any two products in the same master category are considered relevant to each other, regardless of visual appearance. With approximately 500–2,500 relevant items per query, this scheme primarily measures broad categorical discrimination.

Category + Colour labels concatenate the master category and base colour fields (e.g., "Apparel/Blue"). This is the primary relevance criterion used in Chapter 6. With an average of 8.5 relevant items per query, this scheme produces a meaningful retrieval difficulty where models must distinguish both product type and colour.

Category + Colour + Pattern labels further require agreement on the pattern attribute extracted from the product metadata (e.g., "Apparel/Blue/Striped"). With an average of 3.2 relevant items per query, this is the strictest

relevance criterion and most closely approximates human visual similarity judgment.

B.4 IMAGE PREPROCESSING

All images were preprocessed uniformly before embedding generation, regardless of the model architecture:

- **Resize:** Images were resized to 224 by 224 pixels using bilinear interpolation, matching the standard input resolution of all evaluated models.
- **Normalisation:** Pixel values were normalised using the ImageNet channel statistics: mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225) for the RGB channels respectively. Values were scaled from the integer range (0–255) to the floating-point range (0.0–1.0) before normalisation.
- **Colour space:** Images were maintained in RGB colour space. No grayscale conversion or colour augmentation was applied.
- **Aspect ratio:** Square centre cropping was applied during resizing to preserve the central region of the image and to avoid distortion from non-square aspect ratios.

C HARDWARE SPECIFICATIONS

All benchmark results reported in Chapter 3 and Appendix A were collected on a single workstation.

C.1 COMPUTE HARDWARE

Table 77: Benchmark Workstation Configuration

Component	Specification
CPU	Intel (11th Gen Core i7-1165G7, 4 cores / 8 threads, 2.80 GHz base / 4.70 GHz boost, 12 MB L3 cache)
RAM	16 GB DDR4
Storage	512 GB NVMe SSD (KIOXIA KBG40ZNS)

All benchmarks were executed on CPU (no GPU acceleration). The Intel i7-1165G7 processor provides sufficient compute throughput for the evaluated models at interactive latencies: EfficientNet-B0 completes inference in 21.6 ms on average, while the transformer-based CLIP models complete in 84–106 ms. PyTorch executed in CPU mode with all model weights held in system memory. All inference timing measurements include preprocessing (resize, normalise) and postprocessing in the reported per-image latency.

C.2 SOFTWARE STACK

Table 78: Benchmark Software Environment

Component	Version and Details
Operating System	Linux (Ubuntu 24.04 LTS, kernel 6.8)
Python	3.12.x (CPython)
PyTorch	2.13.0 with CUDA 13.0 backend
TorchVision	0.28.0
Transformers (HuggingFace)	5.14.1
OpenCLIP	3.3.0
Fashion-CLIP	0.2.x
NumPy	2.5.1
CUDA Toolkit	13.0
cuDNN	8.9.x
NVIDIA Driver	580.173.02 (Linux)

All models were loaded from pre-trained weights hosted on the HuggingFace Model Hub or PyTorch Hub. Model weights were cached locally in `~/.cache/huggingface/` and `~/.cache/torch/` after the first download. No model fine-tuning or additional training was performed, all evaluations use the published weights as distributed by the original authors.

C.3 PRECISION AND DETERMINISM SETTINGS

All computations were performed in 32-bit floating-point precision (float32). Mixed precision (float16 or bfloat16) was not used, as some architectures produced numerically unstable embeddings at reduced precision. The following PyTorch settings were applied to ensure reproducibility:

- Random seed: 42 (Python `random`, NumPy, PyTorch CPU, PyTorch GPU)
- `torch.backends.cudnn.deterministic = true`
- `torch.backends.cudnn.benchmark = false`
- Model evaluation mode: `model.eval()` with `torch.no_grad()`

Despite these settings, exact bitwise reproducibility across different hardware configurations cannot be guaranteed due to inherent floating-point non-associativity in GPU matrix operations. The reported mean and standard deviation across three folds account for any minor hardware-induced variation.

C.4 REPRODUCIBILITY NOTES

Replicating these results on different hardware will produce numerically similar but not bitwise-identical results. The following factors contribute to cross-platform variation:

- **CPU architecture:** Inference latency is primarily determined by single-core performance and cache size, as the benchmark was executed on CPU without GPU acceleration. A processor with higher IPC (instructions per clock) or AVX-512 support would reduce inference times, particularly for the matrix operations in transformer models. The **relative ranking** of models (EfficientNet-B0 fastest, CLIP-based models slowest) should remain consistent across CPU architectures, though absolute values will vary.
- **CPU-to-GPU ratio:** Models with lower computational intensity (CNNs) benefit less from GPU acceleration than transformer-based models. On CPU-only systems, the latency gap between EfficientNet-B0 and FashionCLIP will narrow relative to GPU-accelerated deployments.
- **Memory capacity:** The 16 GB system memory of the development workstation exceeded the memory requirements of all individual models. Systems with less than 16 GB of system memory may experience swapping during concurrent model loading, particularly for the larger CLIP-based models which require over 600 MB each for model weights plus PyTorch runtime overhead.
- **Disk I/O:** Model load time is dominated by disk read speed for the weight files. An NVMe SSD (as used here) provides faster load times than a SATA SSD or HDD.

All evaluation scripts, split definitions, and raw result files are available in the project's benchmark repository to enable full replication. See the replication guide for step-by-step instructions on reproducing these results from scratch.

D DATABASE SCHEMA

This appendix documents the ReSys.Shop database schema extracted from the implementation codebase. The database uses PostgreSQL 17 with the pgvector extension, accessed via Entity Framework Core 10. Five migrations define the schema; entity configuration is managed through 33 `IEntityTypeConfiguration<T>` classes across eight bounded contexts.

Global conventions applied to all tables include UUID primary keys, automatic `created_at_utc` and `modified_at_utc` audit timestamps via EF Core save interceptors, soft deletion with `is_deleted` boolean and global query filters, and row version columns for optimistic concurrency control on contention-sensitive entities.

D.1 CATALOG SCHEMA

The Catalog schema contains 13 tables managing products, variants, images, taxonomy, and ML embeddings.

D.1.0.1 products

Table 79: Product entity. 1:N with variants, product_option_types, classifications (cascade).

No	Field name	Data type	Description
1	`id`	uuid	Primary key
2	`name`	varchar(255)	Required
3	`slug`	varchar(255)	Unique
4	`description`	varchar(2000)	Long-form description
5	`status`	text	Enum: Draft, Active, Archived
6	`style_code`	text	Internal fashion style code
7	`season_name`	text	Seasonal collection name
8	`material_composition`	text	Fabric composition description
9	`department`	text	Product department classification
10	`gender_target`	text	Target gender demographic
11	`master_variant_id`	uuid	FK to variants
12	`is_deleted`	bool	Soft delete

D.1.0.2 variants

Table 80: Variant entity. 1:N with prices, product_images, option_value_variants (cascade). FK to products.

No	Field name	Data type	Description
1	id	uuid	Primary key
2	sku	varchar(255)	Unique
3	is_master	bool	Default false
4	position	int	Display ordering position
5	barcode	text	Barcode identifier
6	weight	numeric(18,2)	Physical weight
7	price	numeric(18,2)	Unit price snapshot
8	cost_price	numeric(18,2)	Wholesale cost price
9	product_id	uuid	FK to products (cascade)
10	is_deleted	bool	Soft delete

D.1.0.3 variant_images

Table 81: Variant image entity. 1:N with image_embeddings (cascade).

No	Field name	Data type	Description
1	id	uuid	Primary key
2	url	varchar(2048)	Public image URL
3	storage_path	varchar(500)	Internal storage file path
4	alt	varchar(500)	Accessibility alt text
5	content_type	varchar(100)	Default image/jpeg
6	file_name	varchar(255)	Original uploaded filename
7	file_size	int	File size in bytes
8	width	int	Image width in pixels
9	height	int	Image height in pixels
10	position	int	Display ordering position
11	variant_id	uuid	FK to variants (cascade)

D.1.0.4 image_embeddings

Table 82: Image embedding entity. pgvector vector(512) column with IVFFlat cosine distance index.

No	Field name	Data type	Description
1	id	uuid	Primary key
2	model_name	varchar(100)	Required
3	model_version	varchar(50)	Embedding model version string

No	Field name	Data type	Description
4	vector	vector(512)	Nullable, IVFFlat index with vector_cosine_ops (lists=100)
5	dimensions	int	Actual embedding dimension count
6	status	text	Default Completed, tracks generation state
7	hangfire_job_id	text	Background job tracking
8	error	text	Error message for failed embeddings
9	variant_image_id	uuid	FK to variant_images (cascade)

D.1.0.5 option_types

Table 83: Option type entity. 1:N with option_values (cascade).

No	Field name	Data type	Description
1	id	uuid	Primary key
2	name	varchar(100)	Required
3	presentation	varchar(100)	Customer-facing display text
4	position	int	Default 1
5	filterable	bool	Default false
6	is_deleted	bool	Soft delete

D.1.0.6 option_values

Table 84: Option value entity. FK to option_types.

No	Field name	Data type	Description
1	id	uuid	Primary key
2	name	varchar(100)	Required
3	presentation	varchar(100)	Customer-facing display text
4	position	int	Default 1
5	option_type_id	uuid	FK to option_types (cascade)

D.1.0.7 product_option_types

Table 85: Product-option type join entity. Associative table between products and option_types.

No	Field name	Data type	Description
1	id	uuid	Primary key

No	Field name	Data type	Description
2	position	int	Display ordering position
3	product_id	uuid	FK to products, Unique (product_id, option_type_id)
4	option_type_id	uuid	FK to option_types, Unique (product_id, option_type_id)

D.1.0.8 option_value_variants

Table 86: Variant-option value join entity. Associative table between variants and option_values.

No	Field name	Data type	Description
1	id	uuid	Primary key
2	variant_id	uuid	FK to variants, Unique (variant_id, option_value_id)
3	option_value_id	uuid	FK to option_values, Unique (variant_id, option_value_id)

D.1.0.9 taxonomies

Table 87: Taxonomy entity. 1:N with taxa (cascade).

No	Field name	Data type	Description
1	id	uuid	Primary key
2	name	varchar(100)	Required
3	presentation	varchar(100)	Customer-facing display text
4	position	int	Display ordering position
5	is_deleted	bool	Soft delete

D.1.0.10 taxa

Table 88: Taxon entity. Nested set model for hierarchical classification. FK to taxonomies. Self-referencing FK for tree structure.

No	Field name	Data type	Description
1	id	uuid	Primary key
2	name	varchar(255)	Required
3	presentation	varchar(255)	Customer-facing display text
4	description	varchar(2000)	Long-form description

No	Field name	Data type	Description
5	position	int	Display ordering position
6	lft	int	Nested set left boundary
7	rgt	int	Nested set right boundary
8	depth	int	Tree depth
9	slug	varchar(255)	Unique (taxonomy_id, slug)
10	taxonomy_id	uuid	FK to taxonomies (cascade)
11	parent_id	uuid	Self-ref FK to taxa (restrict)
12	is_deleted	bool	Soft delete

D.1.0.11 taxon_rules

Table 89: Taxon rule entity. FK to taxa. Defines automatic classification rules.

No	Field name	Data type	Description
1	id	uuid	Primary key
2	taxon_id	uuid	FK to taxa (cascade)
3	type	varchar(50)	Rule type identifier
4	value	varchar(255)	Rule match value
5	match_policy	varchar(50)	Matching policy identifier

D.1.0.12 classifications

Table 90: Classification entity. Associative table between products and taxa. Each product-taxon pair is unique.

No	Field name	Data type	Description
1	id	uuid	Primary key
2	position	int	Display ordering position
3	is_automatic	bool	Whether auto-assigned by rules
4	product_id	uuid	FK to products, Unique (product_id, taxon_id)
5	taxon_id	uuid	FK to taxa, Unique (product_id, taxon_id)

D.1.0.13 prices

Table 91: Price entity. Time-bound pricing with currency specification. FK to variants.

No	Field name	Data type	Description
1	id	uuid	Primary key

No	Field name	Data type	Description
2	amount	numeric(18,2)	Default 0
3	compare_at_amount	numeric(18,2)	Original comparison price
4	currency	varchar(3)	Default USD
5	country_iso	varchar(2)	Country ISO code
6	is_default	bool	Whether default price
7	variant_id	uuid	FK to variants (cascade)
8	price_list_id	uuid	FK: parent price list

D.2 IDENTITY SCHEMA

The Identity schema extends ASP.NET Identity Core with JWT refresh tokens and passkey support. Entity configuration uses inherited IdentityDbContext conventions.

D.2.0.1 users

Table 92: User entity. Extends IdentityUser. 1:1 with user_profiles. 1:N with refresh_tokens, passkeys, wishlists.

No	Field name	Data type	Description
1	id	uuid	Primary key
2	user_name	text	Required
3	normalized_user_name	text	Unique, indexed
4	email	text	Contact email
5	normalized_email	text	Indexed
6	password_hash	text	Salted password hash
7	security_stamp	text	Security version stamp
8	first_name	text	Recipient given name
9	last_name	text	Recipient family name
10	is_active	bool	Default true
11	last_login_at_utc	timestamptz	Last login timestamp
12	sign_in_count	int	Total login count

D.2.0.2 refresh_tokens

Table 93: Refresh token entity. Single-use rotation with reuse detection. Self-referencing FK for token chain.

No	Field name	Data type	Description
1	id	uuid	Primary key
2	token_hash	varchar(500)	Required
3	user_id	uuid	FK to users
4	token_family_id	uuid	Indexed for family lookup
5	expires_at_utc	timestamptz	Required
6	revoked_at_utc	timestamptz	Token revocation timestamp
7	replaced_by_token_id	uuid	Self-ref FK to refresh_tokens
8	device_id	text	Client device identifier
9	user_agent	text	Client user agent string
10	ip_address	text	Client IP address

D.2.0.3 roles

Table 94: Role entity. Extends IdentityRole. N:M with users via user_roles.

No	Field name	Data type	Description
1	id	uuid	Primary key
2	name	text	Entity display name
3	normalized_name	text	Unique
4	description	text	Long-form description
5	is_system	bool	System-defined immutable role

D.2.0.4 user_roles

Table 95: User-role join entity. Associative table between users and roles.

No	Field name	Data type	Description
1	user_id	uuid	FK to users, Composite PK
2	role_id	uuid	FK to roles, Composite PK

D.2.0.5 user_claims

Table 96: User claim entity. Stores direct permission grants per user.

No	Field name	Data type	Description
1	id	int	Identity, Primary key
2	user_id	uuid	FK to users

No	Field name	Data type	Description
3	claim_type	text	Permission claim type
4	claim_value	text	Permission claim value

D.2.0.6 user_logins

Table 97: User login entity. Stores external login provider links (Google OAuth).

No	Field name	Data type	Description
1	login_provider	text	Composite PK
2	provider_key	text	Composite PK
3	user_id	uuid	FK to users
4	provider_display_name	text	Provider display name

D.2.0.7 user_tokens

Table 98: User token entity. Stores password reset tokens and email confirmation tokens.

No	Field name	Data type	Description
1	user_id	uuid	FK to users, Composite PK
2	login_provider	text	Composite PK
3	name	text	Composite PK
4	value	text	Rule match value

D.2.0.8 role_claims

Table 99: Role claim entity. Stores permission claims inherited by role members.

No	Field name	Data type	Description
1	id	int	Identity, Primary key
2	role_id	uuid	FK to roles
3	claim_type	text	Permission claim type
4	claim_value	text	Permission claim value

D.2.0.9 passkeys

Table 100: Passkey entity. Supports FIDO2/WebAuthn passwordless authentication.

No	Field name	Data type	Description
1	id	uuid	Primary key
2	user_id	uuid	FK to users
3	credential_id	bytea	Required

D.3 ORDERING SCHEMA

The Ordering schema manages orders, line items, and adjustments. The cart is represented by orders with status `Draft`, linked to anonymous users via `session_id`.

D.3.0.1 orders

Table 101: Order entity. Forward-only checkout state machine (Address through Complete). Indexed on (user_id, status) and (session_id, status). 1:N with line_items, adjustments (cascade).

No	Field name	Data type	Description
1	id	uuid	Primary key
2	number	varchar(50)	Unique
3	session_id	varchar(100)	Indexed
4	status	text	Enum state: Draft through Completed
5	checkout_state	text	Enum: Address, Delivery, Payment, Confirm, Complete
6	currency	varchar(3)	Default USD
7	item_total	numeric(18,2)	Sum of line item totals
8	adjustment_total	numeric(18,2)	Line-level adjustment sum
9	shipment_total	numeric(18,2)	Shipping cost total
10	total	numeric(18,2)	Monetary balance: item + adjustment + shipment
11	payment_total	numeric(18,2)	Total captured payments
12	outstanding_balance	numeric(18,2)	Unpaid remaining balance
13	email	varchar(255)	Contact email
14	completed_at_utc	timestamptz	Order completion timestamp
15	canceled_at_utc	timestamptz	Order cancellation timestamp
16	user_id	uuid	Indexed (user_id, status)
17	shipping_method_id	uuid	FK: parent method
18	is_deleted	bool	Soft delete

D.3.0.2 line_items

Table 102: Line item entity. Price snapshots protect historical orders from catalog updates.

No	Field name	Data type	Description
1	id	uuid	Primary key
2	quantity	int	Default 1
3	price	numeric(18,2)	Snapshot at purchase time
4	total	numeric(18,2)	Line item total
5	adjustment_total	numeric(18,2)	Line-level adjustment sum
6	currency	varchar(3)	Default USD
7	order_id	uuid	FK to orders (cascade)
8	variant_id	uuid	FK to variants (no action)

D.3.0.3 adjustments

Table 103: Adjustment entity. Polymorphic adjustments (tax, shipping fee, discounts). FK to orders.

No	Field name	Data type	Description
1	id	uuid	Primary key
2	label	varchar(255)	User-defined address label
3	amount	numeric(18,2)	Sale price amount
4	eligible	bool	Default true
5	included	bool	Default false
6	mandatory	bool	Default false
7	state	varchar(50)	Default open
8	adjustable_id	uuid	Polymorphic: target entity ID
9	adjustable_type	varchar(100)	Polymorphic: target entity type
10	source_id	uuid	Stripe source/payment method ID
11	source_type	varchar(100)	Polymorphic: source entity type
12	order_id	uuid	FK to orders (cascade)

D.4 PAYMENT SCHEMA

D.4.0.1 payment_methods

Table 104: Payment method entity. Preferences stored as jsonb. Settings encrypted. 1:N with payment_captures (set null).

No	Field name	Data type	Description
1	id	uuid	Primary key
2	name	varchar(255)	Required
3	code	varchar(50)	Unique method code
4	description	varchar(1000)	Long-form description
5	provider_key	varchar(50)	Gateway provider identifier
6	active	bool	Default true
7	auto_capture	bool	Default false
8	display_on	text	Enum: FrontEnd, BackEnd, Both
9	position	int	Default 0
10	preferences	jsonb	Gateway-specific configuration
11	settings	text	Encrypted credentials
12	webhook_enabled	bool	Default false
13	is_deleted	bool	Soft delete

D.4.0.2 payment_captures

Table 105: Payment capture entity. Uses xmin for optimistic concurrency. processed_stripe_event_ids stored as jsonb for webhook idempotency.

No	Field name	Data type	Description
1	id	uuid	Primary key
2	number	varchar(50)	Human-readable reference
3	amount	numeric(18,2)	Required
4	currency	varchar(3)	Default USD
5	state	text	Enum: Checkout through Refunded
6	response_code	varchar(255)	Indexed for non-null values
7	intent_client_secret	varchar(500)	Stripe intent client secret
8	refunded_amount	numeric(18,2)	Total refunded amount
9	provider_key	varchar(50)	Gateway provider identifier
10	processed_stripe_event_ids	jsonb	Idempotency tracking
11	order_id	uuid	FK to orders (cascade)
12	payment_method_id	uuid	FK to payment_methods (set null)
13	source_id	varchar(200)	Stripe source/payment method ID

D.5 INVENTORY SCHEMA

D.5.0.1 stock_locations

Table 106: Stock location entity. 1:N with stock_items, stock_movements (cascade).

No	Field name	Data type	Description
1	id	uuid	Primary key
2	name	varchar(255)	Required
3	code	varchar(50)	Unique method code
4	address1	varchar(255)	Street address line 1
5	city	varchar(100)	City name
6	postal_code	varchar(10)	Postal code
7	phone	varchar(50)	Contact phone
8	country_id	uuid	FK: parent country
9	state_id	uuid	FK: state reference
10	active	bool	Default true
11	default	bool	Default false
12	low_stock_threshold	int	Default 5
13	notify_on_low_stock	bool	Default false
14	is_deleted	bool	Soft delete

D.5.0.2 stock_items

Table 107: Stock item entity. Tracks quantity per variant per location. Uses xmin for optimistic concurrency. 1:N with stock_movements (cascade).

No	Field name	Data type	Description
1	id	uuid	Primary key
2	count_on_hand	int	Default 0
3	backorderable	bool	Default false
4	stock_location_id	uuid	FK to stock_locations (cascade), Unique (location, variant)
5	variant_id	uuid	Unique (location, variant)

D.5.0.3 stock_movements

Table 108: Stock movement entity. Immutable audit trail recording quantity deltas and balance snapshots.

No	Field name	Data type	Description
1	id	uuid	Primary key
2	quantity	int	Required
3	previous_count_on_hand	int	Value before adjustment
4	action	varchar(50)	Movement type identifier
5	stock_item_id	uuid	FK to stock_items (cascade)
6	stock_location_id	uuid	FK to stock_locations (cascade)
7	originator_id	uuid	Operator identity
8	originator_type	varchar(200)	Operator entity type
9	reason	varchar(500)	Reason for adjustment

D.5.0.4 stock_reservations

Table 109: Stock reservation entity. Temporary holds during checkout. Uses xmin for optimistic concurrency. Expires after configurable timeout (default 15 min).

No	Field name	Data type	Description
1	id	uuid	Primary key
2	variant_id	uuid	Required
3	stock_location_id	uuid	FK: reporting location
4	order_id	uuid	Indexed (order_id, state)
5	quantity	int	Required
6	state	text	Indexed
7	expires_at_utc	timestamptz	Auto-release after timeout
8	cart_token	text	Indexed (cart_token, state)

D.5.0.5 stock_transfers

Table 110: Stock transfer entity. Manages inter-location stock movement with state lifecycle (Created, In-Transit, Received, Cancelled). Uses xmin.

No	Field name	Data type	Description
1	id	uuid	Primary key
2	number	varchar(50)	Human-readable transfer reference
3	reference	varchar(255)	External reference note
4	state	varchar(20)	Indexed
5	source_location_id	uuid	FK to stock_locations (restrict), Indexed

No	Field name	Data type	Description
6	destination_location_id	uuid	FK to stock_locations (restrict), Indexed

D.5.0.6 transfer_items

Table 111: Transfer item entity. Individual line items within a stock transfer. FK to stock_transfers.

No	Field name	Data type	Description
1	id	uuid	Primary key
2	stock_transfer_id	uuid	FK to stock_transfers (cascade)
3	variant_id	uuid	Required
4	quantity	int	Required
5	received_quantity	int	Default 0

D.6 SHIPPING SCHEMA

D.6.0.1 shipping_methods

Table 112: Shipping method entity. 1:N with shipping_method_zones (cascade).

No	Field name	Data type	Description
1	id	uuid	Primary key
2	name	varchar(255)	Required
3	code	varchar(50)	Unique
4	tracking_url	varchar(2048)	Carrier tracking URL template
5	admin_name	varchar(255)	Admin-facing display name
6	position	int	Default 0
7	available_to_users	bool	Default true
8	calculator_type	varchar(100)	Rate calculation algorithm
9	presentation	text	Customer-facing display text
10	is_deleted	bool	Soft delete

D.6.0.2 shipping_rates

Table 113: Shipping rate entity. Weight- and value-based tiered pricing per method.

No	Field name	Data type	Description
1	id	uuid	Primary key
2	name	varchar(255)	Entity display name
3	selected	bool	Default false
4	cost	numeric(18,2)	Base shipping cost
5	final_price	numeric(18,2)	Calculated final price
6	display_price	varchar(50)	Formatted price string
7	delivery_range	varchar(100)	Estimated delivery timeframe
8	min_weight	numeric(18,2)	Minimum weight threshold
9	max_weight	numeric(18,2)	Maximum weight threshold
10	free_shipping_threshold	numeric(18,2)	Free shipping order value threshold
11	shipping_method_id	uuid	FK to shipping_methods

D.6.0.3 shipping_method_zones

Table 114: Shipping method zone entity. Geographic zone restriction per method. Composite index on (shipping_method_id, country_code, state_code).

No	Field name	Data type	Description
1	id	uuid	Primary key
2	shipping_method_id	uuid	FK to shipping_methods (cascade)
3	country_code	varchar(2)	Composite index (method, country, state)
4	state_code	varchar(10)	Composite index

D.7 PROFILE SCHEMA

D.7.0.1 user_profiles

Table 115: User profile entity. 1:1 with users via shared primary key. Includes preferences as embedded value objects (style, fit, colors, sizes). 1:N with addresses (cascade).

No	Field name	Data type	Description
1	user_id	uuid	Primary key, FK to users (cascade, 1:1)
2	first_name	varchar(100)	Recipient given name
3	last_name	varchar(100)	Recipient family name
4	email	varchar(255)	Contact email
5	phone_number	varchar(20)	Contact phone number

No	Field name	Data type	Description
6	date_of_birth	date	Date of birth
7	gender	varchar(20)	Gender identity
8	avatar_url	varchar(500)	Profile avatar image URL
9	notifications_enable_email	bool	Email notifications flag
10	notifications_enable_sms	bool	SMS notifications flag
11	accepts_email_marketing	bool	Default false
12	default_billing_address_id	uuid	FK to addresses (set null)
13	default_shipping_address_id	uuid	FK to addresses (set null)
14	orders_count	int	Default 0
15	total_spent	numeric(18,2)	Default 0

D.7.0.2 addresses

Table 116: Address entity. Supports shipping and billing types with default designation.

No	Field name	Data type	Description
1	id	uuid	Primary key
2	address_type	text	Enum: Shipping, Billing
3	first_name	varchar(100)	Recipient given name
4	last_name	varchar(100)	Recipient family name
5	address1	varchar(200)	Street address line 1
6	address2	varchar(200)	Street address line 2
7	city	varchar(100)	City name
8	zip_code	varchar(20)	Postal code
9	phone	varchar(20)	Contact phone
10	label	varchar(50)	User-defined address label
11	is_default	bool	Default false
12	country_name	varchar(100)	Country display name
13	state_province	varchar(100)	State/province name
14	country_code	varchar(3)	Indexed
15	state_code	varchar(10)	State/province code
16	user_profile_id	uuid	FK to user_profiles (cascade), Indexed (address_type, user_profile_id)

D.7.0.3 wishlists

Table 117: Wishlist entity. FK to users. 1:N with wished_items (cascade). Indexed by (user_id, is_default).

No	Field name	Data type	Description
1	id	uuid	Primary key
2	name	varchar(200)	Required
3	token	varchar(128)	Unique
4	is_default	bool	Default false
5	is_private	bool	Default false
6	user_id	uuid	FK to users (cascade), Indexed (user_id, is_default)
7	is_deleted	bool	Soft delete

D.7.0.4 wished_items

Table 118: Wished item entity. Each variant appears at most once per wishlist.

No	Field name	Data type	Description
1	id	uuid	Primary key
2	quantity	int	Default 1
3	variant_id	uuid	Required
4	wishlist_id	uuid	FK to wishlists (cascade), Unique (wishlist_id, variant_id)

D.8 LOCATION SCHEMA

D.8.0.1 countries

Table 119: Country entity. ISO 3166-1 reference data. 1:N with states (cascade).

No	Field name	Data type	Description
1	id	uuid	Primary key
2	name	varchar(100)	Required
3	iso_code	varchar(3)	ISO 3166-1 alpha-2
4	iso3code	text	ISO 3166-1 alpha-3
5	calling_code	varchar(10)	Country calling code
6	states_required	bool	Default false
7	zipcode_required	bool	Default false
8	is_active	bool	Default true

D.8.0.2 states

Table 120: State entity. ISO 3166-2 reference data. FK to countries.

No	Field name	Data type	Description
1	`id`	uuid	Primary key
2	`name`	varchar(100)	Required
3	`abbreviation`	varchar(10)	State abbreviation
4	`country_id`	uuid	FK to countries (cascade)
5	`is_active`	bool	Default true

D.9 GLOBAL CONVENTIONS

All schemas share common conventions applied via EF Core configuration. UUID primary keys prevent sequential enumeration and enable safe client-side key generation. Audit columns (`created_at_utc`, `modified_at_utc`, `created_by`, `modified_by`) populate automatically via save interceptors on entities implementing `IAuditable`. Soft deletion uses `is_deleted` and `deleted_at_utc` with global query filters on entities implementing `ISoftDeletable`. Row versioning via `xmin` PostgreSQL system column provides optimistic concurrency control on `stock_items`, `stock_reservations`, `stock_transfers`, and `payment_captures`. Enum types are stored as text for portability. Decimal columns use precision (18, 2) for monetary values. DateTime columns use `timestamp with time zone`.

D.10 PGVECTOR INTEGRATION

Vector embeddings are stored in `catalog.image_embeddings` with column type `vector(512)`, made nullable in migration 20260804013350. An IVFFlat index with cosine distance operator (`vector_cosine_ops`) and `lists = 100` enables approximate nearest neighbour search. The embedding column is provider-adaptive: on PostgreSQL/Npgsql, it maps directly to the native `vector` type; on other providers, embeddings serialize as JSON strings via a value converter.

Search queries use the `<=>` cosine distance operator, ranking results by similarity score `1 - cosine_distance`. Every embedding record includes `model_name` and `model_version` columns, enabling strict model-level filtering during search to ensure mathematical alignment between query and gallery embeddings.